



XTyphonTC: nube táctica basada en serverless computing para creación y difusión de malware

Por

David Castro García - Nota: 10

Ricardo Luque Mansilla - Nota: 10



**UNIVERSIDAD COMPLUTENSE
MADRID**

Dirigido por

José Luis Vázquez Poletti

David Pacios Izquierdo

MADRID, 2025–2026

Abstract / Resumen

Serverless computing has transformed the development of cloud applications, but it has also enabled new mechanisms for code fragmentation, obfuscation, and malicious delivery.

This work presents xTyphonTC, a proof of concept that distributes the stages of a memory-injection payload across several AWS Lambda functions. The proposed architecture relies on a local *Loader* that requests the obfuscated fragments, reconstructs the final logic in memory, and executes it without persisting the complete artifact on disk.

The proposal has been evaluated in a controlled academic environment. The results confirm the technical feasibility of the architecture and show its ability to evade several local controls under the tested conditions, while also exposing the limitations of static-signature and purely behavioral approaches when faced with fragmented delivery over legitimate cloud services.

El paradigma *serverless* ha transformado el desarrollo de aplicaciones en la nube, pero también ha abierto nuevas posibilidades para la fragmentación, la ofuscación y la entrega maliciosa de código.

Este trabajo presenta xTyphonTC, una prueba de concepto que distribuye las fases de un *payload* de inyección en memoria entre varias funciones AWS Lambda. La arquitectura propuesta se apoya en un *Loader* local que solicita los fragmentos ofuscados, reconstruye la lógica final en memoria y la ejecuta sin persistir el artefacto completo en disco.

La propuesta se ha evaluado en un entorno académico controlado. Los resultados confirman la viabilidad técnica de la arquitectura y muestran su capacidad para evadir varios controles locales en las condiciones ensayadas, al tiempo que ponen de relieve las limitaciones de los enfoques basados exclusivamente en firmas estáticas o en comportamiento cuando se enfrentan a mecanismos de entrega fragmentada sobre servicios legítimos de nube.

Palabras Clave: Serverless, Cybersecurity, Malware Evasion, Fragmentation, Obfuscation, AWS Lambda.

Índice general

Abstract	III
Capítulo 1 Introduction	1
Capítulo 1 Introducción	5
Capítulo 2 Estado del Arte	9
Capítulo 3 Tecnologías	35
Capítulo 4 Diseño de la Solución	43
Capítulo 5 Desarrollo y Arquitectura	51
Capítulo 6 Resultados, Conclusiones y Trabajo Futuro	71
Capítulo 6 Conclusions and future work	91
Contribuciones de los autores	95
Bibliografía	100

Agradecimientos

*A toda la gente que me ha acompañado estos años.
A la Universidad Complutense y a la Facultad de Informática, por haberme
facilitado los mejores medios para convertirme en un profesional.
A mi núcleo de amigos, por siempre estar para apoyarme.
A mi familia y mi gata Kira, por haberme enseñado a ser el hombre que soy hoy
en día.
A mis tutores, por haberme contagiado su gran pasión por la ciberseguridad.
Y en especial, a Ricardo, por ser el mejor amigo y compañero que se puede
tener.
David.*

*En primer lugar, A mi familia por ser el motor de mi existencia.
A mis amigos, por ser mi refugio durante todos estos años.
A mis profes, por ayudarme a comprender y enseñarme los conceptos tan
abstractos de la universidad.
A mis tutores, por haberme contagiado su gran pasión por la ciberseguridad y
por guiarnos en este reto.
Al rayito, por enseñarme que con coraje y valentía se puede contra cualquier
gigante. Estamos en la final de la Conference League. ¡Vallecas Forever!
Y, por supuesto, a mi compañero y amigo David, por formar parte del equipo y
por su apoyo constante en este trabajo
Ricardo.*

Chapter 1. Introduction

In today's digital society, cybersecurity has become a fundamental pillar. The constant emergence of new technologies requires a proactive perspective and a clear understanding that each technological advance also introduces new challenges and threats.

Within this context, one of the paradigms that has expanded most rapidly in recent years is serverless computing, which enables the distributed and ephemeral execution of functions in the cloud. Although this model offers significant advantages, such as improved development efficiency and elasticity, it also complicates the application of traditional security controls and therefore demands new mechanisms for prevention, detection, and pattern analysis.

1.1. Motivation

In the field of cybersecurity, two competing agents are identified that feed back into each other, forcing continuous improvement: attackers and defenders.

The attackers are responsible for studying and exploiting existing vulnerabilities in systems with specific aims that, to some extent, harm the targeted user. The malware employed by these agents is not static but evolves as defenders take action or when new technologies emerge.

On the other hand, defenders have the responsibility to respond to the attackers and offer solutions so that malware, that has already acted, does not cause a significant risk again.

The constant need for innovation inherent to attackers, with the goal of overcoming existing defensive actions, grants them significant temporary and technical advantages. Thus, the obligation for defenders to stay ahead of events has led to the thought that, to prevent attackers from acting, one might have to think like one of them, simulating possible attack scenarios in systems before they can materialize for the first time.

Consequently, the emergence of serverless technologies represents a new front within a persistent asymmetric conflict. It creates a scenario in which some previously established studies and protection mechanisms may prove insufficient. For that reason, it is necessary to identify as early as possible which areas of these architectures may become attack surfaces

and which defensive measures should be developed to mitigate those risks.

Within this academic and ethical framework, xTyphonTC is presented as an educational laboratory that simulates the fragmented delivery of malware components through serverless services in a controlled environment. Its purpose is to analyze, from an offensive perspective oriented toward defense, the evasion, fragmentation, and covert-communication techniques that could be exploited in these environments in order to improve their detection and mitigation.

1.2. Objectives

The primary objective of this work is to develop an experimental laboratory for the study of threat delivery in distributed computing environments. To achieve this goal, the following specific objectives are defined:

- Conduct a literature review on the evolution of malware and relevant research in serverless security.
- Select and describe the tools and technologies (Python, AWS Lambda, VirtualBox) that are part of the laboratory.
- Design and implement the architecture of the xTyphonTC system, including the victim's environment and the attack infrastructure based on microservices.
- Evaluate the effectiveness of component fragmentation as an evasion technique for traditional and modern detection systems.
- Study the operational costs, availability, and stealth of secret communication channels through legitimate services.

1.3. Justification and Scope

This Final Year Project is justified by the need to update defensive methodologies in response to the migration of threats toward hybrid and serverless-based infrastructures. From that standpoint, the study offers a practical approach for analyzing these techniques in a controlled manner and for obtaining evidence about the viability of fragmented attacks built on serverless computing.

The scope of the project covers the conceptual design of the architecture, its implementation in the cloud, and the technical evaluation of the

proposed evasion mechanism. In doing so, it aims to provide a useful foundation for future systems devoted to monitoring and responding to incidents in distributed computing environments.

1.4. Structure of the Thesis

This document is structured into seven chapters detailing the project's life cycle:

- **Chapter 1:** Introduces the motivation, objectives, and justification of the work.
- **Chapter 2:** Presents the state of the art and theoretical foundations about malware and serverless technologies.
- **Chapter 3:** Describes the tools and methodologies selected for implementation.
- **Chapter 4:** Details the conceptual design of the xTyphonTC solution.
- **Chapter 5:** Details the implementation and technical architecture of the xTyphonTC solution.
- **Chapter 6:** Analyzes the results obtained in the proofs of concept, as well as final conclusions and lines of future work.

Capítulo 1. Introducción

En la sociedad digital contemporánea, la ciberseguridad se ha consolidado como un pilar esencial. La aparición constante de nuevas tecnologías obliga a adoptar una postura proactiva y a reconocer que cada avance incorpora, al mismo tiempo, nuevas superficies de exposición y nuevos vectores de amenaza.

En este contexto, uno de los paradigmas con mayor crecimiento en los últimos años ha sido la computación *serverless*, que permite ejecutar funciones en la nube de forma distribuida y efímera. Aunque este modelo aporta ventajas claras en términos de eficiencia y escalabilidad, también dificulta la aplicación de controles de seguridad tradicionales y exige mecanismos de prevención, detección y análisis más adaptados a entornos distribuidos.

1.1. Motivación

En el ámbito de la ciberseguridad se identifican dos agentes que compiten entre sí y que se retroalimentan obligándose a una mejora continua: atacantes y defensores.

Los primeros se encargan de estudiar y explotar vulnerabilidades existentes en los sistemas con unos determinados fines que dañan en cierta medida al usuario atacado. El *malware* que emplea estos agentes no es estático, sino que evoluciona a medida que los defensores actúan o cuando aparecen nuevas tecnologías.

Por otro lado, los defensores tienen la responsabilidad de dar respuesta a los anteriores y ofrecer soluciones para que, aquel *malware* que ya ha actuado, no vuelva a suponer un riesgo significativo.

La necesidad constante de innovar propia de los atacantes, con el objetivo de superar las medidas defensivas existentes, les otorga una ventaja temporal y técnica significativas. Así, la obligación de adelantarse a los acontecimientos por parte de los defensores ha llevado a la reflexión de que para prevenir que los atacantes actúen, quizá haya que pensar como uno de ellos, simulando posibles escenarios de ataque en los sistemas antes de que puedan materializarse por primera vez.

En consecuencia, la aparición de las tecnologías *serverless* introduce un nuevo frente dentro del conflicto asimétrico entre atacantes y defensores.

Se trata de un escenario en el que parte de los estudios y de las soluciones previamente consolidadas pueden resultar insuficientes. Por ello, conviene identificar con rapidez qué áreas de estas arquitecturas pueden convertirse en puntos de ataque y qué mecanismos de protección deben desarrollarse para mitigar dichos riesgos.

En este marco, y exclusivamente con fines académicos y éticos, se sitúa xTyphonTC. La propuesta consiste en un laboratorio controlado que simula la entrega fragmentada de componentes de *malware* mediante servicios *serverless*. Su finalidad es analizar, desde una perspectiva ofensiva las técnicas de evasión, fragmentación y comunicación encubierta que podrían explotarse en este tipo de entornos, con el objetivo de mejorar su detección y mitigación.

1.2. Objetivos

El objetivo principal de este trabajo es desarrollar un laboratorio experimental para el estudio de la entrega de amenazas en entornos de computación distribuida. Para alcanzar este fin, se definen los siguientes objetivos específicos:

- Realizar una revisión de la literatura sobre la evolución del *malware* y las investigaciones relevantes en seguridad *serverless*.
- Seleccionar y describir las herramientas y tecnologías (Python, AWS Lambda, VirtualBox) que componen el laboratorio.
- Diseñar e implementar la arquitectura del sistema xTyphonTC, incluyendo el entorno de la víctima y la infraestructura de ataque basada en microservicios.
- Evaluar la eficacia de la fragmentación de componentes como técnica de evasión para sistemas de detección tradicionales y modernos.
- Estudiar los costes operativos, la disponibilidad y el sigilo de los canales de comunicación encubierta mediante servicios legítimos.

1.3. Justificación y Alcance

La realización de este Trabajo de Fin de Grado se justifica por la necesidad de actualizar las metodologías defensivas ante el desplazamiento de las amenazas hacia infraestructuras híbridas y entornos *serverless*. Desde esta perspectiva, el estudio aporta un enfoque práctico que permite analizar estas técnicas en condiciones seguras

y extraer evidencias sobre la viabilidad de los ataques fragmentados apoyados en servicios de nube.

El alcance del proyecto abarca el diseño conceptual de la arquitectura, su implementación en AWS y la evaluación técnica del mecanismo de evasión propuesto. Con ello, se pretende sentar una base útil para el desarrollo futuro de sistemas de monitorización y respuesta adaptados a entornos de computación distribuida.

1.4. Estructura de la memoria

El presente documento se estructura en siete capítulos que detallan el ciclo de vida del proyecto:

- **Capítulo 1:** Introduce la motivación, los objetivos y la justificación del trabajo.
- **Capítulo 2:** Presenta el estado del arte y los fundamentos teóricos sobre *malware* y tecnologías *serverless*.
- **Capítulo 3:** Describe las herramientas y metodologías seleccionadas para la implementación.
- **Capítulo 4:** Detalla el diseño conceptual de la solución xTyphonTC.
- **Capítulo 5:** Detalla la implementación y la arquitectura técnica de la solución xTyphonTC.
- **Capítulo 6:** Analiza los resultados obtenidos en las pruebas de concepto, así como las conclusiones finales y las líneas de trabajo futuro.

Capítulo 2. Estado del Arte

En este capítulo se contextualiza el marco técnico y teórico en el que se sitúa este trabajo a partir de la evolución del *malware*, de los paradigmas de detección y de los nuevos escenarios que surgen en los entornos *serverless*. El objetivo es construir una línea argumentativa que pueda ayudar a entender cómo la fragmentación deliberada del código, las técnicas de evasión y la arquitectura distribuida en la nube convergen para dar lugar a un nuevo tipo de amenaza: el *malware* «nativo de *serverless*».

2.1. El Panorama Evolutivo de las Amenazas del *malware*

En esta sección se presenta una visión general del concepto de *malware* y su evolución, basándose fundamentalmente en los trabajos de Tahir (2018), Aslan y Samet (2020) y Namanya et al. (2018), que examinan no solo la taxonomía tradicional, sino también las tácticas actuales de evasión [1, 2, 3]

En Tahir (2018) [1] se establece una base teórica de las técnicas de camuflaje, desde cifrado simple hasta el metaformismo. Su importancia radica en que permite justificar por qué las firmas estáticas ya no son suficientes.

Por otro lado Aslan y Samet (2020) [2] define con precisión las diferencias entre el *malware* tradicional y el de nueva generación. Para este proyecto, es vital su análisis sobre cómo las amenazas modernas utilizan múltiples técnicas de ocultación para persistir en los sistemas, lo cual valida la necesidad de mecanismos de detección más complejos.

Finalmente, Namanya et al. (2018) [3] aporta una visión del análisis moderno, destacando que la supervivencia del *malware* define su éxito. Es importante porque clasifica técnicas de anti-análisis y ofuscación, demostrando como las amenazas intentan engañar tanto a sistemas automáticos como a analistas humanos.

Con el fin de comprender el «arma» que emplean los atacantes en los sistemas informáticos, se ofrece unas pinceladas básicas sobre el *malware*, su clasificación y sus mecanismos de ofuscación y evasión, para establecer así una base sólida sobre una de las amenazas más peligrosas de nuestro tiempo.

2.1.1. ¿Qué es un *malware*?

El término *malware* (abreviatura de *malicious software*) hace referencia a cualquier tipo de programa destinado a dañar sistemas informáticos, interrumpir su funcionamiento o robar información. Con el paso de los años, este concepto ha evolucionado desde virus o gusanos informático con códigos estáticos, hasta amenazas modulares desarrolladas que pueden adaptarse de forma dinámica al entorno, como se puede observar en la tabla 2.1

Parámetro de Comparación	Tradicional	Nueva Generación
Nivel de implementación	código simple	código complejo
Estado de los comportamientos	estático	dinámico
Proliferación	cada copia es similar	cada copia es diferente
Propagación	usa extensión .exe	usa también diferentes extensiones
Permanencia en el sistema	temporal	persistente
Interacción con procesos	pocos procesos	múltiples procesos
Uso de técnicas de ocultación	ninguna	sí
Tipo de ataque	general	dirigido
Desafío defensivo	fácil	difícil
Dispositivos objetivo	ordenadores generales	muchos dispositivos diferentes

Tabla 2.1: Comparativa entre *malware* tradicional y *malware* de nueva generación, adaptada de Aslan y Samet (2020) [2]

El *malware* tiene varias características básicas: replicación, métodos de ocultamiento, activación, manifestación, explotación y escalado de privilegios.

2.1.2. Taxonomía del *Malware*

La taxonomía es la ciencia que permite definir, describir y clasificar entidades en grupos basados en rasgos comunes. Por tanto, una vez que se sabe qué es un *malware*, se ha de poner énfasis en esta taxonomía basada en características bastante marcadas:

- **Virus:** programa informático diseñado para infectar archivos, normalmente ejecutables, y realizar acciones maliciosas. Puede ir dentro del código de otros programas. Ejemplo: Brain (1986), el primer

virus para PC, que ocultaba su presencia en disco para evitar ser detectado.

- **Gusano:** es un tipo de virus con la particularidad de que puede replicarse a sí mismo en otros sistemas informáticos. Su principal objetivo suele ser propagarse al mayor número de dispositivos posibles. Ejemplo: Morris Worm (1988), uno de los primeros en propagarse por Internet, o Stuxnet (2010), diseñado específicamente para atacar plantas nucleares en Irán.
- **Troyano:** programa malicioso, camuflado normalmente como *software* legítimo, que permite acceso remoto al sistema infectado y realiza acciones no autorizadas por el usuario (robo de información, instalación de programas, etc). Ejemplo: Zeus (2007), utilizado para robar información bancaria y crear infraestructuras de ataque para otros *malware* como CryptoLocker.
- **Rootkits:** conjunto de herramientas que encubre objetos o actividades del sistema cuyo objetivo es adquirir privilegios y/o ocultar ficheros y programas.
- **Ransomware:** virus informático que cifra los archivos de sus víctimas y pide un rescate para liberarlos (*malware* secuestrador). Ejemplo: WannaCryptor (2017), que afectó a más de 300.000 máquinas en 150 países.
- **Bombas lógicas:** parte de código insertada intencionalmente en un programa informático que permanece oculto hasta cumplirse una o más condiciones. En ese momento se ejecuta una acción maliciosa (borrar o cifrar información, apagar el sistema, etcétera).
- **Botnets:** conjunto de equipos infectados (*bots*) que realizan acciones maliciosas, normalmente un ataque DDoS ¹, de manera autónoma y automática, el dueño de la *botnet* tiene el control de todos los sistemas informáticos infectados de la red. Ejemplo: Mirai (2016), la primera gran *botnet* que infectó dispositivos del Internet de las Cosas (IoT) ² para lanzar ataques DDoS masivos.

¹Un DDoS (Distributed Denial of Service) o ataque de denegación de servicio distribuido es un ataque a un sistema informático que causa la interrupción de un servicio debido al agotamiento de recursos de los componentes de la red. Es distribuido cuando lo realizan varias máquinas.

²El Internet de las Cosas (IoT) son objetos físicos equipados con sensores, software y tecnologías que se conectan e intercambian datos a través de Internet. Un ejemplo podrían ser los relojes inteligentes o los robots aspiradora.

- **Adware:** programa diseñado para reproducir publicidad, redirigir búsquedas a sitios web publicitarios y recopilar información de marketing para crear un perfil de gustos y que la publicidad sea personalizada.
- **Pornware:** programa que reproduce material pornográfico y que, sin consentimiento del usuario, patrocina servicios y sitios web de contenido pornográfico sujetos a suscripción.
- **Riskware:** programa legítimo que puede provocar daño si es utilizado por ciberdelincuentes, como por ejemplo, herramientas de conexión remota, de gestión de contraseñas, de escaneo de puertos, etcétera.
- **Spyware:** programa espía que, tras su transmisión, recopila información de un sistema informático sin el consentimiento del propietario.

2.1.3. Técnicas de Ofuscación y Evasión

Los *malware* utilizan diferentes técnicas para evadir y ofuscar a los métodos de detección y así dificultar el análisis por parte de quienes defienden los sistemas informáticos. En esta sección se analizan los distintos procedimientos que usan los *malware* para sobrevivir en los entornos y poder alcanzar sus objetivos de forma exitosa, evitando ser detectados por la seguridad del sistema como antivirus³, cortafuegos⁴, sistemas de detección de intrusos (IDS⁵).

Según el trabajo de Singh et al. (2018) [4], la utilización sistemática de técnicas de ofuscación, que dificultan el análisis dinámico y estático, es la razón por la que el *malware* moderno se vuelve cada vez más sofisticado. Esto complica el trabajo de los analistas permitiendo a los atacantes crear variantes cada vez más difíciles de detectar.

³Un Antivirus es un programa informático cuyo objetivo es detectar y eliminar *malware* de un computador. Tradicionalmente son basados en firmas, es decir, utilizan una base de datos donde guardan la firma de un *malware* conocido y lo comparan con la de otros programas. Sin embargo, han aparecido nuevos tipos de antivirus como los basados en detección heurística, en detección de comportamientos, en detección por IA, etcétera.

⁴Un cortafuegos (*firewall*) impide el acceso de tráfico no autorizado a una red o sistema. Ofrece seguridad perimetral a una red local contra ataques provenientes de Internet y actúa como un punto central para implementar medidas de seguridad y auditoría.

⁵Los IDS (Sistemas de detección de intrusos) son sistemas que detectan un uso no autorizado de un computador, una red o una infraestructura de comunicaciones. Cuando lo detectan disparan una alarma y toman medidas adicionales como mandar un correo electrónico al administrador, cambiar reglas en el cortafuegos o cerrar una conexión.

Haciendo una revisión de la literatura sobre *malware*, se ha encontrado diferentes técnicas de evasión y ofuscación. En este trabajo, dichas técnicas se han agrupado en técnicas de anti-desensamblaje, que se realizan en el código estático; técnicas dinámicas, que actúan cuando se está ejecutando el *malware* y técnicas híbridas, que son una mezcla de las dos anteriores [4].

Técnicas de ofuscación

Las técnicas de ofuscación se emplean para modificar el código malicioso haciendo que su lectura sea difícil o imposible. Actúan como técnicas de anti-desensamblaje, evitando el proceso de ingeniería inversa. Pueden llevar a los analistas a estancamientos, información incorrecta y bucles infinitos.

- **Inserción de código muerto:** consiste en insertar código basura o instrucciones basura (como NOP) que no alteran el comportamiento del programa.
- **Reemplazo de instrucciones:** se sustituyen las instrucciones originales por instrucciones que realizan la misma funcionalidad pero cambian la apariencia del código.
- **Renombramiento de identificadores:** se cambia los nombres de los identificadores (registros, variables, etcétera.) sin alterar el funcionamiento del código.
- **Reordenamiento del código:** consiste en cambiar el orden de las instrucciones del código para que cambie en apariencia, sin afectar a su comportamiento.
- **Integración del código:** se incrusta el código malicioso en un programa pasando a formar parte de este, lo que hace que no sea fácil detectarlo.
- **Empaquetado de código:** es una técnica avanzada de ofuscación para crear variantes más complejas y sofisticadas del código malicioso, que dificultan el análisis estático. Para ello, el *malware* se comprime o cifra y se modifica o no, dando como resultado un paquete encapsulado y otro sin encapsular. Se puede definir cuatro tipos:
 1. Cifrado: una parte del *malware* se cifra para evitar que las herramientas de análisis puedan identificarlo como código malicioso. La otra parte se utiliza, normalmente, para que el código se pueda descifrar. Además los ciberdelincuentes suelen

tener su propio abecedario de cifrado, lo que dificulta el proceso de descifrado para los analistas de *malware*. En la figura 2.1 se puede apreciar la estructura de este cifrado.

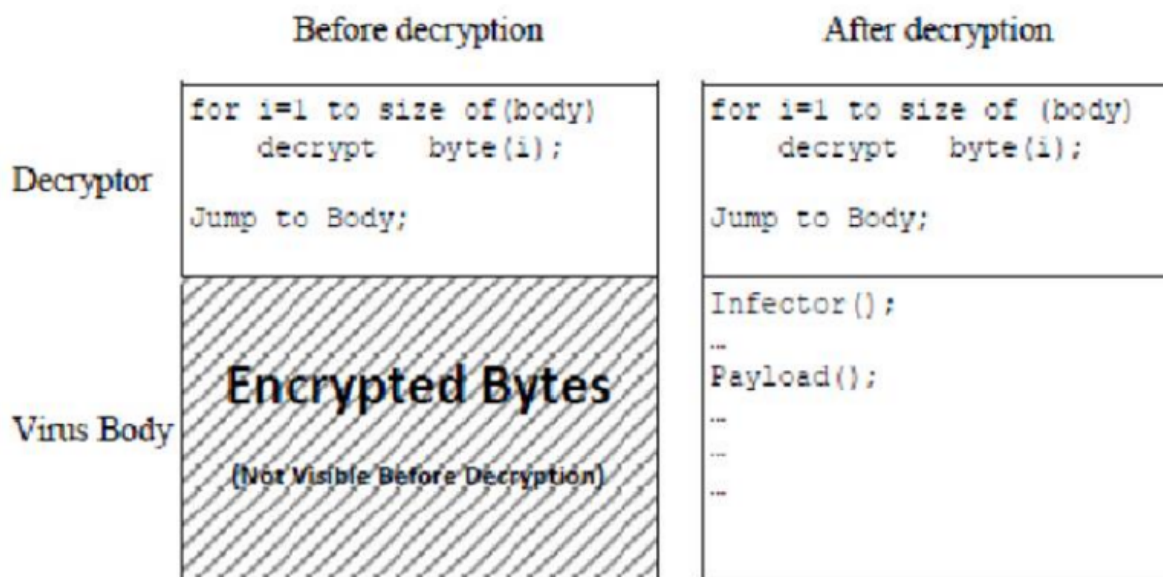


Figura 2.1: Estructura de un virus cifrado en Tahir (2018) [1]

2. Oligomorfismo: Opera de manera similar al cifrado con la diferencia de que se utiliza una clave distinta (escogida aleatoriamente de una lista por cada víctima infectada) para cifrar y descifrar la sección cifrada del código, con el fin de eludir la detección basada en firmas. Sin embargo, todavía existe la posibilidad de ser detectado por los antivirus al revisar todos los descifradores de la lista.
3. Polimorfismo: Técnica sofisticada que permite al *malware* cambiar su apariencia constantemente cada vez que se ejecuta o propaga. Usa el cifrado y la técnica de oligomorfismo, usando una clave para cifrar y descifrar diferente cada vez que se replica. Además usan motores de mutación para modificar parte de su código descifrador, manteniendo su algoritmo original intacto, con otras técnicas de ofuscación (como la inserción de código muerto), lo que lo hace muy difícil de detectar.
4. Metamorfismo: Reescribe su código (sin usar cifrado), produciendo variantes en la sintaxis, pero sin modificar el funcionamiento de este. Para ello utiliza diferentes técnicas de ofuscación diferentes. Es muy difícil de detectar porque cada copia del código malicioso produce una firma diferente.

Técnicas de Evasión dinámicas

El análisis dinámico del *malware* se realiza diseñando un entorno virtual aislado o usando herramientas de depuración para estudiar su comportamiento en ejecución. Sin embargo, existen técnicas para que un *malware* pueda detectar que se encuentra en un entorno monitorizado y oculte su comportamiento real.

- **Detección de entornos virtuales:** Los *malware* pueden detectar el entorno en el que se encuentran de dos formas: comparando las firmas de las herramientas de virtualización (como artefactos del sistema de ficheros) o del sistema operativo en el que se encuentran.
- **Detección de artefactos de red:** Se trata de la capacidad de los *malware* de usar los artefactos de la red para detectar que esta siendo analizado. Un ejemplo claro sería el aislamiento de la red, esta acción avisa al *malware* de que ha sido detectado y, por lo tanto, esta siendo analizado.
- **Detección de depuradores:** Los *malware* también pueden detectar cuándo están siendo analizados en un depurador. Pueden conseguirlo de diferentes formas: el *malware* puede preguntar a una interfaz de programación de aplicaciones (API)⁶ para detectar herramientas de depuración; también puede detectar servicios o manejadores (que delatan a un depurador) y puede comparar la firma o la dirección del depurador para identificarlo.
- **Detección a través del navegador:** En un entorno de análisis, los navegadores se comportan de forma diferente que en un sistema operativo anfitrión y contienen características (como características del lenguaje JavaScript⁷, el manejo de excepciones o el análisis sintáctico) que pueden avisar al *malware* de que está siendo analizado.

Técnicas Híbridas

Las técnicas híbridas, que mezclan métodos estáticos de ofuscación con comprobaciones dinámicas de entorno, generan estrategias de evasión más resistentes y complicadas de contrarrestar. Las técnicas de ofuscación

⁶Una API (*application programming interface*) o interfaz de programación de aplicaciones es un código que permite la comunicación entre dos aplicaciones para compartir funcionalidades e información, así pueden trabajar de manera más efectiva.

⁷JavaScript es un lenguaje de programación interpretado y orientado a objetos, que se utiliza principalmente en el lado del cliente en las páginas web.

intentan esconder la forma del código (por ejemplo, utilizando cifrado o metamorfismo), y las dinámicas tienen como objetivo ocultar el comportamiento durante la ejecución (detección de depuradores o de máquinas virtuales).

Un ejemplo de estas técnicas podría ser el desempaquetado dinámico condicionado. Es una técnica donde el código malicioso se encuentra «empaquetado» o cifrado estáticamente para evitar su análisis. Sin embargo, el código no se descifra al ejecutarse; primero, el programa realiza comprobaciones dinámicas de anti-análisis para detectar si se encuentra en un entorno virtual.

Si el *malware* no detecta un entorno de pruebas, realiza el descifrado en memoria y, utiliza metamorfismo para reconstruir su cuerpo con una estructura diferente en cada ejecución. Esta combinación híbrida asegura que el analista no puede analizar el código de forma estática (por estar cifrado) ni usando un entorno virtual o *sandbox* (porque el *malware* no se activa si está siendo observado).

En conclusión, las técnicas híbridas combinan ambos métodos para conseguir que el *malware* pase desapercibido ante los análisis manuales y automatizados.

2.1.4. Reflexión final: la fragmentación del código como eje de la evasión moderna

A lo largo de esta sección se ha mostrado cómo el *malware* ha evolucionado desde estructuras básicas y fácilmente analizables hasta convertirse en ecosistemas complejos, capaces de mutar, ocultarse y adaptarse dinámicamente. Tanto las técnicas de evasión como las de ofuscación tradicionales continúan siendo importantes, pero la situación actual muestra una tendencia particularmente importante: el uso de la fragmentación del código como método fundamental para sobrevivir.

La fragmentación permite dividir la funcionalidad maliciosa en múltiples piezas independientes (módulos, *scripts*, configuraciones, cargas cifradas o incluso funciones distribuidas) que solo adquieren sentido cuando se combinan en el entorno adecuado. Este enfoque dificulta la detección estática, impide el análisis dinámico completo y hace más difícil que las soluciones de seguridad monitoricen todo el ciclo de vida del *malware*.

En los ataques actuales, esta técnica no solo se utiliza para ocultar el

código, sino también para distribuir responsabilidades operativas entre diversos componentes, dificultando la atribución, el análisis forense y la correlación de eventos. También es cierto que esta técnica es utilizada por los atacantes cuando el tiempo no es un factor clave, porque el hecho de mandar los fragmentos a la víctima implica ser paciente para evitar la detección.

Por estas razones, la fragmentación se posiciona como una de las estrategias de evasión más representativas del *malware* de última generación, y es importante destacar que la fragmentación del código no es solo una técnica teórica mencionada en la literatura, sino un enfoque viable y probado para evadir mecanismos de defensa. De hecho, existe evidencia experimental sólida que demuestra que dividir un código malicioso en partes pequeñas —y distribuirlas de forma separada antes de su reconstrucción o ejecución— permite evadir los controles de seguridad. Un ejemplo claro es el trabajo de A. Suliman et al.(2008) [5], donde los autores presentan un *proof-of-concept* de ataques mediante fragmentación de *malware* en sistemas Linux y Windows, explotando vulnerabilidades con *SQL injection*⁸ en algunas bases de datos (como *Microsoft SQL Server 2005*), demostrando que «tal método de entrega es posible».

2.2. Paradigmas de Detección de *Malware*: De Firmas a la IA

Como se ha comentado en la sección anterior, el *malware* está en constante evolución y se precisa, por tanto, de una evolución similar de los métodos que existen para detectarlos.

No obstante, esta situación no es baladí, pues se ha llegado a demostrar, según se comenta en el artículo de Aslan y Samet (2020) [2], que el problema de la detección del *malware* se categoriza como NP-completo⁹ e incluso indecidible (no existe un algoritmo capaz de detectar todo el *malware*).

En esta tesitura, los mecanismos de detección de *malware* deben tratar de ser lo más precisos posibles y es por ello que el enfoque de estas técnicas ha ido evolucionando desde las que se van a considerar como «clásicas» hasta las «modernas» que ya emplean tecnologías como *Machine Learning* (ML) o IA. La presente sección, por tanto, va a tratar de analizar en qué consiste cada mecanismo de detección y qué resultados ha tenido, tanto

⁸*SQL injection* o inyección SQL es un ataque que explota una vulnerabilidad informática presente en una aplicación en el nivel de validación de las entradas para realizar operaciones sobre bases de datos SQL.

⁹Un problema NP-completo es un problema para el que no existe algoritmo eficiente que lo resuelva pero que su solución se puede verificar de forma rápida

sus fortalezas como sus posibles debilidades conocidas.

2.2.1. Análisis del *malware*

Si en algo coinciden todas las técnicas es que lo primero que se ha de hacer es analizar el *malware* encontrado antes de extraer sus características y clasificarlo. Para ello, hay dos clases de métodos de análisis: estático y dinámico.

Análisis Estático

En este análisis se estudia el código fuente del *malware* a examinar. De esta forma, se puede comprender su naturaleza (maligna o benigna) y tener una idea de su funcionamiento mediante ingeniería inversa [1].

Un ejemplo clásico de este tipo de análisis es el basado en firmas y el cálculo de *hashes*. En esta fase, se calcula una huella digital única del código sospechoso (utilizando algún tipo de algoritmo criptográfico) y después se busca en repositorios globales de inteligencia de amenazas si el fragmento ya ha sido catalogado como *malware*.

Otro ejemplo más profundo de análisis estático involucra el uso de herramientas de desensamblado como Ghidra. Gracias a estas herramientas, el analista es capaz de reconstruir la lógica interna del código, comprender los algoritmos de cifrado que pueda tener o descubrir puertas traseras sin llegar a ejecutar en ningún momento el *malware*.

Análisis Dinámico

El análisis dinámico se centra en el «qué hace» más que en el «qué». Examina el comportamiento del *malware* en entornos controlados para estudiar cómo actúa y qué se puede hacer para contrarrestarlo. Suele tener mejores resultados que el estático pero toma más tiempo [1].

Un ejemplo representativo de este tipo de análisis es la ejecución del *malware* en un entorno aislado y seguro. Mientras se ejecuta, el analista monitoriza en tiempo real las interacciones del código con el sistema operativo y las acciones que acomete en el mismo, como creaciones o eliminaciones de archivos.

Otro ejemplo que se ha de mencionar es la monitorización del tráfico de red que genera el código una vez entra en ejecución. Utilizando programas de análisis como Wireshark, los analistas capturan y estudian los paquetes de datos entrantes y salientes, permitiendo identificar intentos de conexión

a servidores remotos para descarga de *payloads* o la entrega silenciosa de información robada del usuario.

2.2.2. Técnicas «Clásicas»

Se denomina como técnicas «clásicas» a aquellos mecanismos de detección que se aplicaban antes de la aparición del ML o la IA. Sin embargo, en la actualidad estas tecnologías se están aplicando en estas técnicas tradicionales para mejorar significativamente su rendimiento.

Detección basada en firmas

Esta técnica es la tradicional por excelencia. Se basa en la búsqueda de firmas, es decir, partes del código desensambladas que se repiten y que pueden hacer que se identifique al agente dentro de una familia de *malware* ya conocida con la misma firma. Es la empleada por muchos antivirus [1].

A pesar de que tiene un gran desempeño con *malware* conocido, pues es eficiente y rápido, resulta nulo contra el *malware* nuevo, siendo esa la gran desventaja de este método y la razón de por qué se empiezan a usar otros métodos como los heurísticos [2].

Detección heurística

Pretende llegar a una conclusión utilizando el camino óptimo. No es necesario conocer la estructura interna del código sino que mediante algoritmos o reglas se buscan patrones o comportamientos conocidos [3].

En los últimos años, se ha empezado a añadir técnicas de ML a los algoritmos y reglas de este método y se han presentado estudios y artículos para tratar de mejorarlo, como el de Naval et al. (2015) [6], basado en la creación de grafos en el que las llamadas al sistema son los nodos y las secuencias entre ellas aristas y con el objetivo de encontrar patrones de comportamiento típicos del *malware*. A pesar de que estas técnicas que se explican en el artículo de Aslan y Samet (2020) [2] son muy útiles contra *malware* desconocido y contra el conocido, todas tienen algunas desventajas y problemas ante el *malware* complejo de nueva generación, tal y como concluyen en el artículo.

Detección basada en comportamiento

Este tipo de detección estudia el comportamiento del sistema ejecutando el malware en entornos controlados analizando llamadas al sistema, procesos creados o archivos modificados y comparando este

comportamiento con patrones de comportamiento malicioso.

Según Aslan y Samet (2020) [2], los sistemas que realizan esta técnica se construyen en tres fases:

1. Análisis del comportamiento mediante diversas monitorizaciones en entornos controlados.
2. Recopilación de las características analizadas. Se extraen las características y se seleccionan las más relevantes mediante minería de datos.
3. Clasificación en benigno o maligno dependiendo de las conclusiones obtenidas por el estudio de las características anteriores usando algoritmos de ML.

En los últimos años se ha llevado a cabo investigaciones sobre el empleo de minería de datos y ML en este tipo de detección con el objetivo de ensalzar la principal ventaja de este método -el poder identificar el *malware* sin necesidad de conocer su código- y paliar las desventajas. No obstante, todas estas investigaciones se encuentran con problemas que Aslan y Samet (2020) [2] destacan:

- La detección es lenta y costosa.
- Algunos *malware* no funcionan correctamente en máquinas virtuales.
- Los *malware* más modernos tienen mecanismos de ofuscación capaces de impedir su correcto análisis.

Detección basada en *model checking*

El *model checking* es una técnica de verificación automática que examina todas las ejecuciones de un sistema para comprobar si cumple con una corrección formal dada y emplea especificación formal y matemática.

En el ámbito de la detección de *malware*, se extrae manualmente el comportamiento del mismo y se codifica en LTL, un lenguaje formal para describir comportamientos maliciosos como ocultación, propagación e inyección de código. Observando y comparando la actuación del programa «no infectado» con esos patrones de «programa infectado» se puede llegar a identificar si en el sistema hay o no *malware* [2].

Esta técnica puede llegar a identificar cierta cantidad de *malware* desconocido, pero no puede detectar todo el *malware* de nueva generación [2].

2.2.3. Técnicas «modernas»

Se conoce como técnicas «modernas» aquellas que aplican tecnologías actuales y en auge como el *deep learning*¹⁰, el ML o la IA para detectar *malware*.

El proceso completo de detección actual se ha explicado con anterioridad, pero conviene repetirlo para un mayor entendimiento en esta sección tal y como se aprecia en la Figura 2.2:

1. En primer lugar se analiza de forma estática y dinámica el supuesto *malware*.
2. Este análisis genera un *dataset*¹¹ de características que son filtradas por relevancia usando minería de datos.
3. Las características del *dataset* son estudiadas y permiten determinar si es o no verdaderamente un *malware*. Es precisamente en este último punto donde se sitúan las nuevas tendencias como las basadas en *deep learning* o las basadas en *cloud*.

¹⁰El *deep learning* es una rama del *machine learning* que emplea redes neuronales profundas para simular la toma de decisiones humana

¹¹Un *dataset* es una colección de datos estructurados y ordenados utilizados para analizar características o entrenar modelos. Es básicamente el contenido de una tabla dentro de una base de datos.

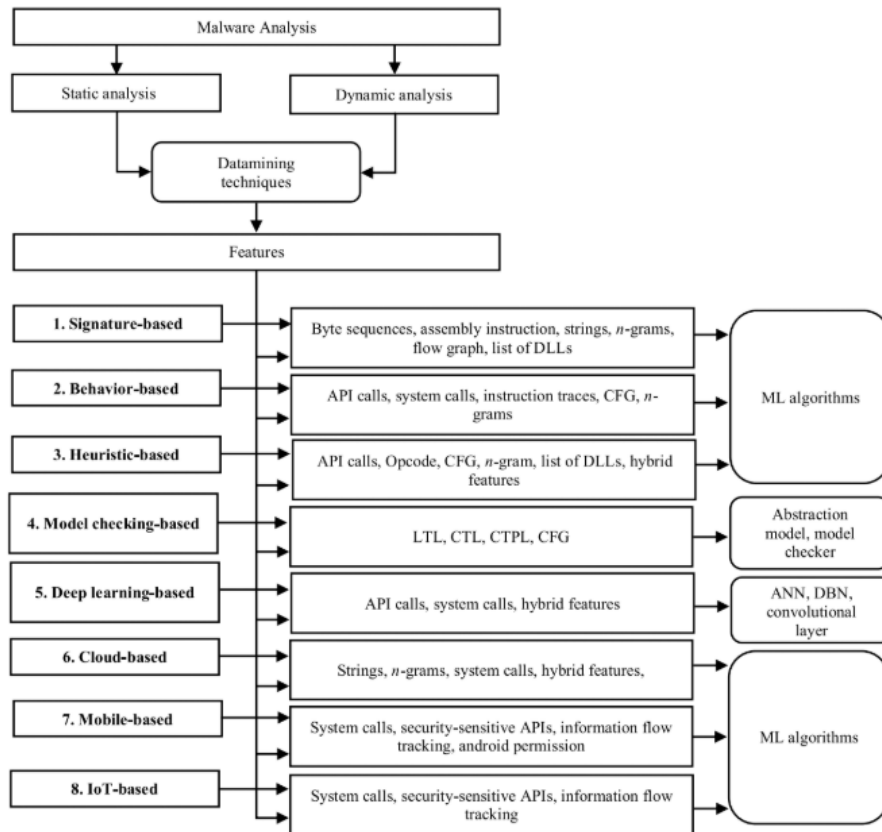


Figura 2.2: Proceso de detección de un *malware*, así como los tipos de detección que existen. [2]

Detección basada en *deep learning*

A pesar de que el *deep learning* es ampliamente utilizado en otras circunstancias, no lo es suficiente en el contexto de detección de *malware*. Esto se debe principalmente, según Aslan y Samet (2020), [2] a que no es resistente a los ataques de evasión y que añadir capas no suele mejorar el rendimiento. Todo esto causa que este tipo de detección aún se encuentre en etapas tempranas de su desarrollo.

Para explicar su ineficacia contra técnicas de evasión, el artículo menciona un estudio de Grosse et Al. en el que se llegó a datos demoledores: la tasa de clasificación errónea llegó hasta el ochenta por ciento [2].

Detección basada en *Cloud*

La tecnología *cloud* es una de las más utilizadas en la actualidad debido a las múltiples ventajas que aporta. Esto ha llevado a que también se emplee para la detección del *malware* por la mejora de rendimiento que ofrece gracias a bases de datos más grandes y recursos computacionales

intensivos [2].

Este auge en su uso ha llevado a los investigadores a tratar de dar con propuestas que puedan ser útiles y no tengan desventajas, como por ejemplo:

- SplitScreen, un nuevo sistema antimalware propuesto por Sang Kil et al. [7], explicado en el artículo de Aslan y Samet (2020) [2], que actúa como un esquema de detección de *malware* que da un paso previo de preselección antes de la fase de coincidencia de firmas. Este proceso bifásico consigue detectar el *malware* de forma rápida y eficiente y además puede dividirse en un proceso cliente/servidor, lo que reduce la cantidad de almacenamiento necesario. Este sistema posee dos debilidades claves, la primera es que tiene dificultades con los virus desconocidos y la segunda es que utiliza un único servidor en la nube, lo que le hace susceptible si ocurre un ataque de denegación de servicio.
- Plataforma de detección de *malware* en *cloud* presentada por Mitchell et al. (2024) [8]. Esta plataforma propuesta en el artículo monitoriza contenedores en Kubernetes¹² mediante llamadas al sistema mediante eBPF¹³. La comprobación de si es o no *malware* se realiza por *machine learning*.
- *Malware-Detection-as-a-Service* (MDaaS) es una plataforma de detección de *malware* basada en DRL¹⁴ y diseñada para entornos *cloud* y *serverless* [9]. Pretende detectar *malware* de manera eficiente y con el menor coste posible en cualquier plataforma *cloud* y conseguir tasas de detección cercanas al óptimo. La plataforma escogida para la implementación fue AWS Lambda y los autores afirman que tuvieron resultados exitosos.

2.2.4. Reflexión final: la fragmentación de código como técnica de evasión ante las técnicas de detección

En esta sección se ha explicado las diversas técnicas que se emplean hoy en día para analizar, detectar y clasificar el *malware*. La fragmentación

¹²Kubernetes es una plataforma para el manejo de contenedores que permite desplegar y automatizar aplicaciones en contenedores

¹³eBPF es una máquina virtual que se ejecuta dentro del kernel de Linux y que permite la escritura de programas de manera eficiente y segura

¹⁴DRL (*Deep Reinforcement Learning*) es una forma de *deep learning* en la que se emplea *Reinforcement Learning* (aprendizaje basado en prueba y error) junto con las redes neuronales profundas del *deep learning*.

descoloca a aquellas técnicas basadas en firmas y que no son capaces de detectar *malware* novedoso, mientras que también deja en jaque aquellos sistemas modernos basados en *machine learning* debido a que estos dependen de la calidad y la importancia de los datos de entrenamiento. Por otro lado, la incesante necesidad moderna de la buscar la máxima optimización puede ser también un problema para detectar este malware fragmentado, pues los sistemas se centran más en ver cómo pueden ser más eficientes. La aparición por tanto de esta nueva modalidad de *malware* dificulta que los modelos aprendan patrones confiables. Es por todo esto que estas técnicas enfrentan desafíos significativos ante la aparición de este tipo de malware que fragmenta su código.

2.3. La Arquitectura *Serverless* como Nuevo Campo de Batalla

Uno de los paradigmas que más ha influido en el desarrollo de aplicaciones en la nube es la computación sin servidor. Su propuesta principal consiste en eliminar por completo la gestión de servidores, de manera que el proveedor *cloud* ¹⁵ se encargue del aprovisionamiento, la ejecución y la ampliación de recursos. En este modelo, el código se ejecuta de forma temporal y bajo demanda, siguiendo una arquitectura orientada a eventos, lo que impulsa aplicaciones más distribuidas, modulares y escalables. Servicios como AWS Lambda han acelerado esta transición al automatizar la infraestructura subyacente, eliminar la capacidad ociosa y optimizar costes operativos. [10].

De forma paralela, el auge del *serverless* está muy relacionado con la creciente automatización de procesos en la nube. Según Refonaa et al. (2025) [11], este modelo permite la sustitución de tareas repetitivas, la reducción de la carga operacional y la unificación de servicios internos y externos para construir flujos altamente cohesionados.

Esta sección examina los fundamentos del paradigma *serverless* y el papel de AWS Lambda como infraestructura central, destacando su relevancia en escenarios de evasión y despliegue fragmentado de componentes maliciosos.

¹⁵La nube (*cloud*) es el uso de una red de servidores remotos conectados a internet para almacenar, administrar y procesar datos, servidores, bases de datos, redes y software. En lugar de depender de un servicio físico instalado, se tiene acceso a una estructura donde el software y el hardware están virtualmente integrados.

2.3.1. ¿Qué es *Serverless*?

El término *serverless* hace referencia a un modelo de computación en el que el desarrollador solo se preocupa de escribir el código sin necesidad de gestionar servidores, sistemas operativos o aprovisionamiento de recursos. Aunque los servidores no desaparecen, su administración queda en manos del proveedor *cloud* (Google Cloud, Amazon Web Services (AWS), Azure, etcétera). Por tanto, los costes se basan en el uso real del código (pago por uso).

Según el estudio de Shrestha (2019) [10], este modelo consigue disminuir significativamente el coste operativo porque suprime la necesidad de gestionar instancias persistentes, automatiza el mantenimiento del sistema y gestiona la escalabilidad sin necesidad de intervención humana.

Por otro lado, en Refonaa et al. (2025) [11] el enfoque *serverless* favorece la automatización de procesos en la nube, lo que hace posible incorporar funciones con bases de datos, API gateway¹⁶, servicios externos o flujos de datos en tiempo real. Esto reduce los errores humanos, acelera los ciclos de despliegue y optimiza la eficiencia de los sistemas complejos.

2.3.2. AWS Lambda

AWS Lambda es el servicio *serverless* más ampliamente adoptado y representa la manifestación más avanzada del modelo *Function as a Service* (FaaS)¹⁷. Lanzado por Amazon en 2014, permite ejecutar la ejecución de funciones que son activadas por eventos procedentes de servicios como Amazon S3¹⁸, DynamoDB¹⁹, Simple Queue Service (SQS²⁰), API Gateway o flujos de datos en tiempo real. Su arquitectura se basa en contenedores efímeros que Amazon gestiona y que se activan automáticamente cuando se requieren. Cuando un contenedor se inicializa por primera vez, se produce lo que se llama *cold start*. Las invocaciones posteriores, en cambio, vuelven a usar el mismo entorno (*warm start*), lo que reduce la latencia.

¹⁶Una API gateway o pasarela API es un código que presenta un único punto de entrada para que clientes (como aplicaciones web) accedan a múltiples servicios del lado del servidor, al tiempo que gestiona las interacciones cliente/servidor.

¹⁷Función como servicio (FaaS) es un servicio de computación en la nube que posibilita a los usuarios ejecutar código como respuesta a eventos, sin tener que manejar la complicada infraestructura que normalmente acompaña el desarrollo y la implementación de aplicaciones de microservicios.

¹⁸Amazon S3 es un servicio ofrecido por AWS que proporciona almacenamiento de objetos.

¹⁹DynamoDB es un servicio de base de datos NoSQL totalmente administrado por AWS.

²⁰SQS o Amazon Simple Queue Service es un servicio de AWS que permite desacoplar y escalar microservicios, sistemas distribuidos y aplicaciones sin servidor.

En el artículo de Shrestha (2019) [10], se examina en profundidad los fundamentos de Lambda, destacando aspectos como la integración completa con el ecosistema AWS (uso de servicios como Amazon S3, DynamoDB, CloudWatch ²¹, etcétera), los diversos modelos de invocación (*push* y *pull*) y la necesidad de que las funciones sean totalmente *stateless* ²². Además, se detalla cómo las discrepancias entre lenguajes compilados e interpretados impactan en el rendimiento, la duración de arranque y el tamaño del paquete de despliegue. No se establece que exista un lenguaje de programación objetivamente superior para AWS Lambda, sino que se destaca que las variaciones de rendimiento entre los lenguajes nativos son mínimas y que la selección generalmente está basada en lo que el proyecto requiera y en lo familiarizado que esté el programador con cada entorno. Sin embargo, el autor detecta algunos patrones comparativos: los lenguajes interpretados, como Python y Node.js, tienen tiempos de arranque (*cold starts*) más cortos y paquetes de despliegue más ligeros. Por otro lado, los lenguajes compilados, como Java o C, requieren inicializaciones más largas, aunque pueden proporcionar un desempeño más estable después de la primera ejecución.

Refonaa et al. (2025) [11], añaden a esta perspectiva que AWS Lambda es un elemento clave en la automatización de la nube. Aseguran que su capacidad para escalar automáticamente, su ejecución bajo demanda y la operación sin servidores reduce de manera significativa la carga administrativa. Esto posibilita la coordinación de procesos complejos a través de eventos, funciones distribuidas y flujos de trabajo.

2.3.3. Nuevos Riesgos de Seguridad

Aunque la arquitectura *serverless* reduce considerablemente la superficie de ataque relacionada con la gestión de servidores tradicionales, presenta también un conjunto de nuevos riesgos de seguridad. El informe de OWASP Top 10 (2017) [12] examina cómo las amenazas «clásicas» evolucionan en este nuevo paradigma y cómo emergen nuevos vectores de ataque que deben ser considerados en el diseño de aplicaciones basadas en la arquitectura *serverless*.

²¹CloudWatch es un servicio, proporcionado por AWS, que supervisa las aplicaciones, responde a los cambios de rendimiento, optimiza el uso de los recursos y proporciona información sobre el estado operativo.

²²*stateless* se refiere a un sistema informático que trata cada solicitud como independiente sin guardar un registro de las anteriores y sin guardar su estado.

Inyección

En primer lugar, el riesgo de inyección se incrementa por la expansión de las superficies de entrada. En los sistemas *serverless*, a diferencia de las arquitecturas unitarias donde el origen de los datos proviene generalmente de una sola API, las funciones tienen la capacidad de ser activadas por varias fuentes como el almacenamiento, flujos de datos, servicios de mensajería, cambios de bases de datos o correo electrónico. Esto significa que la entrada proviene de canales que no pueden ser controlados a través de cortafuegos o perímetros tradicionales, lo que incrementa de manera significativa las posibilidades de que se entregue código malicioso hacia funciones vulnerables.

Fallos de autenticación

La autenticación fallida adquiere una dimensión diferente. Como las funciones son totalmente *stateless* y se ejecutan de forma independiente, los flujos de autenticación y autorización no se basan en un servidor centralizado, sino en varios puntos de acceso. Un atacante tiene la posibilidad de aprovecharse de recursos que están mal diseñados —por ejemplo, una dirección de correo que funciona como disparador interno o un *bucket*²³ público olvidado— para llevar a cabo ejecuciones no autorizadas sin requerir credenciales.

Exposición de datos sensibles

Una de las vulnerabilidades más frecuentes continúa siendo el guardado de secretos en variables de entorno, en archivos temporales dentro del directorio */temp* o en código fuente contenido en el paquete de despliegue, a pesar de que los proveedores en la nube ofrecen técnicas sofisticadas para la gestión de claves y cifrado. Esto hace que los datos residuales entre ejecuciones pueden ser encontrados y reutilizados por los ciberdelincuentes.

Fallos de control de acceso

Además, la arquitectura sin servidor está sujeta a fallos de control de acceso que resultan de configuraciones erróneas en IAM²⁴. Cada

²³El *bucket* o contenedor de datos en la nube es un espacio de almacenamiento virtual donde se pueden guardar archivos y datos en plataformas de almacenamiento en la nube, que actúa como una carpeta donde puedes organizar y acceder a tus archivos de manera fácil y segura, con la ventaja de que están disponibles desde cualquier lugar con conexión a Internet.

²⁴La gestión de identidad y acceso (IAM) es un modo de saber quién es el usuario y qué tiene permitido hacer en la nube.

función puede tener permisos excesivos, que en el caso de ser explotados, posibilitan que un atacante lleve a cabo acciones destructivas como la eliminación del *bucket* de almacenamiento, la alteración de datos e incluso el control parcial de una cuenta. Por eso, es importante seguir el principio de privilegio mínimo (*Zero Trust* ²⁵).

Mala configuración de seguridad

Para llevar a cabo lógicas internas fuera del flujo esperado, se emplea políticas que permiten la carga de archivos sin validación, recursos públicos sin intención y funciones con límites temporales excesivos. En algunos casos, estas vulnerabilidades posibilitan ataques de denegación de servicio (DoS) o incluso ataques de denegación de billetera (DoW)²⁶, que buscan no solo interrumpir el servicio, sino generar un uso excesivo de recursos con consecuencias económicas directas.

Insuficiencia de monitorización y registro

Los sistemas *serverless* dependen en gran medida de los servicios de monitorización y registro de sus proveedores en la nube, cuyos límites de almacenamiento y formato pueden obstaculizar la identificación de entradas maliciosas. La naturaleza efímera de las funciones complica la reconstrucción de ataques en ausencia de un mecanismo de auditoría meticulosamente elaborada.

Conclusión

En conjunto, estos riesgos y algunos más demuestran que la arquitectura *serverless* no elimina algunas de las amenazas existentes, sino que las transforma y extiende hacia nuevas direcciones. La combinación de varios puntos de acceso, la ejecución distribuida y el modelo de privilegios finos requieren un enfoque de seguridad adaptado que tenga en cuenta la ausencia de perímetros tradicionales y la necesidad de validar, auditar y aislar cada función como un componente autónomo del sistema.

2.3.4. Reflexión final: Nuevo Campo de Batalla

La arquitectura *serverless* implica un cambio significativo en la manera de diseñar, implementar y ejecutar aplicaciones en la nube. Su capacidad para suprimir la administración de servidores, escalar automáticamente

²⁵*Zero trust* (confianza cero) es una estrategia de ciberseguridad que consiste en no confiar en nada ni nadie de manera implícita, es decir, verificar todo.

²⁶Un ataque de denegación de billetera (*Denial of Wallet* - DoW) es un tipo de ciberataque que busca generar cargos o costes excesivos para agotar los recursos financieros de una empresa.

y ejecutar funciones bajo demanda, entre otras cosas, ha promovido su adopción a gran escala y lo ha transformado en un modelo fundamental para el progreso de la computación distribuida. No obstante, estas mismas propiedades —dependencia total de sucesos, fragmentación funcional, efimeridad e integración intensa con servicios externos— generan un área de ataque que es diferente a la de las estructuras tradicionales.

Como se ha mencionado, servicios como AWS Lambda permiten un nivel de modularidad y automatización sin precedentes, lo que ha hecho más fácil tanto la ejecución eficiente como el desarrollo ágil. Pero su ecosistema plantea importantes desafíos: varios puntos de entrada, configuraciones IAM delicada, almacenamiento incorrecto de datos sensibles y una monitorización complicada a causa de la naturaleza efímera de las funciones.

En resumen, la arquitectura *serverless* se ha establecido como un nuevo campo de batalla digital: un escenario en el que los beneficios operativos y la eficiencia coexisten con tendencias de ataque emergentes y conductas complicadas de rastrear a través de métodos convencionales. Esta dualidad es la razón por la que se necesita investigar, modelar y entender cómo los ciberdelincuentes podrían explotar estas características.

2.4. Convergencia: *Malware* «Nativo de *Serverless*» y Estrategias de Evasión Avanzadas

En este apartado se aborda la unión entre el *malware* y los entornos *cloud* que se explican en las secciones anteriores, poniendo especial énfasis en todas aquellas técnicas de evasión que se están llevando a cabo en la actualidad y cómo la innovación del *malware* va ligada al *serverless*. Éste es el punto fundamental del proyecto que se va a realizar en los siguientes capítulos. Para ello, se hará uso del artículo de Radunović y Reinović (2020) [13] y de la Tesis de Nikita Ponomarev (2022) [14].

2.4.1. La importancia del *Command and Control* (C2) y las nuevas estrategias de evasión

Una vez un sistema se ha infectado, el *malware* está configurado para conectarse a servidores C2²⁷ para recibir instrucciones o enviarse información robada. Según Radunović y Reinović (2020) [13], la mayoría de ciberataques de hoy en día no serían posibles sin una estructura C2. Por

²⁷El C2 (Control and Command) es la infraestructura utilizada para comunicarse y dar órdenes a los dispositivos que han sido infectados por *malware*

ello, los defensores han ido desarrollando nuevos métodos de detección y derribo de estos servidores. Por otro lado, los atacantes, conscientes de esta situación; se han visto obligados a buscar nuevos métodos de alojamiento de los servidores C2 y nuevas formas de dar instrucciones a sus sistemas controlados de la manera más sigilosa posible.

Pero antes del análisis de estas nuevas formas, primero se debe conocer qué objetivos tiene la comunicación entre *botnet* y servidor. En el artículo antes mencionado [13] se definen tres características u objetivos:

- Obtener actualizaciones o comandos específicos.
- Desplegar funciones como reconocimiento , robar credenciales, cargar contenido robado, etcétera.
- Informar del éxito o fracaso del sistema infectado.
- Ser informado de cuándo «apagarse» o cesar las operaciones.

En este contexto, un aliado muy poderoso para estos atacantes son las redes sociales (RRSS), que han facilitado mucho las cosas en cuanto a la transmisión de órdenes de forma discreta y que, en muchos casos, se convierten en los propios servidores C2. Normalmente, se crean cuentas en estas RRSS que encriptan la información. Radunović y Reinović identifican cinco formas de hacerlo.

Ocultar órdenes mediante publicaciones con texto

Una de las técnicas más clásicas en este apartado es, sin duda, la de crearse un perfil en una red social y entrenar al *malware* para, una vez infectada la víctima, buscar comandos en las publicaciones específicas de esa cuenta o página. No obstante, el punto débil de este método está claro, pues una vez cerrada esa cuenta, el *botnet* se queda sin comunicación.

Para solucionar este problema y poder acceder a nuevas cuentas una vez se ha cerrado la anterior, se emplean símbolos de aviso, que delimitan cuándo empieza una cadena de comandos en cualquiera de sus cuentas controladas. Así, el *botnet* no debe analizar cuentas, sino cadenas de texto marcadas para descifrar y que puede llevar a otras cuentas o a servidores C2. Estos símbolos empleados, pueden ser normales o más sofisticados (utilizando por ejemplo Unicode cheroqui²⁸).

²⁸El Unicode cheroqui es un bloque Unicode que contiene los caracteres silábicos necesarios para la escritura del idioma cheroqui procedente de los pueblos indígenas de los bosques del sureste de los Estados Unidos (los *Cherokee*).

Otro enfoque mencionado por el artículo es el realizado por Compagno et Al., en el que el *bot* oculta su comunicación con el servidor en la propia cuenta del usuario infectado. Una vez este usuario sube una publicación a una red social, se envía también una secuencia de caracteres que no se imprimen y que son «invisibles» al usuario. Para recibir la información del servidor, el *botnet* debe revisar continuamente las publicaciones de sus amigos buscando la misma cadena invisible. Este modelo tiene la debilidad de que depende mucho de la infección de los amigos de las víctimas para recibir órdenes.

Ocultar comandos C2 en imágenes compartidas mediante esteganografía

Mediante esquemas de esteganografía²⁹ se puede ocultar código dentro de imágenes de forma que pasen desapercibidas. Esto quiere decir que se puede producir una comunicación entre servidor y *botnet* a través de estas imágenes.

Uno de los ejemplos que dan Radunović y Reinović en su artículo es el del *malware* HAMMERTOS, que utilizó una cuenta de Twitter falsa para proporcionar al *botnet* un *link* y un *hashtag*. El enlace llevaba a un documento con una fotografía con información escondida, mientras que el *hashtag* le indicaba al *bot* en qué parte de la imagen buscar y cómo descifrar el mensaje una vez lo identificase.

Uso de las nubes públicas

Se ha puesto de moda el empleo de sistemas basados en la nube para proporcionar enlaces que lleven a servidores C2 o como servidor en sí.

Un ejemplo de esto es LOWBALL, que utilizó Dropbox como servidor C2. Esta puerta trasera se comunicaba con la nube pública para todas sus acciones, como recibir comandos o exfiltrar archivos robados. Como esto se ejecuta en HTTPS y en un puerto TCP 443 común, es muy complicado distinguirlo del tráfico habitual. Por tanto, transferir los comandos directamente a través de una nube pública utilizando HTTPS evita que los *botnet* se tengan que conectar a un servidor C2 [13].

²⁹La esteganografía es el arte de esconder información dentro de otro mensaje, archivo u objeto físico para ocultarla.

Uso de DGA

Para que un *bot* sepa a que URL conectarse, estas URLs deben estar codificadas estáticamente en el código del malware. Esto tiene como desventaja que una vez se descubre cuál es la URL y se inhabilita, el bot deja de estar operativo. Es por este motivo por el que se describe el Algoritmo de Generación de Dominios (DGA) como la táctica a emplear, pues lo que hace es generar miles de dominios que el *botnet* debe analizar y conectarse al único o únicos que funcionen. Estos DGAs binarios se incrustan en el código y son muy útiles para crear nuevos dominios [13].

Siguiendo esta idea, Cui et al. (2011) [15] construyeron un modelo que crea miles de cuentas de usuario (siendo por tanto un Algoritmo de Generación de Usuarios o UGA) permitiendo cambiar rápidamente de cuenta utilizada para C2 [13].

Uso de cuentas famosas

Es importante tener un canal de comunicación C2 dinámico, siendo por tanto un problema ir generando cuentas falsas que van a ser eliminadas. Por esto, otra tendencia consiste en publicar mensajes cifrados como comentarios en publicaciones de gente famosa, como el grupo de piratería *Turla*, quienes publicaron estos mensajes en la cuenta pública de Instagram de Britney Spears y lo único que debía hacer el bot era buscar símbolos de inicio o patrones que le llevasen a poder descifrar el contenido del comentario [13].

Para este método es muy importante que las cuentas sean lo más famosas posibles para pasar más desapercibidos.

2.4.2. *Malware en Serverless y xTyphonTC*

En el proceso de conseguir que comandos y servidores C2 pasen desapercibidos, es normal preguntarse si se pueden utilizar los nuevos servicios *serverless* para crear, manipular y controlar el *malware*. Los beneficios de utilizar estos servicios ya están explicados con detalle en este documento y hace imaginar las ventajas que pudiera tener el *malware* y alojar el servidor C2 en dichas plataformas.

Una de las personas que se ha formulado esta pregunta es Nikita Ponomarev, quien en su tesis [14] quiso diseñar una prueba de concepto que pudiera demostrar que es posible generar código malicioso en AWS Lambda.

El objetivo de su herramienta sería exfiltrar datos sensibles y mandarlos a un canal C2 para su análisis. El proceso sería el siguiente [14]:

- El atacante consigue acceso a la cuenta AWS de un usuario con funciones Lambda existentes.
- El atacante lee el código de la función de la víctima y selecciona una biblioteca que es importada. Tras esto, utilizando la herramienta genera la biblioteca maliciosa.
- Después, se crea un archivo *zip* que contiene el *malware* en la máquina del atacante y se aloja en una cuenta AWS del atacante como una capa Lambda pública.
- El atacante activa el oyente DNS y usando las credenciales de la víctima importa la capa Lambda.
- Una vez hecho esto, el atacante solo debe esperar a que esa función *Lambda* se ejecute y se envía la información a un servidor que debe decodificarlo y descifrarlo.

Ponomarev (2022) [14] concluyó en su tesis que no solo se puede generar *malware* en AWS Lambda, sino que su prueba de concepto podría usarse como herramienta de seguridad ofensiva. Además, afirmó que su estudio podría ser de los primeros en abordar este tema, por lo que todavía hay muchas vulnerabilidades para explorar y explotar.

En este contexto se sitúa xTyphonTC, que pretende fragmentar el código del *malware* (que como ya se ha analizado anteriormente conlleva grandes ventajas) empleando funciones Lambda en AWS para su control. Mientras que el trabajo de Ponomarev (2022) [14] se centra en la persistencia y la exfiltración de datos, este estudio propone un enfoque diferente: el uso de la arquitectura *serverless* como un motor de sigilo para la inyección de fragmentos en un sistema. El hueco que ocupa xTyphonTC en la literatura radica en demostrar que el *serverless* es un sistema distribuido que explota la modularidad de las funciones Lambda para que el código malicioso nunca resida completo en un solo lugar.

2.4.3. Comparativa de aproximaciones y posicionamiento de xTyphonTC

Después de analizar los estudios actuales en *malware*, *serverless* y métodos de evasión, resulta fundamental posicionar a xTyphonTC y

diferenciarlo frente a dichos trabajos previos. Mientras que la literatura se ha centrado en el uso de la nube como servidores de mando y control (C2) o en la infección directa de funciones Lambda, este proyecto propone el uso de la infraestructura *serverless* como un motor de entrega fragmentada para evadir defensas locales en sistemas tradicionales.

En la tabla 2.2 se presenta una comparativa técnica que permite identificar el aporte específico de esta propuesta frente a los antecedentes revisados:

Criterio	C2 en Redes Sociales [13]	Malware en Lambda [14]	Fragmentación Clásica [5]
Objetivo Principal	Sigilo en la comunicación C2.	Persistencia y exfiltración en la nube.	Evasión de firmas mediante fragmentación de archivos.
Infraestructura	Redes Sociales (X, Instagram).	AWS Lambda.	Servidores convencionales / SQL.
Técnica de Evasión	Esteganografía, uso de cuentas famosas...	Inyección de librerías maliciosas en la nube.	División de paquetes.
Payload	Comandos de texto / enlaces.	Agente de exfiltración de datos <i>Cloud</i> .	Binarios fragmentados en disco.
Limitación Clave	Depende de la cuenta activa en la red social.	Necesita acceso a credenciales de la nube.	Mayor visibilidad en inspección de archivos.

Tabla 2.2: Comparativa de xTyphonTC frente a investigaciones y técnicas previas.

A diferencia de las propuestas de Radunović y Veinović (2020) [13], que buscan ocultar la comunicación, xTyphonTC busca ocultar el artefacto mismo durante su tránsito. Por otro lado, frente al enfoque de Ponomarev (2022) [14], que se limita al entorno *cloud-to-cloud*, este trabajo explora el vector de ataque *cloud-to-host*. El valor diferencial de xTyphonTC consiste en demostrar que la fragmentación apoyada de el uso de fragmentos temporales permite que el código malicioso nunca resida completo en un solo punto de la infraestructura de red, rompiendo así los esquemas de detección perimetral y de comportamiento estándar.

Capítulo 3. Tecnologías

Tras el estudio de la documentación relevante, procede detallar las tecnologías empleadas en xTyphonTC y justificar su selección en función de las necesidades del proyecto.

Para ello, este capítulo se organiza en tres bloques: las tecnologías asociadas al entorno del atacante, las utilizadas en el entorno de la víctima y un conjunto de herramientas de propósito general que han servido de apoyo durante el desarrollo y la evaluación.

3.1. Tecnologías «del atacante»

Se entiende como tecnologías propias «del atacante» aquellas que van a ser empleadas para ejecutar la creación y difusión de *malware* en *serverless*. Esto es, todo aquello que se use para crear los fragmentos e introducirlos con el objetivo de infectar una víctima.

Se emplea un enfoque *bottom-to-top*, empezando desde lo que se considere más bajo nivel (como el lenguaje de programación) hasta lo de más alto nivel.

3.1.1. Python

Python (creado por Guido Van Rossum en 1989) es un lenguaje de programación interpretado y de alto nivel conocido por su sintaxis clara, su fácil aprendizaje y su amplio ecosistema de librerías¹.

Se emplea para la realización del código de las funciones Lambda que van a ejecutar el *malware*.

Se ha elegido este lenguaje debido a que, según los experimentos de Shrestha (2019) [10], Python muestra tiempos de inicio típicos de entre 0.20 y 0.32 segundos, lo que lo coloca entre los más veloces, y los lenguajes interpretados son recomendados explícitamente cuando la prioridad es reducir la latencia o los costes operativos. Por lo tanto, considerando que el artículo también indica que la experiencia y comodidad del equipo son factores válidos para elegir el lenguaje, se ha considerado que Python es particularmente apropiado para este trabajo.

¹<https://python.org>

3.1.2. AWS Lambda

AWS Lambda es un servicio de *serverless* basado en eventos proporcionado por Amazon Web Services (AWS). Permite ejecutar código en la nube sin necesidad de administrar o aprovisionar servidores ².

Se utiliza para ejecutar de manera temporal y controlada las distintas funciones que componen el *malware*.

En este trabajo se ha optado por utilizar AWS Lambda como plataforma FaaS frente a alternativas como Google Cloud Functions o Azure Functions por varios motivos técnicos y prácticos.

En primer lugar, en términos de integración con otros servicios, AWS ofrece un ecosistema amplio y maduro, como explica Shrestha (2019) [10]. Es destacable la integración con API Gateway (nativo de Amazon), que va a permitir que el tráfico cifrado se camufle bajo dominios de alta reputación (como amazonaws.com). Además, AWS Lambda tiene la ventaja de integrarse con otros servicios propios de Amazon como API Gateway o S3 [16].

En cuanto a latencia, AWS Lambda se muestra muy competente y tiene un rendimiento bastante optimizado. A pesar de que Google Cloud Functions o Azure Functions puede presentar ventajas, no tiene una diferencia determinante para la realización de este trabajo.

Respecto al coste, los tres presentan planes gratuitos similares con modelos de pago por uso que en ningún caso van a provocar problemas en el proyecto [16].

En cuanto a visibilidad del tráfico y monitorización, AWS proporciona herramientas robustas como CloudWatch [16], que permiten analizar *logs* y métricas de manera detallada. Esto facilita la observabilidad del sistema.

En definitiva, se ha optado por AWS Lambda en vez de las otras porque, según Shrestha (2019) [10], ofrece un modelo basado en eventos que suprime la necesidad de administrar la infraestructura, ofrece un entorno aislado y sin estado (*stateless*) para cada invocación, además posibilita una integración sencilla con servicios como S3, DynamoDB o SNS que tienen la capacidad de funcionar como disparadores. Estas propiedades hacen que sea una plataforma apropiada para diseñar un sistema basado en la fragmentación y ejecución modular de *malware*.

²<https://docs.aws.amazon.com/lambda/>

3.1.3. API Gateway

Amazon API Gateway es un servicio de AWS que permite a los desarrolladores crear, publicar y mantener APIs de forma escalable y segura. Actúa como la «puerta de entrada» para que las aplicaciones externas se comuniquen con los servidores de Amazon.³

En este proyecto, se utiliza para exponer las funciones Lambda a través de un *endpoint*⁴ HTTP. Esto permite que el cliente (la víctima) realice peticiones a la nube y esta comprenda lo que se le pide. La estructura se puede observar en la figura 3.1.

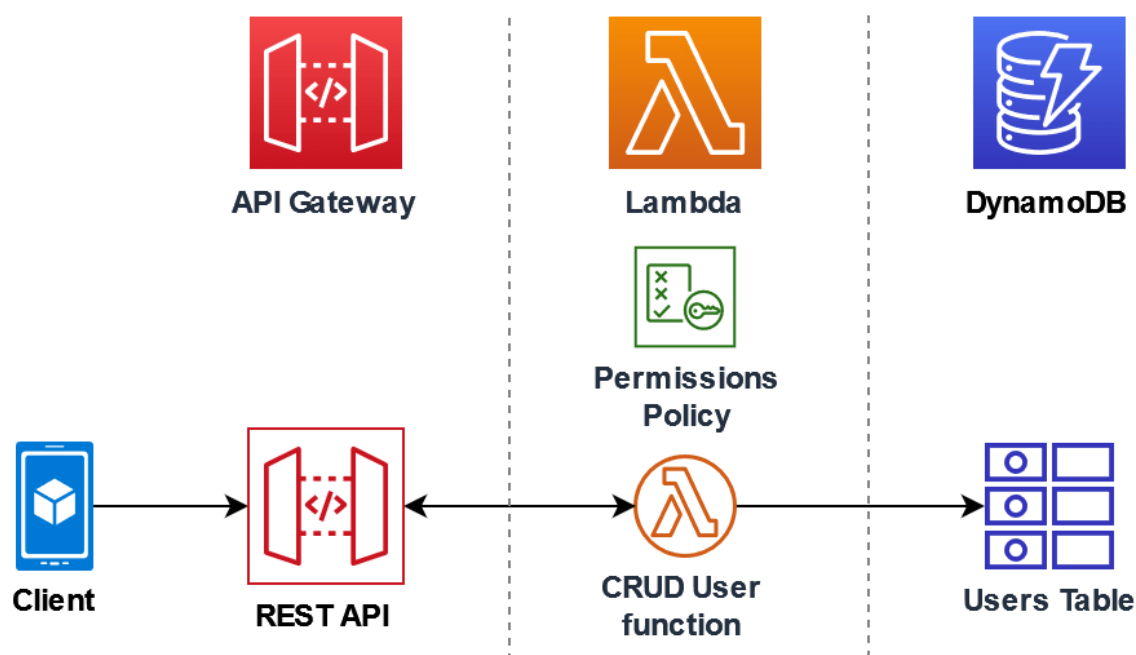


Figura 3.1: Ejemplo del uso de API Gateway con AWS Lambda⁵.

Según Shrestha (2019) [10], su uso permite gestionar el tráfico y las autorizaciones sin necesidad de configurar un servidor web (con una instancia EC2, por ejemplo). Para xTyphonTC, esto es clave porque el tráfico de red generado hacia un dominio legítimo de Amazon (execute-api.amazonaws.com) es mucho menos sospechoso para un cortafuegos que una conexión directa a una IP desconocida.

3.1.4. S3

Amazon S3 es un servicio de almacenamiento de objetos que ofrece escalabilidad, disponibilidad de datos y seguridad. Se organiza mediante

³<https://docs.aws.amazon.com/apigateway/>

⁴Un *endpoint* es una «puerta de entrada» que utilizan los sistemas para acceder a información. Por ejemplo, las aplicaciones web utilizan una url para acceder a un recurso específico, dicha url indica que puerta o *endpoint* hay que atravesar para conseguir dicho recurso.

buckets donde se depositan los archivos.

Se emplea como un *buffer* intermedio y temporal para almacenar los fragmentos y el fichero final de la aplicación.

Se ha elegido por su naturaleza orientada a eventos. Tal y como indica la documentación oficial de AWS ⁶, la subida de un archivo puede disparar automáticamente una función Lambda. Esta característica permite que en cuanto un fragmento seleccionado (por ejemplo, el último) llega al *bucket*, el sistema pueda despertar una función Lambda que vaya a utilizarla. Un ejemplo de uso de S3 con AWS Lambda se puede apreciar en la figura 3.2.

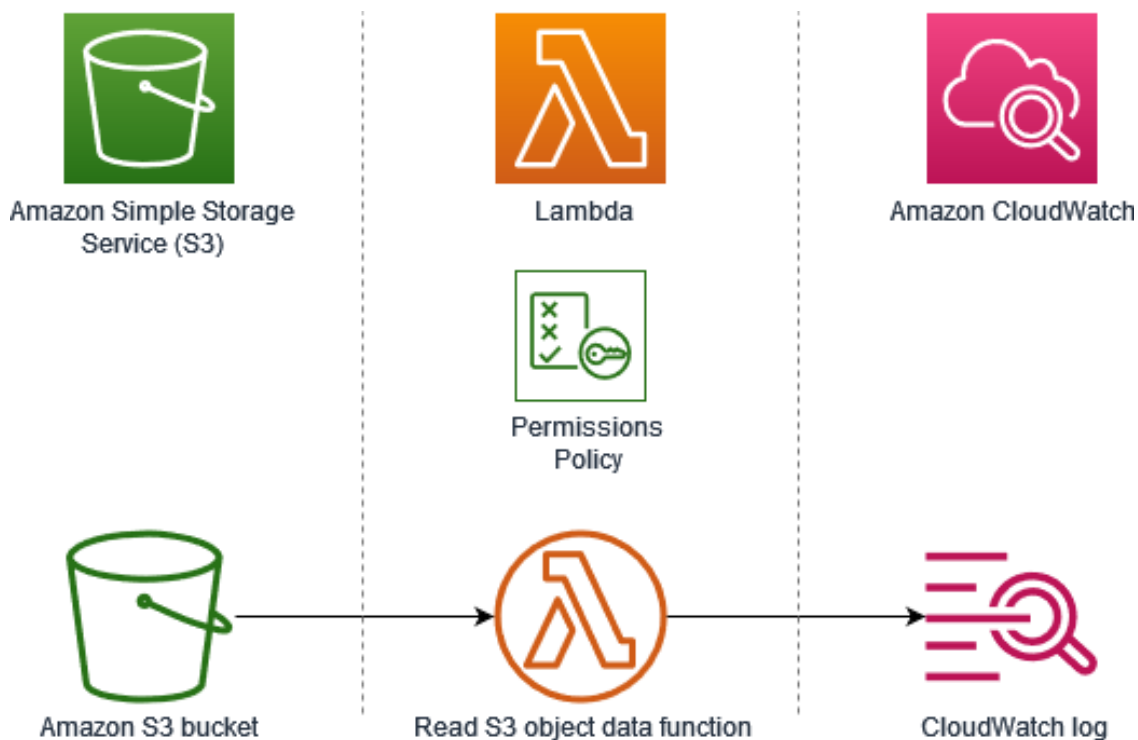


Figura 3.2: Ejemplo del uso de S3 con AWS Lambda ⁷.

3.1.5. PyInstaller

PyInstaller es una herramienta que empaqueta aplicaciones de Python en ejecutables independientes. Agrupa el *script* junto con todas sus dependencias y el intérprete de Python en un solo archivo .exe. ⁸

Se ha utilizado para transformar el código del *Loader* en un artefacto ejecutable distribuable.

⁶<https://docs.aws.amazon.com>

⁸<https://pyinstaller.org/en/stable/>

3.2. Tecnologías «de la víctima»

En este apartado se describen las herramientas que conforman el entorno objetivo. Para que la simulación de un ciberataque sea rigurosa y segura, no se puede ejecutar el código en el equipo de desarrollo; es fundamental contar con un entorno controlado que replique las condiciones de un equipo real. Esto implica el uso de sistemas aislados donde se puedan monitorizar los efectos del *malware* sin comprometer la integridad del sistema anfitrión.

3.2.1. VirtualBox

Virtualbox es un software de virtualización gratuito y de código abierto para generar máquinas virtuales con instalaciones de sistemas operativos ⁹.

El papel de esta tecnología en el proyecto es fundamental: es el entorno seguro que simula el sistema de la víctima que va a recibir el ataque de los fragmentos sin riesgo para el equipo real.

Se ha escogido esta tecnología por ser de código abierto, fácil de usar, por tener compatibilidad con múltiples sistemas operativos y ser una de las tecnologías más utilizadas durante la carrera.

3.2.2. Administrador de tareas

El Administrador de tareas es una aplicación integrada en los sistemas operativos Windows que proporciona información detallada sobre los programas y procesos que se ejecutan en el equipo, así como el estado de la memoria RAM y el uso de la CPU.

En este proyecto se utiliza para monitorizar el proceso RunTimeBroker.exe (el afectado por el *malware*) y observar el impacto de la ejecución del *Loader* en los recursos del sistema.

Al ser la herramienta de monitorización por defecto para cualquier usuario de Windows, su uso es esencial para demostrar que el artefacto no genera anomalías visibles que pudieran alertar a una víctima no técnica.

3.2.3. Antivirus (Windows Defender, AVG, Avast, Panda)

Representan el conjunto de soluciones de seguridad encargadas de detectar, prevenir y eliminar *malware* en el equipo de la víctima.

⁹<https://www.virtualbox.org>

Actúan como la alarma en la fase de evaluación, y se han utilizado de uno en uno.

La inclusión de múltiples proveedores permite obtener una métrica objetiva sobre la eficacia de la fragmentación y la inyección en memoria frente a los estándares de seguridad actuales de la industria.

3.2.4. Wireshark

Wireshark es el analizador de protocolos de red más utilizado del mundo. Permite capturar y examinar el tráfico que atraviesa una interfaz de red en tiempo real.¹⁰

Se utiliza para monitorizar el tráfico generado por el *Loader*. Además, se observa el contenido de los fragmentos y las peticiones hacia los *enpoints* de Amazon. Por último, también sirve para demostrar que los fragmentos del *malware* viajan cifrados de extremo a extremo.

Se ha elegido por su capacidad para desglosar la pila de protocolos TCP/IP y filtrar sus diferentes protocolos, mostrando información importante como las direcciones IP o los número de puerto. Además, al tratarse de un programa gratis y de código abierto lo hace fiable y seguro.

3.2.5. Burp Suite (Burp Proxy)

Burp Suite es una plataforma integrada para realizar pruebas de seguridad de aplicaciones web. Su componente principal, Burp Proxy, permite interceptar y modificar el tráfico HTTP/S entre el cliente y el servidor.¹¹

En el proyecto se ha empleado para realizar pruebas de interceptación activa (*Man-in-the-Middle*). Su objetivo es inspeccionar los paquetes que contienen los fragmentos.

Se ha utilizado ya que es la herramienta estándar en auditorías de seguridad, permitiendo evaluar si un analista podría reconstruir el *malware* simplemente escuchando la red, demostrando así la eficacia del cifrado y la autenticación del canal.

3.2.6. Proxifier

Proxifier es una herramienta que permite utilizar cualquier aplicación con un *proxy*. Esta herramienta puede enrutar cualquier conexión de red

¹⁰<https://www.wireshark.org/>

¹¹<https://portswigger.net/burp/documentation/desktop/tools/proxy>

de una aplicación a través de un servidor *proxy*, proporcionando una vía para redirigirlas.

Se utiliza para forzar al *Loader* a atravesar el Burp Proxy.

Su uso es esencial para el análisis de *malware* que intenta conectar directamente a IPs o dominios remotos (como AWS) sin respetar los ajustes de *proxy* del sistema.

3.3. Misceláneo

En este apartado se explica aquellas tecnologías que no se pueden asignar fácilmente a ninguno de los entornos anteriores debido a que son de propósito general o tienen otro objetivo en el proyecto.

3.3.1. Visual Studio Code

Visual Studio Code (VS Code) es un editor de código fuente desarrollado por Microsoft. Soporta una gran cantidad de lenguajes y cuenta con un ecosistema de extensiones que facilitan el desarrollo.

Se ha empleado como el entorno de desarrollo principal para la escritura de todos los scripts del proyecto. Esto incluye tanto el código del *Loader* (destinado a ejecutarse en la máquina víctima) como la lógica de las funciones Lambda, facilitando la preparación del código antes de su despliegue manual en AWS.

Su elección se justifica por su compatibilidad con el lenguaje Python y su bajo consumo de recursos. Además de la capacidad de trabajar con múltiples archivos de forma simultánea permitiendo mantener la coherencia entre los diferentes módulos del sistema de xTyphonTC. Por último, su terminal integrada facilita las pruebas rápidas de sintaxis, asegurando que el código no contenga errores antes de ser trasladado al entorno de ejecución de AWS Lambda.

3.3.2. LaTeX

LaTeX es un sistema de composición de textos orientado a la creación de documentos escritos de alta calidad¹².

Se emplea en la creación de la documentación, siendo elegido por ser un software libre y de código abierto, por su facilidad en el trabajo colaborativo y por estar orientado a la escritura, no debiendo preocuparse el escritor por el formato. Además, proporciona herramientas avanzadas

¹²<https://www.latex-project.org>

para la gestión de citas, figuras y ecuaciones, características esenciales en un documento de índole académica.

3.3.3. GitHub

GitHub es una plataforma basada en la nube donde se puede almacenar, compartir y trabajar junto con otros usuarios para escribir código¹³.

En este trabajo, se emplea como repositorio principal tanto para la documentación como para el código, permitiendo así la entrega de ambos: <https://github.com/ri3car1do4/xTyphonTC/>

Su extenso uso en el campo académico y profesional lo convierten en una herramienta apropiada para la gestión del proyecto.

¹³<https://www.github.com>

Capítulo 4. Diseño de la Solución

A diferencia de los *malware* tradicionales, en los que el artefacto ejecutable suele residir íntegramente en un servidor o en el sistema comprometido, la solución propuesta simula un mecanismo de entrega fragmentada apoyado en servicios *serverless*. Desde el punto de vista del diseño, pueden distinguirse tres bloques principales:

- el cliente (*Loader*);
- la capa de transporte; y
- la capa lógica.

4.1. Primera solución

La arquitectura propuesta se divide en dos entornos principales, derivados de la naturaleza fragmentada antes descrita de la solución: el cliente y la nube.

La infraestructura en la nube se encarga de la fragmentación del *malware*, dividiendo el código en varias funciones independientes. La viabilidad del uso de tecnologías *serverless* para la manipulación de *malware* se fundamenta en trabajos previos como la tesis de Ponomarev (2022) [14].

En el entorno del cliente, es necesario reconstruir el código antes de su ejecución, además de invocar a las funciones generadoras que tienen el código. Para ello se implementa el *Loader*, un componente encargado de invocar a las funciones, descargar sus fragmentos, ensamblarlos y ejecutar en memoria el código final. La víctima debe interactuar con el *Loader* para que se inicie el procedimiento. El flujo secuencial de este proceso se puede apreciar en la figura 4.1.

4.1.1. Diseño de la lógica central

Cliente (*Loader*)

Es el encargado de todo el proceso principal del sistema. Invoca a las funciones generadoras de fragmentos, se descarga los mismos y los ensambla. Además, ejecuta el código final ensamblado.

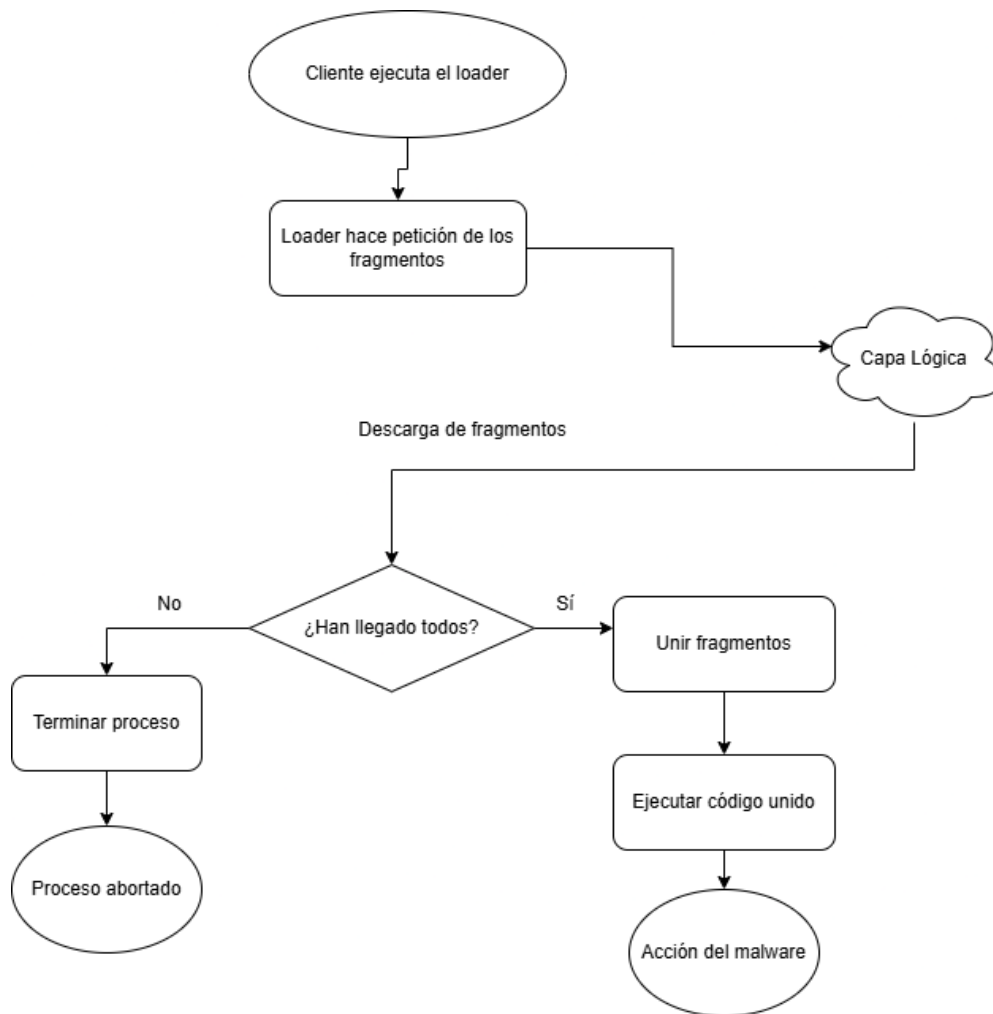


Figura 4.1: Proceso de interacción entre cliente y capa lógica en la primera versión de xTyphonTC desde el punto de vista del cliente.

Únicamente ensamblará el código si todos los fragmentos (un número n especificado en su código) han conseguido ser descargados.

Capa de transporte

Se encarga de actuar como el punto de enlace entre el *Loader* y la infraestructura en la nube. Su función principal es proporcionar una interfaz pública y segura que permite la recepción de señales de activación por parte del cliente y la entrega controlada de los fragmentos.

- Permite que el sistema sea independiente del protocolo utilizado, centrando su responsabilidad en comunicarse con la lógica de generación.
- Actúa como la frontera del sistema, validando las peticiones para asegurar que solo los usuarios autorizados puedan interactuar con la lógica.

Capa lógica

Se trata de las funciones que generan los fragmentos del *malware* que posteriormente ensamblará el *Loader*. Su objetivo es únicamente el de entregar este código.

- Cada unidad lógica (función) tiene acceso solo a una porción específica del código final. Esto garantiza que, en caso de que una de las comunicaciones sea interceptada, el analista de seguridad solo obtenga un fragmento incompleto e inoperativo por sí mismo.
- La capa lógica no entrega el código en texto plano sino que transforma el fragmento en un formato apto para el transporte y añadir una capa que dificulte la detección durante la descarga.
- Las funciones no mantienen un estado entre ellas (*stateless*), y actúan de forma atómica y bajo demanda.

4.1.2. Requisitos y restricciones

El sistema tiene una serie de requisitos funcionales y restricciones. Por parte de los requisitos, existen los siguientes:

- El sistema no debe descargar archivos binarios completos que puedan ser detectados por el antivirus de la víctima.
- El *Loader* debe ser capaz de solicitar y descargar cada fragmento de código desde la nube.
- El sistema debe garantizar que el código solamente se ejecute cuando se hayan recibido todos los fragmentos.
- El *Loader* debe incluir la lógica necesaria para ensamblar los fragmentos descargados previamente y ejecutarlo.
- Las funciones deben ser capaces de generar y entregar los fragmentos ofuscados de forma independiente y bajo petición del *Loader*.
- El *Loader* debe permitir la decodificación y ensamblado correcto de los fragmentos en el cliente.

En cuanto a las restricciones de esta versión:

- El sistema requiere conectividad a Internet para la descarga de fragmentos.
- El número de fragmentos n está predefinido en el código del *Loader*.
- Las funciones son independientes, lo que limita la coordinación entre fragmentos en la nube.
- El código está íntegro en el *Loader*, por lo que si la defensa lo intercepta puede descubrir cuántos fragmentos hay y dónde se encuentran.

4.2. Segunda solución

A pesar de que la aproximación anterior cumple su propósito, presenta una alta carga de funcionalidades para el *Loader*. Esto incrementa su tamaño y la posibilidad de ser detectado en el entorno de la víctima. Para tratar de mitigar esto, se ha diseñado una segunda solución en la que se traslada las funciones no esenciales del *Loader* a la nube.

De las tareas del *Loader*, la invocación, descarga y ejecución tienen que mantenerse en el lado del cliente, por lo que la fase de orquestación se externaliza. De esta forma, los fragmentos de código se unen en la nube y el *Loader* se convierte en un cliente ligero cuya única responsabilidad es descargar el código final y ejecutarlo. Esto garantiza la atomicidad del proceso, pues el lado del cliente obtiene el código funcional en su totalidad o, en caso de fallo, no recibirá nada.

Tras la interacción inicial, el *Loader* se limita a invocar una nueva función, que activa la lógica en la nube, y se queda en estado de espera hasta recibir el código ensamblado, que posteriormente ejecutará, tal y como se aprecia en la figura 4.2.

Al recibir la petición del *Loader*, la función activadora desencadena la ejecución de las funciones que tienen el código fragmentado. Como en esta solución los fragmentos no se transmiten directamente al cliente, los fragmentos se depositan en una nueva capa, la capa de almacenamiento, permitiendo su acceso posterior por parte de una función orquestadora.

Posteriormente, la función orquestadora verifica la llegada de los fragmentos (por ejemplo, al llegar el último), los ensambla y guarda el código resultante. Este almacenamiento intermedio es estrictamente necesario porque la función orquestadora carece de canal de comunicación directo para enviar el código ensamblado al *Loader* y por otro lado, el

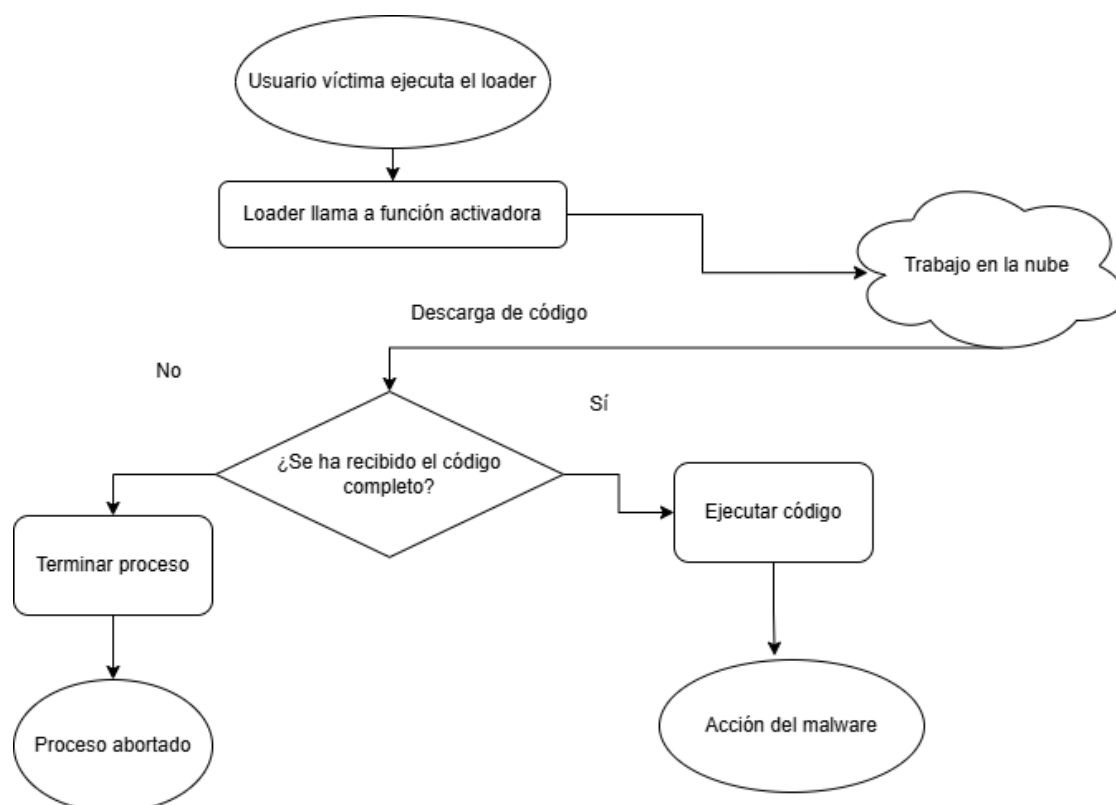


Figura 4.2: Flujo de acciones del *Loader* de la capa cliente y su interacción con la capa lógica de la nube.

Loader no puede llamar a esta función orquestadora porque se dispara automáticamente al aparecer el último fragmento.

Finalmente, una función de descarga, invocada por el *Loader*, recupera el código completo almacenado en el *buffer* (capa de almacenamiento) y lo trasmite al cliente. Como medida adicional, esta función elimina todo rastro de código en el almacén para evitar posibles detecciones.

El conjunto de estas operaciones se puede apreciar en la figura 4.3.

4.2.1. Diseño de la lógica central

Cliente (*Loader*)

Actúa como el motor de la ejecución local. Su función es conectarse con la capa de transporte para comunicarse con la capa lógica, que le envía el código resultante el cuál tendrá que ejecutar.

El cliente delega la reconstrucción del código a la nube. Su única responsabilidad es gestionar las peticiones y recepciones, y su posterior ejecución en memoria. Se utiliza un sistema de reintentos para controlar el tráfico de red y evitar posibles errores de latencia en las comunicaciones

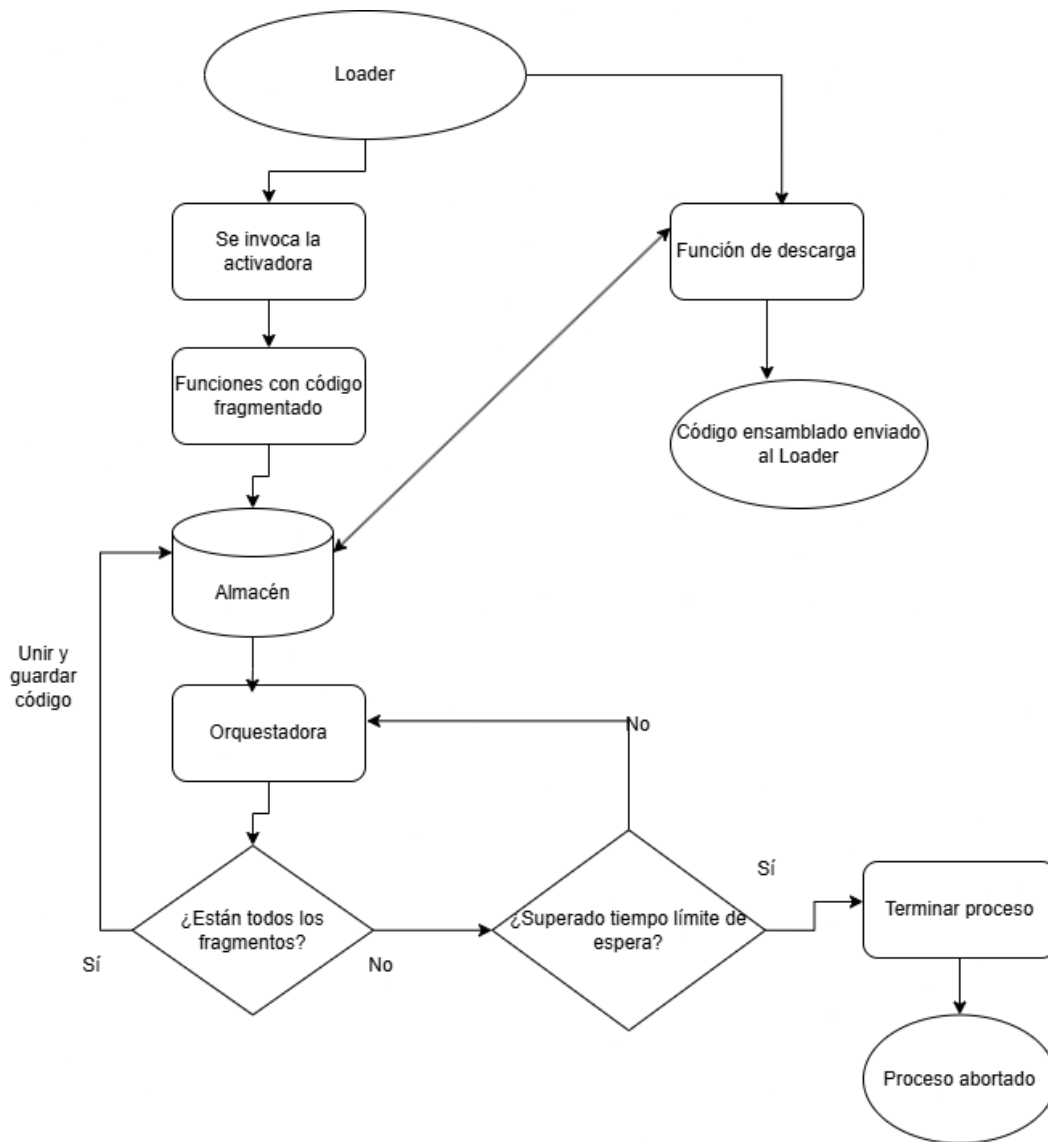


Figura 4.3: Flujo de acciones que ocurren en la nube tras ejecutarse el *Loader*.

con la nube.

Capa de Transporte

La capa de transporte en esta solución mantiene la infraestructura de interfaz basada en una pasarela de enlace, siguiendo los principios definidos en la solución 1 (véase sección 4.1.1). Sin embargo, la interfaz ahora recibe solo dos peticiones, optimizando el número de interacciones entre el cliente y la nube:

- La activación que activa la lógica de ensamblado en la nube.
- La descarga del *payload* final reconstruido.

Capa de Almacenamiento

Dicha capa se ha diseñado no como un almacén persistente, sino como un *buffer* intermedio donde se almacenan los fragmentos hasta su posterior ensamblado, y se deja el resultado a la espera del envío hacia el cliente. Para que sea temporal, se eliminan todos los ficheros después de su procesamiento.

Capa lógica

- **Algoritmo de Activación:** Este módulo utiliza un algoritmo que al recibir una señal, no procesa datos, sino que activa múltiples módulos que entregan fragmentos al *buffer* anterior.
- **Algoritmo de Ensamblado:** Monitoriza el estado del *buffer*, activándose ante la escritura de ciertos datos. Actúa para unir los fragmentos y entregar al *buffer* el resultado para su posterior envío. Después de ensamblar todos los fragmentos los elimina del *buffer*.
- **Estrategia de Descarga, Limpieza y Sigilo:** Es el encargado de monitorizar el *buffer* para buscar el resultado, borrarlo y enviarlo al cliente. Para ello se aplica una regla de «Lectura Destructiva»:
 1. Identificar el recurso solicitado por el *Loader*.
 2. Cargar el recurso en la memoria volátil de la función.
 3. Emitir orden de eliminación física en la Capa de Almacenamiento.
 4. Transmitir datos al *Loader*.

4.2.2. Requisitos y restricciones

Los requisitos de esta segunda solución son:

- El sistema debe permitir la orquestación y ensamblado de fragmentos en la capa lógica, delegando esta responsabilidad fuera del *Loader*.
- El *Loader* debe ser capaz de invocar una función activadora, recuperar el código completo ensamblado y ejecutarlo.
- Las funciones generadoras de fragmentos deben poder generar y depositar fragmentos en la capa de almacenamiento intermedia.
- El sistema debe garantizar que el ensamblado únicamente ocurra cuando todos los fragmentos estén disponibles.

- La capa de almacenamiento debe permitir el acceso temporal a los fragmentos por parte del orquestador, así como permitir que el mismo introduzca el código final ensamblado.
- La comunicación entre capas debe realizarse a través de la capa de transporte.

En cuanto a las restricciones:

- La solución depende de servicios en la nube ajenos al control del usuario (atacante).
- El sistema requiere total sincronización entre funciones.
- El *Loader* depende totalmente de la red y requiere conectividad a Internet.
- La descarga del código final ensamblado puede provocar la detección por parte de los sistemas de defensa de la víctima.

Capítulo 5. Desarrollo y Arquitectura

Tras definir el plano conceptual de las soluciones en el capítulo anterior, procede materializar dicho diseño mediante las tecnologías descritas en el capítulo 3. En consecuencia, este capítulo expone las arquitecturas de *software* y de despliegue empleadas, su relación con el diseño lógico propuesto y los principales problemas surgidos durante la implementación, así como las decisiones adoptadas para resolverlos.

Todo el código fuente completo de la aplicación está disponible en el siguiente repositorio de GitHub:

<https://github.com/ri3car1do4/xTyphonTC/>

5.1. Primera solución

5.1.1. Arquitectura de *software* empleada

Patrón Arquitectónico Principal

En esta primera solución el sistema se basa en un modelo de orquestación descentralizada, donde la lógica de control y ensamblado reside únicamente en el cliente. En esta arquitectura, la nube solo se comporta como un proveedor de recursos atómicos e independientes.

Este diseño adopta un modelo en el que el *Loader* es el responsable de gestionar la descarga, reconstrucción y ejecución de los fragmentos:

1. El cliente pide los fragmentos a la nube.
2. Descarga los fragmentos.
3. Si están todos los ensambla; si no, aborta.
4. Ejecuta el código ensamblado.

La infraestructura en la nube se mantiene simple. Cada función Lambda opera de forma independiente, desconociendo la existencia de los demás fragmentos y limitándose a entregar su código.

Este enfoque prioriza la simplicidad y reduce los costes operativos en la nube, aunque traslada mayor carga computacional y de red al equipo de la víctima.

Componentes Principales

El sistema presenta una estructura simple donde la lógica está distribuida entre el cliente y las unidades de generación atómicas. Los componentes principales son:

- **Cliente (*Loader*):** Es el componente crítico que asume las funciones de orquestación, control de flujo y ejecución final. Sus responsabilidades incluyen:
 1. Realizar peticiones individuales y secuenciales a los generadores de fragmentos.
 2. Gestionar el almacenamiento temporal de los segmentos en la memoria local.
 3. Ejecutar el algoritmo de reconstrucción una vez verificado el número total de fragmentos (n).
- **Generadores de Fragmentos (*Ladon*, *Orthrus*, *Hydra*):** Son funciones *serverless* independientes que actúan como almacenes dinámicos de código. Su diseño es reactivo y sin estado. Sus funciones se limitan a:
 1. Recibir la invocación directa (vía API Gateway).
 2. Aplicar una codificación básica (como Base64) al segmento de código que custodian para evitar la detección por firmas durante el transporte.
 3. Entregar el fragmento como cuerpo de la respuesta HTTP.
- **Interfaz de Transporte (*Endpoint*):** Actúa como la pasarela de comunicación que expone las funciones generadoras al exterior. En esta solución, simplemente mapea las peticiones del *Loader* hacia la función Lambda correspondiente.

Mecanismos de Comunicación

En esta arquitectura, la comunicación es simple, el *Loader* establece conexiones HTTPS síncronas individuales con el *endpoint* de la API, que enviará la petición convertida en un evento hacia las funciones Lambda y devolverá los fragmentos en formato JSON al cliente.

5.1.2. Infraestructura, procesos clave y flujo de datos

Entorno de despliegue

- **Parte del cliente:** Se ha configurado una máquina virtual en VirtualBox con Windows 11 Pro, configurada con 4 GB de RAM y 2 CPUs para emular un equipo de oficina estándar. Además, se ha configurado el adaptador de red para tener acceso a Internet de dos formas:
 - NAT: Crea una red privada para la máquina virtual y usa la IP del *host* para salir a internet.
 - Adaptador Puente: La máquina virtual intenta obtener su propia IP directamente del *router*. Es la opción usada cuando el *host* utiliza Ethernet.

El *Loader* se ejecuta en un espacio de usuario sin privilegios. Se ha deshabilitado el portapapeles y las carpetas compartidas.

- **Nube:** La infraestructura de despliegue utilizada se encuentra en la región us-east-1 de AWS, seleccionada por ser la región con mayor disponibilidad de servicios y menor latencia global. Se utiliza el modelo FaaS (*Function as a Service*), lo que significa que no es necesario mantener una infraestructura muy robusta ya que dicha responsabilidad cae sobre las infraestructuras de Amazon. [10]

Además, las funciones Lambda se ejecutan solamente en un tiempo mínimo (en milisegundos), lo que significa que no hay procesos en ejecución de forma permanente que puedan ser detectados por herramientas de escaneo de infraestructura en la nube.

Por otro lado, los costes operativos son muy buenos para este TFG, ya que permiten ejecutar miles de pruebas de ensamblado con un coste irrisorio, aprovechando los créditos gratuitos que ofrece AWS.

Topología de red y arquitectura

- **Conexión externa a AWS:** El nodo inicial de todas las transacciones de red es la máquina víctima, desde la cual el cliente establece múltiples conexiones síncronas hacia la nube. Dichas transacciones están canalizadas a través del puerto 443 y el protocolo HTTPS, comunicándose con API Gateway.

- **Conexión en AWS (interna):** En el entorno de despliegue de AWS, la topología no tiene conexión interna. Las funciones Lambda actúan como nodos finales completamente aislados e inconexos entre sí, accesibles de forma directa y exclusiva a través de sus *endpoints* públicos de API Gateway.

Desde la perspectiva de la seguridad y la evasión, esta topología distribuida resulta altamente efectiva, pues se permite camuflar el proceso de recolección de los fragmentos dentro de tráfico web legítimo alojado dificultando su detección y bloqueo.

Para finalizar y para resumir, la arquitectura tiene como nodo clave de comunicación a API Gateway, mientras que el *Loader* se encarga de hacer peticiones para que se comunique con las funciones Lambda y se produzca la transacción de datos. Toda esta topología o arquitectura se puede apreciar en la figura 5.1

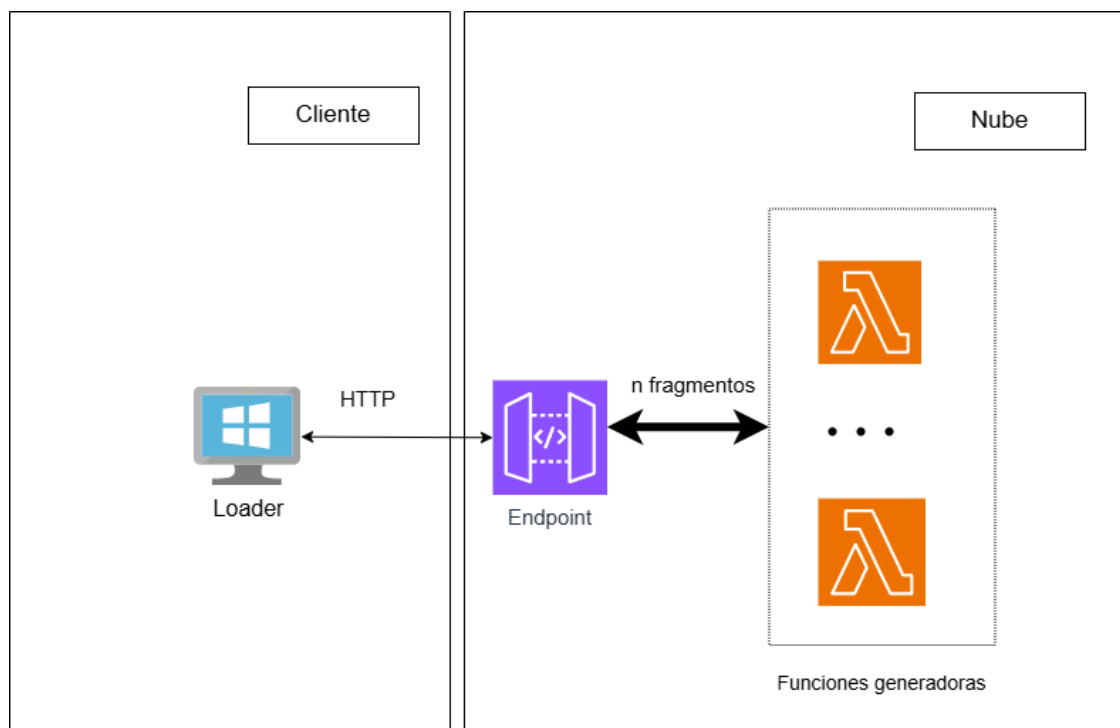


Figura 5.1: Arquitectura de la primera solución, donde se aprecia la comunicación entre el cliente y la nube a través de API Gateway.

Configuración de servicios clave

- **API Gateway:** Se ha configurado una API de tipo REST para actuar como interfaz de entrada. Se han definido rutas específicas para cada función generadora, en este caso 3 (*Ladon*, *Orthrus*, *Hydra*), como se puede apreciar en la figura 5.2.



Figura 5.2: Configuración árbol de recursos en API Gateway.

- **AWS Lambda:** Las funciones se han configurado bajo un modelo de ejecución sin estado utilizando el entorno de ejecución *Python 3.9*. Las funciones generadoras devuelven un JSON donde el campo *'body'* contiene otro JSON. El contenido, el fragmento de código (ya sea en formato binario o de alto nivel), se encuentra dentro del campo *'body'*, serializado como JSON y codificado en Base64, como se puede observar en la figura 5.3.

```
{
  'statusCode': 200,
  'body': json.dumps({
    "module_id": X,
    "content": encoded,
    "is_bin": X
  })
}
```

Figura 5.3: Estructura del JSON enviado por una función generadora [?].

Flujo de datos

El ciclo de vida de los datos se divide en tres fases secuenciales lideradas por la capa de cliente:

1. **Peticiones salientes a la nube:** El flujo de datos se inicia en el entorno cliente, cuando el *Loader* genera las peticiones de información a la infraestructura de AWS. Como modo de evadir la detección de patrones, el cliente emplea una aleatorización que altera el orden con el que se pide cada fragmento en cada ejecución. Todas las peticiones se canalizan hacia el exterior mediante el puerto 443 mediante conexiones HTTPS dirigidas a los *endpoints* públicos de API Gateway, camuflando las peticiones maliciosas como tráfico legítimo.

2. **Tránsito y encapsulamiento:** En la fase de respuesta, el flujo de datos viaja desde nube hacia el equipo de la víctima (cliente). Para garantizar la integridad del código y evitar que se detecte firmas de *malware* en texto plano, la carga útil de los fragmentos viaja codificada en Base64. Además, la respuesta de las funciones Lambda está estructurada en formato JSON con el código codificado en el cuerpo del mensaje.
3. **Decodificación, orquestación y ejecución:** La fase más importante del flujo ocurre cuando los datos se hallan en el cliente y se transfieren directamente a la memoria RAM del equipo de la víctima, sin interactuar en ningún momento con el sistema de archivos local. El *Loader* intercepta las respuestas, deserializa los objetos JSON y aplica una decodificación Base64 ^{1 2}

A medida que los fragmentos llegan (en orden aleatorio), el *Loader* los almacena temporalmente en un diccionario. Una vez han llegado todos los fragmentos, los extrae y los concatena siguiendo su orden original predefinido. Finalmente, este texto concatenado se convierte en código ejecutable y se ejecuta mediante la función *exec()*, materializando la infección.

5.1.3. Integración de las tecnologías

Interacción externa y librerías del cliente

Para establecer la comunicación entre la máquina víctima y la nube, el *Loader* implementa un mecanismo basado en peticiones web para recopilar los fragmentos utilizando el módulo *urllib.request*. Esta librería pertenece a la biblioteca estándar de Python, por lo que se prescinde de confiar en librerías de terceros ³. Esta estrategia de diseño sigue la filosofía de *Zero Dependency*, una técnica de evasión utilizada en los ciberataques que favorece la autonomía del código y facilita que se oculte en el sistema comprometido.

El cliente realiza invocaciones dirigidas a los *endpoints* de cada fragmento en la nube. A diferencia de un sistema de descargas convencional, el contenido descargado no se escriben en el sistema de archivos local, sino que el *Loader* lo guarda directamente en variables dentro de la memoria RAM.

¹<https://docs.aws.amazon.com/lambda/latest/dg/python-handler.html>

²<https://docs.python.org/es/3/library/base64.html>

³<https://docs.python.org/3/reference/index.html>

Para realizar su función, el *Loader* tiene que implementar una serie de librerías aparte de la ya mencionada `urllib.request`:

- **Base64:** Es utilizada por el *Loader* para decodificar el contenido que viene codificado en Base64 desde las funciones Lambda.
- **JSON:** Empleado para la extracción de la información, pues viene desde la nube en dicho formato.
- **Random:** Usado como técnica de evasión para evitar ser detectado por generar el mismo patrón al solicitar los fragmentos. Es decir, va a solicitar los fragmentos de forma aleatoria y no siempre en el mismo orden para conseguir que los analistas de seguridad no puedan identificar un patrón de tráfico de red idéntico y secuencial en todos los casos.

Permisos de accesos

En esta solución, la gestión de identidades y accesos (IAM) se rige por el Principio de Mínimo Privilegio para minimizar la superficie de exposición. Así, cada componente tiene como únicas capacidades las necesarias para conseguir su propósito.

Los permisos de estructuran de la siguiente manera:

- **Amazon API Gateway:** Es el único componente de la arquitectura que requiere permisos específicos. Se le asigna de forma estricta el permiso `lambda:InvokeFunction`, limitando su alcance mediante el Amazon Resource Name (ARN)⁴ específico de cada función Lambda. Eso evita que el *gateway* pueda ser utilizado para disparar otros recursos ajenos⁵.
- **Generadores de fragmentos:** Dado que su única tarea es devolver una cadena de texto o JSON al cliente, sus políticas de acceso respecto a otros servicios de AWS están vacías. Es decir, no tienen permisos de escritura, lectura o invocación interna.
- **Loader:** El *Loader* está diseñado de forma que no necesite integrar las credenciales de AWS, accediendo a los recursos realizando peticiones a través de los *endpoints* públicos del API Gateway.

⁴Un Amazon Resource Name (ARN) es un identificador único que se utiliza en AWS para referenciar de manera inequívoca a un recurso específico (como una función Lambda, un *bucket* de S3 o un rol de IAM).

⁵<https://docs.aws.amazon.com/lambda/latest/dg/services-apigateway-tutorial.html>

Formato de transferencia y ensamblado local

Con el objetivo de asegurar la integridad del código de los fragmentos y evitar la detección por firmas de elementos como antivirus, la transferencia de estos fragmentos necesita encapsulamiento. De esta forma, las funciones no devuelven texto plano, sino que envían la respuesta en formato JSON, encapsulando codificando cada fragmento de código en Base64 ⁶.

El *Loader* es el encargado de procesar esta información y orquestar los fragmentos. Extrae el contenido del JSON que le ha llegado por parte de una función Lambda, lo decodifica y concatena el fragmento decodificado con el resto de fragmentos de forma secuencial. De esta forma se sigue el orden del *malware* original.

5.2. Segunda solución

5.2.1. Arquitectura de *software* empleada

Patrón Arquitectónico Principal

El sistema sigue una arquitectura *serverless* basada en funciones y que utiliza servicios de AWS, concretamente AWS Lambda.

En este diseño, cada función Lambda encapsula una responsabilidad específica dentro del flujo de acciones, favoreciendo la modularidad y separación de responsabilidades de la solución.

Además, el sistema adopta un modelo de cliente ligero (*thin client*) en el que el cliente o *Loader* se convierte únicamente en un intermediario que se encarga de:

1. Iniciar la ejecución llamando a la función activadora.
2. Descargar el código ensamblado y en caso de no recibirlo, actuar en consecuencia.
3. Ejecutar el resultado recibido.

Así, una gran parte de la lógica de negocio ocurre en el *backend serverless*, mientras que el cliente tiene acciones limitadas.

La coordinación entre componentes se realiza por invocaciones entre funciones y almacenando mediante S3. Esto permitirá que el sistema

⁶<https://docs.python.org/es/3/library/base64.html>

actúe como un conjunto de servicios independientes que colaboran para ensamblar y conseguir que el código final pase desapercibido para el antivirus del cliente.

Por último, dado que el cliente no tiene por qué conocer los detalles de implementación en el *backend*, se descartó la idea de invocar a las funciones Lambda con código fragmentado directamente desde el *Loader*. En su lugar, se hace uso del patrón fachada, implementado en esta solución como la función activadora, que será la encargada de invocar al resto de funciones Lambda.

Componentes Principales

El sistema está muy desacoplado y se divide en los siguientes componentes principales:

- **Cliente (*Loader*):** Es la aplicación ejecutada por el usuario en su equipo. Sus funciones están descritas con anterioridad y no tiene grandes responsabilidades, siendo un cliente ligero, como se muestra en la figura 5.4.
- **Lambda Invocadora (*Campe*):** Se trata de una función inicial que actúa como fachada y punto de entrada al procesamiento en el *backend*. Sus responsabilidades son:
 1. Recibir la petición del *Loader*.
 2. Comenzar la ejecución del sistema invocando a las funciones Lambda que poseen fragmentos de código.
- **Generadores de Fragmentos (*Cerberus*, *Scilla*, *Chimaera*):** Son las funciones encargadas de generar fragmentos de *malware*. El código de un *malware* se fragmentará en distintas funciones Lambda haciendo que pase desapercibido, pues el código dividido es únicamente código. Es decir, cada fragmento no muestra indicios de ser malicioso por sí solo. Las funciones de estas Lambdas son:
 1. Ejecutar lógica específica de ofuscación del fragmento de código.
 2. Persistir el código en un almacén S3.
- **Lambda Orquestadora o de ensamblado (*Echidna*):** Se trata de el componente principal del *backend*. Se activa al existir en el almacén la última Lambda de procesamiento y es el componente central de coordinación del sistema, encargándose de labores fundamentales en el funcionamiento del sistema:

```

// Configuración de intentos para el sondeo
intentos ← 0
max_intentos ← 6
fin ← FALSE

WHILE intentos < max_intentos AND !fin DO
  // Paso 1: Activación del backend (Solo en el primer intento)
  IF intentos == 0 THEN
    respuesta_campe ← PeticionHTTP_GET(Endpoint_Invocadora)

    IF respuesta_campe.codigo != 200 THEN
      AbortarEjecucion()
      BREAK
    END IF

  // Paso 2: Sondeo de entrega
  ELSE
    respuesta ← PeticionHTTP_GET(Endpoint_Descargadora)

    IF respuesta.codigo == 200 THEN
      // El artefacto está listo y ensamblado
      datos_json ← ParsearJSON(respuesta.cuerpo)
      codigo_base64 ← datos_json["codigo_final"]

      // Decodificación y activación en memoria
      codigo_final ← DecodificarBase64(codigo_base64)
      EjecutarEnMemoria(codigo_final)
      fin ← TRUE

    ELSE IF respuesta.codigo == 404 THEN
      // El backend sigue orquestando, esperar
      Esperar(3 segundos)
      intentos ← intentos + 1

    ELSE
      // Error inesperado en la infraestructura
      AbortarEjecucion()
      BREAK
    END IF
  END IF
END WHILE

```

Figura 5.4: Algoritmo *thin client*.

1. Hacer *polling*⁷ para monitorizar la disponibilidad del resto de funciones Lambda de procesamiento.
2. Ensamblar los fragmentos de código dando lugar al *malware* con código unido y completo.
3. Almacenar el resultado en el *buffer* (S3).

- **Lambda Finalizadora o descargadora (Aethon):** Es la función Lambda encargada de responder a la petición del *Loader* y entregar el código final almacenado en el *buffer*. Además, también debe

⁷El *polling* es un método de comunicación en el que un sistema consulta reiteradamente a otro para verificar si hay datos nuevos, cambios o actualizaciones.

encargarse de ocultar cualquier atisbo de información que pueda dejar el código malicioso almacenado en AWS aplicando una estrategia de «limpieza destructiva».

- **Interfaz de Transporte (*Endpoint*):** : Actúa también como la pasarela de comunicación pero expone solo dos funciones Lambda: la activadora y la descargadora.

Mecanismos de Comunicación

Se ha implementado una comunicación síncrona entre el *Loader* y *API Gateway*, pues es necesaria para una espera activa de resultados desde la nube. En cambio, en la lógica en el *backend* se ha usado un modelo de comunicación asíncrona desacoplada, donde la función activadora se comunica con las funciones Lambda generadoras de código mediante el tipo de invocación *Event* de AWS [11]. Este mecanismo garantiza un consumo eficiente de recursos al no bloquear recursos esperando respuestas HTTP de otras lambdas.

- **Polling entre cliente y cloud:** Dado que HTTP es un protocolo unidireccional, es decir, el servidor no puede iniciar una conexión con el cliente, el *Loader* utiliza un sistema de sondeo o *polling* en el que realiza (hasta un número máximo de intentos) llamadas síncronas periódicas a la lambda de entrega.
- **Uso de códigos de estado:** Se hace uso de los códigos de estado del protocolo HTTP para controlar el flujo de comunicación. Se emplea el código 404 como un mecanismo para señalar un estado: lo que se busca no está aún disponible y se debe realizar un intento más para ver si se consigue encontrar. Por otra parte, el código 200 finaliza el sondeo y continúa el proceso entregando el código. El código 500 se envía cuando una generadora no es capaz de guardar su fragmento en el almacén ⁸.

5.2.2. Infraestructura, procesos clave y flujo de datos

Topología de red y Arquitectura

El sistema se despliega sobre una infraestructura híbrida que conecta el entorno del cliente con una red de servicios gestionados en la nube de

⁸<https://docs.aws.amazon.com/lambda/>

AWS. Esta topología permite desacoplar la lógica de ejecución de la lógica de procesamiento y almacenamiento, como se puede apreciar en la figura 5.5.

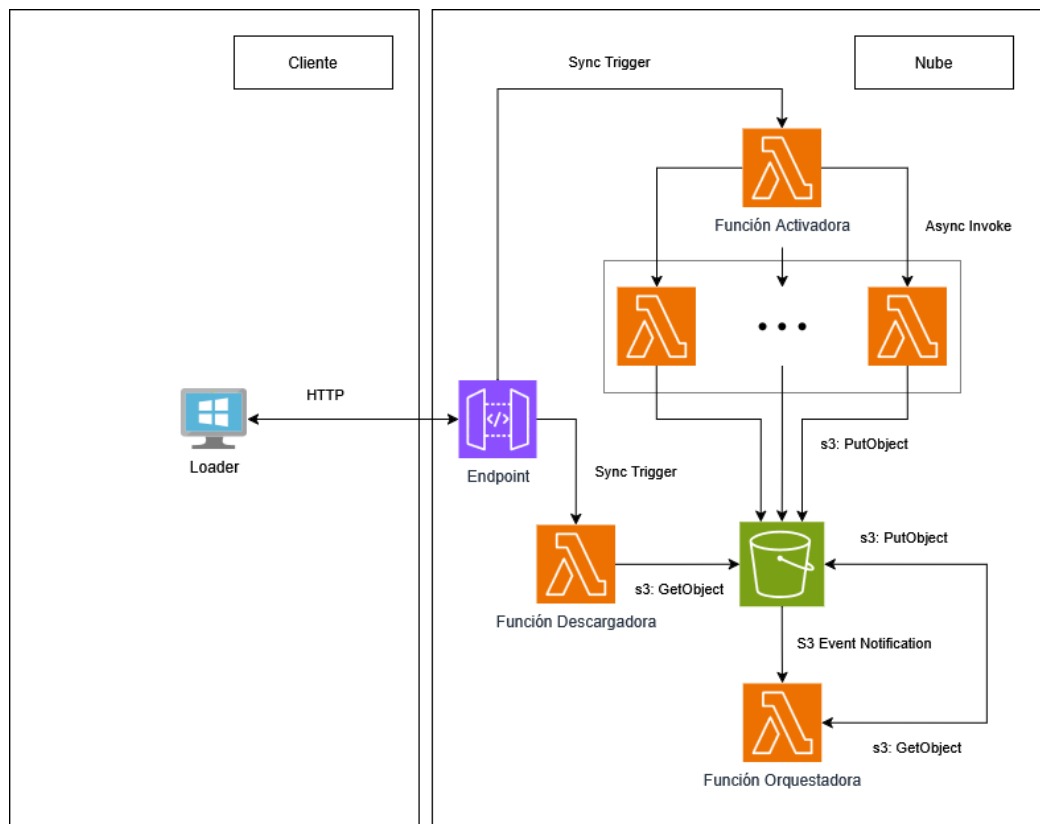


Figura 5.5: Arquitectura de la segunda solución.

- **Conexión externa:** Se ha utilizado API Gateway como *endpoint* entre el cliente y la lógica almacenada en AWS. Este servicio permite ocultar la infraestructura interna y proporciona una capa de transporte protegida por Amazon. Además, la conexión a servidores de Amazon hace que las comunicaciones entre el *Loader* y la nube parezcan legítimas.

Se ha configurado el acceso al *endpoint* en modo estricto, exigiendo la validación de la Indicación del Nombre del Servidor (SNI). Esta medida garantiza que el *backend* solo responda a peticiones que identifiquen el nombre de dominio del API Gateway, dificultando el descubrimiento del recurso mediante escaneos de direcciones IP. Asimismo, se ha forzado el uso de la política de seguridad TLS 1.3 ⁹.

- **Conexión interna en AWS:** Una vez que la petición atraviesa API Gateway, la comunicación entre los servicios del *backend* se realiza

⁹<https://docs.aws.amazon.com/apigateway/>

mediante llamadas a la API interna de AWS gestionada por el SDK oficial de Boto3 ¹⁰.

Por otro lado, las comunicaciones se dirigen solo a los puntos de enlace (*endpoints*) regionales de AWS, ya que todos los servicios se han desplegado en la misma región geográfica, por lo que el sistema minimiza los saltos lógicos y aprovecha la infraestructura de servicios de Amazon.

Configuración de servicios clave

- **API Gateway:** La configuración del API Gateway se basa en un modelo RESTful ¹¹. Se han definido rutas específicas (figura 5.6) para la activación (Campe) y la descarga (Aethon).

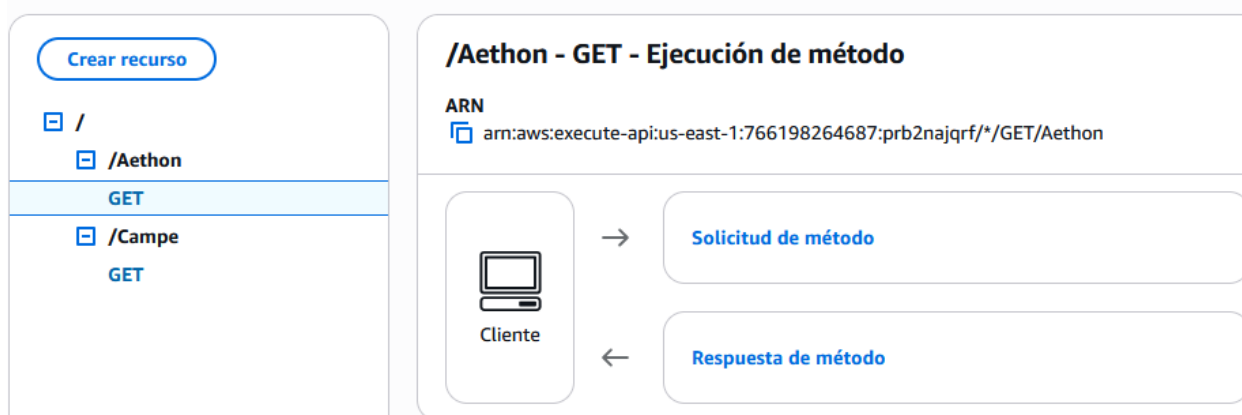


Figura 5.6: Configuración árbol de recursos en API Gateway.

- **Amazon S3:** El *bucket* de S3 se ha configurado como una unidad de almacenamiento temporal y privada. Como se puede observar en la figura 5.7, se han activado de forma estricta las opciones de *Block Public Access*, garantizando que solo las funciones Lambda con el rol de IAM (*Identity and Access Management*) ¹² adecuado puedan interactuar con los objetos ¹³.
- **AWS Lambda:** Las funciones Lambda se han configurado bajo un modelo de ejecución sin estado (*stateless*). Se utiliza el entorno de

¹⁰<https://docs.aws.amazon.com/lambda/>

¹¹<https://docs.aws.amazon.com/lambda/>

¹²AWS IAM (Identity and Access Management) es un servicio de AWS que administra de forma centralizada los permisos y el acceso a los recursos de la nube. Permite definir políticas de seguridad para que solo las entidades autorizadas puedan ejecutar acciones específicas sobre recursos determinados.

¹³<https://docs.aws.amazon.com/s3/>

Configuración de bloqueo de acceso público para este bucket

Se concede acceso público a los buckets y objetos a través de listas de control de acceso (ACL), políticas de bucket, políticas de puntos de acceso o todas las active Bloquear todo el acceso público. Esta configuración se aplica exclusivamente a este bucket y a sus puntos de acceso. AWS recomienda activar Bloquear todo el acceso público para que las aplicaciones funcionarán correctamente sin acceso público. Si necesita cierto nivel de acceso público a los buckets u objetos, puede personalizar la configuración específica. [Más información](#)

☒ Bloquear todo el acceso público

Activar esta configuración equivale a activar las cuatro opciones que aparecen a continuación. Cada uno de los siguientes ajustes son independientes entre sí.

☒ Bloquear el acceso público a buckets y objetos concedido a través de nuevas listas de control de acceso (ACL)

S3 bloqueará los permisos de acceso público aplicados a objetos o buckets agregados recientemente, y evitará la creación de nuevas ACL de acceso público para buckets y recursos de S3 mediante ACL.

☒ Bloquear el acceso público a buckets y objetos concedido a través de cualquier lista de control de acceso (ACL)

S3 ignorará todas las ACL que conceden acceso público a buckets y objetos.

☒ Bloquear el acceso público a buckets y objetos concedido a través de políticas de bucket y puntos de acceso públicas nuevas

S3 bloqueará las nuevas políticas de buckets y puntos de acceso que concedan acceso público a buckets y objetos. Esta configuración no afecta a las políticas ya existentes

☒ Bloquear el acceso público y entre cuentas a buckets y objetos concedido a través de cualquier política de bucket y puntos de acceso pública

S3 ignorará el acceso público y entre cuentas en el caso de buckets o puntos de acceso que tengan políticas que concedan acceso público a buckets y objetos.

Figura 5.7: Configuración de bloqueo de acceso público.

ejecución Python 3.9 ¹⁴. La estructura de las funciones generadoras (Cerberus, Scilla, Chimaera) se puede observar en la figura 5.8. Este algoritmo permite que el atacante solamente tenga que escribir el código de su fragmento (indicando si es binario o código de alto nivel) y la función Lambda se lo entregará a la lógica cuando sea invocada. El JSON enviado sigue la misma estructura mencionada en la solución 1 (véase figura 5.3).

```
FUNCTION generarFragmento(ID_fragmento, contenido):
# 1. Transformacion y codificacion
contenido_base64 ← CodificarBase64(contenido)

# 2. Estructura de metadatos
objeto_json ← {
  "module_id": ID_fragmento,
  "content": contenido_base64
  "bin": es_binario
}

# 3. Almacenamiento en un bucket (S3)
IF AlmacenarEnS3(bucket_nombre, ruta_destino, objeto_json) THEN
  RETURN estado_exito
ELSE
  RETURN estado_error
END IF

FIN FUNCTION
```

Figura 5.8: Algoritmo para generar fragmentos.

¹⁴<https://docs.aws.amazon.com/lambda/>

Flujo de datos

El flujo de datos en esta versión evoluciona hacia un modelo de orquestación asíncrona y distribuida dentro de la parte *cloud*. El ciclo de vida de los datos se articula en cuatro fases:

1. **Inicio y activación:** El flujo se inicia desde el *Loader* con una única petición saliente (HTTP GET) dirigida al *endpoint* de activación en API Gateway. A diferencia de la solución anterior, en esta la petición invoca a una función Lambda activadora. Esta función se encarga de invocar simultáneamente a las funciones Lambda que generan fragmentos, iniciando el proceso lógico dentro de AWS.
2. **Generación y depósito en S3:** Cada función generadora codifica su contenido en Base64 para, inmediatamente después y usando el SDK Boto3, establecer un flujo de volcado de datos (*PutObject*) hacia un *bucket* privado de Amazon S3. El fragmento codificado se deposita como un objeto independiente.
3. **Orquestación, ensamblado y reingreso al *bucket* de S3:** Tras la generación, el *bucket* invoca a la lambda orquestadora. Esta función hace *polling* para comprobar la existencia de todos los fragmentos almacenados por las generadoras y en ese caso realiza operaciones de lectura (*getObject*) a Amazon S3 para recuperarlos. La orquestadora decodifica el contenido de los fragmentos y concatena los fragmentos respetando el orden secuencial del *malware* original. Tras ello, deposita el código completo de nuevo en el *buffer* S3, listo para su descarga.
4. **Polling del cliente, descarga y ejecución:** Paralelamente a las fases 2 y 3, el *Loader* en la víctima ejecuta un *polling* periódico realizando peticiones GET constantes (hasta un límite de intentos) al *endpoint* de API Gateway. Este invoca a la función Lambda descargadora, que consulta la existencia del código final en el S3 y le envía la información de vuelta al *Loader*:
 - Si el código final no está listo envía un código de estado 404 al *Loader*, que tras esperar un tiempo de segundos entre intentos volverá a pedir la información y aumentará en uno el contador de intentos. Si este contador supera al límite, se abortará el proceso.
 - La descargadora recupera el objeto completo de S3 y lo encapsula en una respuesta JSON, codificado en Base64. El *Loader* recibe este paquete de datos, extrae el contenido, lo decodifica y lo inyecta en

el flujo de ejecución en la memoria RAM mediante *exec()*. El flujo de datos termina con la eliminación automática de los fragmentos y el código final del S3 por parte de la función descargadora siguiendo una estrategia de limpieza.

5.2.3. Integración de las tecnologías

Comunicación interna y SDKs

Para permitir la comunicación de Python con las APIs de AWS se ha utilizado la SDK (*Software Development Kit*) Boto3.

En primer lugar, para gestionar las invocaciones a otras funciones se ha utilizado el cliente *boto3.client('lambda')* ¹⁵. Las invocaciones son llamadas asíncronas (*InvocationType='Event'*) que usa la función activadora para simplemente «liberar a la prole» sin esperar su finalización.

Por otro lado, para la orquestación se utiliza el SDK para realizar tareas de control de estado. Se han implementado dos técnicas para mejorar la eficiencia del ensamblado:

- **Uso de *head-object*:** En lugar de descargar los fragmentos para comprobar si están listos (lo cual consumiría ancho de banda y tiempo), se utiliza este método que solo consulta los metadatos del objeto. Esto permite verificar la existencia de los archivos de forma económica y veloz ¹⁵.
- **Manejo de flujos binarios:** Mediante *response['Body'].read()*, el orquestador recupera los fragmentos directamente en memoria, permitiendo la manipulación de los datos (decodificación Base64 y concatenación) sin escribir archivos temporales en el sistema de archivos local de la Lambda ¹⁶.

Finalmente, el SDK se utiliza para la destrucción de evidencias. Tanto para el orquestador como para el descargador utilizan *delete-object* inmediatamente después de procesar la información.

En la figura 5.9, para ilustrar la lógica de sincronización asíncrona mediante el SDK, se presenta un fragmento del bucle de espera implementado en la función orquestadora.

¹⁵<https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>

¹⁶<https://docs.python.org/3/reference/index.html>

```

WHILE intentos < max_intentos AND archivos_existentes < total_esperados DO
    archivos_existentes ← 0

    FOR EACH archivo IN lista_esperados DO
        IF existe_en_S3(archivo) THEN
            archivos_existentes ← archivos_existentes + 1
        END IF
    END FOR

    IF archivos_existentes < total_esperados THEN
        intentos ← intentos + 1
        esperar(tiempo_espera)
    END IF
END WHILE

IF archivos_existentes == total_esperados THEN
    unir_fragmentos(fragmentos[])
ELSE
    abortar()
END IF

```

Figura 5.9: Algoritmo de sincronización y validación de fragmentos.

Permisos de accesos

En esta segunda solución se ha seguido también el Principio de Mínimo Privilegio. Esto se hace con el objetivo de garantizar que cada componente de la infraestructura tenga únicamente las capacidades estrictamente necesarias para su ejecución ^{17 18}. Para ello, se han diseñado roles de IAM específico:

- **Función Activadora (Campe):** Únicamente posee el permiso *lambda:InvokeFunction* limitado mediante un recurso ARN específico. Esto impide que se pueda utilizar la función para disparar otros servicios ajenos a la generación de fragmentos.
- **Generadores de fragmentos (Cerberus, Scilla, Chimaera):** Su política de acceso se limita únicamente a la operación *s3:PutObject* (escritura) sobre el *bucket* de almacenamiento temporal. No tienen permisos de lectura, lo que quiere decir que un generador puede depositar sus piezas, pero no puede conocer las depositadas por los demás.
- **Orquestadora (Echidna):** Al ser el componente encargado de ensamblar los fragmentos, su política es más extensa pero controlada. Dispone de *s3:GetObject* (lectura) para recuperar los fragmentos,

¹⁷<https://docs.aws.amazon.com/lambda/>

¹⁸<https://docs.aws.amazon.com/s3/>

s3:PutObject para depositar el artefacto final y *s3:DeleteObject* para eliminar los fragmentos después de unirlos. También se puede usar *s3:ListBucket* para leer el contenido sin almacenarlo en una variable, simplemente para comprobar la existencia de un archivo.

- **Función Descargadora (Aethon):** Este componente está autorizado a leer y borrar el resultado final del *bucket*. Se asegura que la destrucción de evidencias sea obligatoria antes de entregar de forma exitosa el *payload*.

Para evitar el uso de permisos excesivamente permisivos, todas las sentencias de las políticas especifican el recurso exacto mediante su ARN como se puede observar en la figura 5.10.

```

1  {
2      "Version": "2012-10-17",
3      "Statement": [
4          {
5              "Sid": "PermisosParaArchivos",
6              "Effect": "Allow",
7              "Action": [
8                  "s3:PutObject",
9                  "s3:GetObject",
10                 "s3:DeleteObject"
11             ],
12             "Resource": "arn:aws:s3:::tfg-typhon-bucket/*"
13         },
14         {
15             "Sid": "PermisosParaElContenedor",
16             "Effect": "Allow",
17             "Action": "s3:ListBucket",
18             "Resource": "arn:aws:s3:::tfg-typhon-bucket"
19         }
20     ]
21 }

```

Figura 5.10: Permisos del orquestador de la aplicación

Cifrado de datos y formato de entrega

Al igual que la primera solución, en esta solución se cifra los fragmentos de las funciones Lambda generadoras mediante Base64 y se mantiene así cuando los fragmentos se hayan en reposo en el *buffer* S3. Así, se mitiga el riesgo de exposición de los artefactos ante posibles auditorías.

Por otro lado, para evitar inspecciones durante el tránsito, el código final no puede viajar ni como texto plano ni como código, sino que debe ir

como formato JSON. Este uso de JSON resulta idóneo en esta arquitectura de cliente ligero porque combina metadatos con la carga útil o *payload*. Durante el *polling* del *Loader*, la función descargadora devuelve al cliente una respuesta estructurada con un código de estado que especificará al *Loader* si seguir haciendo *polling* o si por el contrario ya ha llegado el código completo.

El *payload* en el JSON que recibe el *Loader* está codificado en Base64, por lo que deberá decodificarlo y ejecutar la cadena de texto resultante, logrando una transferencia segura, estructurada e indetectable.

5.2.4. Trazabilidad: Conectando diseño e implementación

Desviación respecto a la primera solución

La primera solución contempla un cliente pesado con múltiples responsabilidades. Tras diseñarla e implementarla surgió la idea de reducir al máximo las tareas del *Loader* otorgándoselas a la nube. Así, surge la segunda solución.

Durante la implementación de esta nueva arquitectura, la comunicación con el *backend* requirió la introducción de una función Lambda descargadora. Por otro lado, la compleja comunicación asíncrona en los componentes de la nube provocó que en múltiples ocasiones el *Loader* no recibiera el código completo (problemas de concurrencia). Para resolver esta limitación, se recurrió a la implementación de una estrategia de *polling*. De esta forma, el *Loader*, verifica un número determinado de veces si el código final ya está disponible para descarga y en caso de superar ese límite, aborta el proceso de manera controlada.

Existencia de un *buffer* S3

En la primera solución, el código fragmentado de las funciones Lambda se enviaba directamente al cliente. Sin embargo, al trasladar la orquestación a la nube mediante una función Lambda dedicada, surgió el desafío arquitectónico de gestionar el flujo de salida de las generadoras de fragmentos.

La primera opción que se barajó fue que las funciones Lambda generadoras invocasen directamente a la orquestadora. Sin embargo, esto supondría un problema debido a que entonces existiría un número n (siendo n el número de generadoras de fragmentos) de ejecuciones distintas de la orquestadora y sería complicado sincronizarlas, pues

ninguna ejecución tendría ningún conocimiento de las otras.

Para solventar este inconveniente, se implementó el uso de un *buffer* intermedio que actuara como un almacén temporal y que se comunicase con el resto de las funciones para guardar código o pedirlo. Se optó por integrar el servicio Amazon S3, de forma que ahora las funciones generadoras depositen sus respectivos fragmentos en él. Posteriormente, la orquestadora los recupera para su ensamblado y almacena el código final orquestado. Por último, la función descargadora consulta este código almacenado para entregar el artefacto final al *Loader*, cumpliendo rigurosamente con el flujo de datos ideado en la figura 4.3.

Polling en la orquestación

Dentro del esquema asíncrono del *backend*, fue necesario definir un mecanismo que permitiera a la función orquestadora verificar la disponibilidad de los fragmentos antes de proceder con el ensamblado. La solución implementada se basa en el *polling*.

Esta estrategia, contemplada desde el proceso de diseño de esta solución (figura 4.3) es muy simple: la función orquestadora consulta el estado del almacenamiento aplicando un retardo definido entre cada intento. Si se excede el número máximo de consultas predefinido sin que todos los fragmentos estén disponibles, el proceso se interrumpe de forma controlada. Este mecanismo garantiza la integridad del proceso, evitando tanto el bloqueo indefinido de los recursos en la nube como el ensamblado de código incompleto.

Capítulo 6. Resultados, Conclusiones y Trabajo Futuro

Una vez finalizado el desarrollo de las soluciones, este capítulo expone los resultados obtenidos tras someter la arquitectura a un proceso de pruebas y mediciones en un entorno controlado. Se presenta una comparación entre ambas soluciones, se contrastan los hallazgos con los objetivos definidos al inicio del trabajo y, por último, se plantean las principales líneas de trabajo futuro derivadas del proyecto.

6.1. Entorno de pruebas y *Payload*

Para evaluar la eficacia de la arquitectura *serverless* propuesta, se ha diseñado un artefacto de Prueba de Concepto (PoC). La complejidad del artefacto está justificada por la necesidad de simular un amenaza persistente avanzada (APT), combinando la fragmentación en la nube con un *malware* complejo y demostrando como podría ser explotada la plataforma para fines maliciosos.

6.1.1. Arquitectura del artefacto: Inyector de memoria basado en metaprogramación

Este artefacto actúa como el *payload* que será fragmentado y distribuido en la infraestructura de AWS. Para simular las funciones de un código real se ha implementado con técnicas de inyección en memoria¹ y metaprogramación. Este diseño permite que la lógica maliciosa no sea un bloque estático, sino un conjunto de piezas que se ensamblan y mutan dinámicamente [1]. Su estructura se divide en tres bloques (fragmentos):

- **Carga útil (Shellcode):** El primer bloque contiene el *payload* real (*shellcode* x64). Son instrucciones en código máquina puro. Al no ser un archivo ejecutable, no tiene cabeceras que un antivirus pueda analizar de forma estática.
- **Polimorfismo y mutación:** En este bloque se encuentra toda la configuración del código para evadir los sistemas de detección, utilizando técnicas de metaprogramación.

¹<https://attack.mitre.org/techniques/T1055/002/>

- **Cifrado XOR dinámico:** Se genera una clave aleatoria de cifrado en cada ciclo, asegurando que el *hash* del código cuando sea ensamblado sea diferente en cada ejecución. [1]
 - **Fragmentación de identificadores:** Los nombres de las APIs de Windows (como `OpenProcess` o `VirtualAllocEx`) se fragmentan y reconstruyen en ejecución. Esto impide que los antivirus detecten la intención del programa mediante análisis estático de las funciones. [3]
 - **Código basura:** Se insertan de forma aleatoria operaciones matemáticas y lógicas inútiles para confundir a los análisis heurísticos. [2]
- **Inyección en memoria:** Este último bloque implementa la lógica de inyección, usando la librería `ctypes` para interactuar con el núcleo de Windows y realizar las siguientes interacciones:
 1. Buscar un proceso legítimo (en este caso `RuntimeBroker.exe`).
 2. Reservar un espacio de memoria privada.
 3. Copiar el *shellcode* en dicho espacio de memoria.
 4. Crear un hilo de ejecución que apunte a esa dirección de memoria.

Esta arquitectura garantiza que el código malicioso sólo exista de forma íntegra y legible dentro de la memoria RAM de un proceso confiable, minimizando la superficie de exposición y maximizando la persistencia frente a herramientas de monitorización tradicionales. [17]

Todas las pruebas se han realizado en un entorno controlado y los artefactos maliciosos han sido utilizados exclusivamente con fines académicos para evaluar las capacidades de defensa y detección de dichos artefactos. En ningún caso se ha interactuado con sistemas externos a la red de pruebas.

6.1.2. Entorno de pruebas

Una vez se tiene el *malware* de prueba, se ha preparado el entorno en el que se va a ejecutar.

Especificaciones generales

La estructura general abarca desde las fechas de ejecución hasta la configuración de red empleadas. Estos parámetros que se han aplicado en las pruebas se pueden apreciar en la tabla 6.1.

Parámetro	Detalles y configuración aplicada
Fechas de ejecución	Pruebas realizadas entre el 16 y el 30 de abril de 2026.
Sistema Anfitrión	Oracle VM VirtualBox (Versión 7.1.0 r164728).
Sistema Operativo Víctima	Windows 11 Pro con 4 GB RAM y 2 vCPU.
Configuración de Red	Interfaz en modo NAT. Resolución DNS habilitada para permitir tráfico HTTPS saliente hacia AWS.

Tabla 6.1: Especificaciones del entorno de laboratorio y red

Especificaciones de los antivirus empleados

Para analizar cómo actúa la plataforma frente a defensas que pueda haber en la máquina víctima, se han utilizado algunos antivirus, en concreto Windows Defender, Avast, AVG y Panda Dome.

Por parte de Windows Defender, la última versión de inteligencia de seguridad que se empleó fue la 1.449.367.0 creada el 30 de abril, como se puede apreciar en la figura 6.1. En cuanto a Avast y AVG, sus versiones de definiciones de virus son la 260430-2 y 260501-6 respectivamente, algo que se ve en las figuras 6.2 y 6.3 junto con otra información de los antivirus en cuestión. La tabla 6.2 resume alguna de las informaciones de relevancia.



Figura 6.1: Imagen que muestra información de la inteligencia de seguridad de Microsoft Defender empleada en las pruebas.

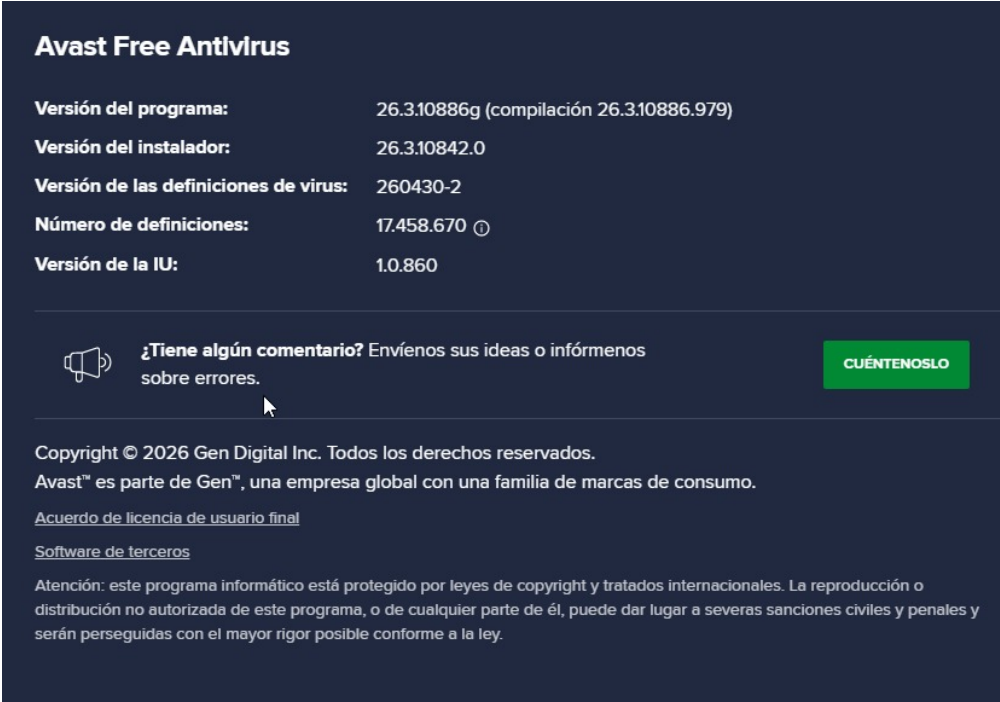


Figura 6.2: Imagen que muestra información de las versiones empleadas en las pruebas con Avast.

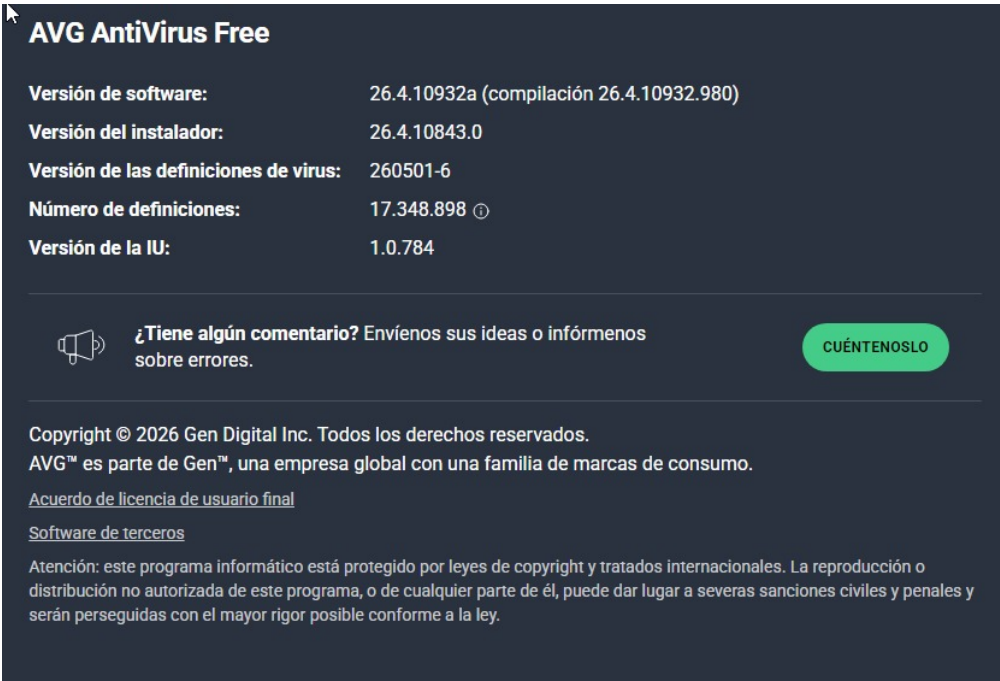


Figura 6.3: Imagen que muestra información de las versiones empleadas en las pruebas con AVG.

6.2. Resultados obtenidos en la evaluación de sistema

El objetivo principal de la fase de pruebas es medir la viabilidad técnica de la herramienta y su eficacia real frente a los mecanismos de defensa actuales. Para ello, se ha utilizado el *malware* anteriormente mencionado en

Parámetro	Detalles y configuración aplicada
Microsoft Defender	Versión de inteligencia de seguridad 1.449.367.0 Fecha de creación de la versión: 30/04/2026 3:23 Última actualización: 30/04/26 12:18
Avast Free Antivirus	Versión 26.3.10886g. Versión de las definiciones de virus: 260430-2 Número de definiciones: 17.458.670
AVG Antivirus	Versión 26.4.10923a Versión de las definiciones de virus: 260501-6 Número de definiciones: 17.348.898
Panda Dome	Versión 23.0.0 Última actualización: 30/04/26 11:15

Tabla 6.2: Especificaciones de los antivirus empleados

un entorno de pruebas controlado que permite auditar el comportamiento de ambas soluciones en cuatro vertientes críticas:

6.2.1. Evasión ante defensas locales

El objetivo de esta fase es evaluar la capacidad de evasión del *Loader* frente a los mecanismos de protección de la víctima, en concreto, antivirus. Para ello, ambas soluciones fueron ejecutados en entornos monitorizados por Windows Defender, Avast Antivirus, AVG y Panda Dome.

Análisis estático en reposo

Primeramente se procedió a analizar la respuesta de los antivirus ante el ejecutable en reposo. Esta prueba evalúa la capacidad de evasión del código frente escaneos manuales bajo demanda, es decir, los escaneos que realizan los antivirus cuando el usuario quiere ver si hay algún archivo malicioso en su dispositivo.

Los resultados fueron gratificantes, ningún antivirus reparó en la existencia maliciosa del ejecutable y todos llegaron a la conclusión de que no había ningún ente sospechoso en el sistema. Esto es así porque el código no posee cadenas de texto sospechosas, sino que actúa de manera legítima comunicándose con Amazon y luego ejecutando lo entregado.

Análisis dinámico durante la ejecución

Durante las pruebas iniciales, tanto la solución 1 como la solución 2 lograron un éxito rotundo al evadir por completo la detección de Windows Defender y AVG. La ausencia de alertas en estos sistemas confirma la

efectividad de la técnica implementada y la correcta evasión por parte del *Loader* al ejecutar el código malicioso.

Por otro lado, durante la ejecución de ambas soluciones en Avast se observó que se interceptó la ejecución inicial del ejecutable (al hacer doble clic sobre él) al carecer de firma digital y reputación global. El motor del antivirus aisló temporalmente el proceso en su *sandbox* para realizar un análisis. Sin embargo, el motor determinó que la ejecución era segura y se pudo proseguir sin ningún problema con el funcionamiento del código, como se puede apreciar en la figura 6.4. Este éxito en la evasión se debe a la naturaleza benigna de las operaciones iniciales del *Loader*, pues se realizan peticiones de red a un dominio de alta confianza (API Gateway) y después todo se realiza en memoria.

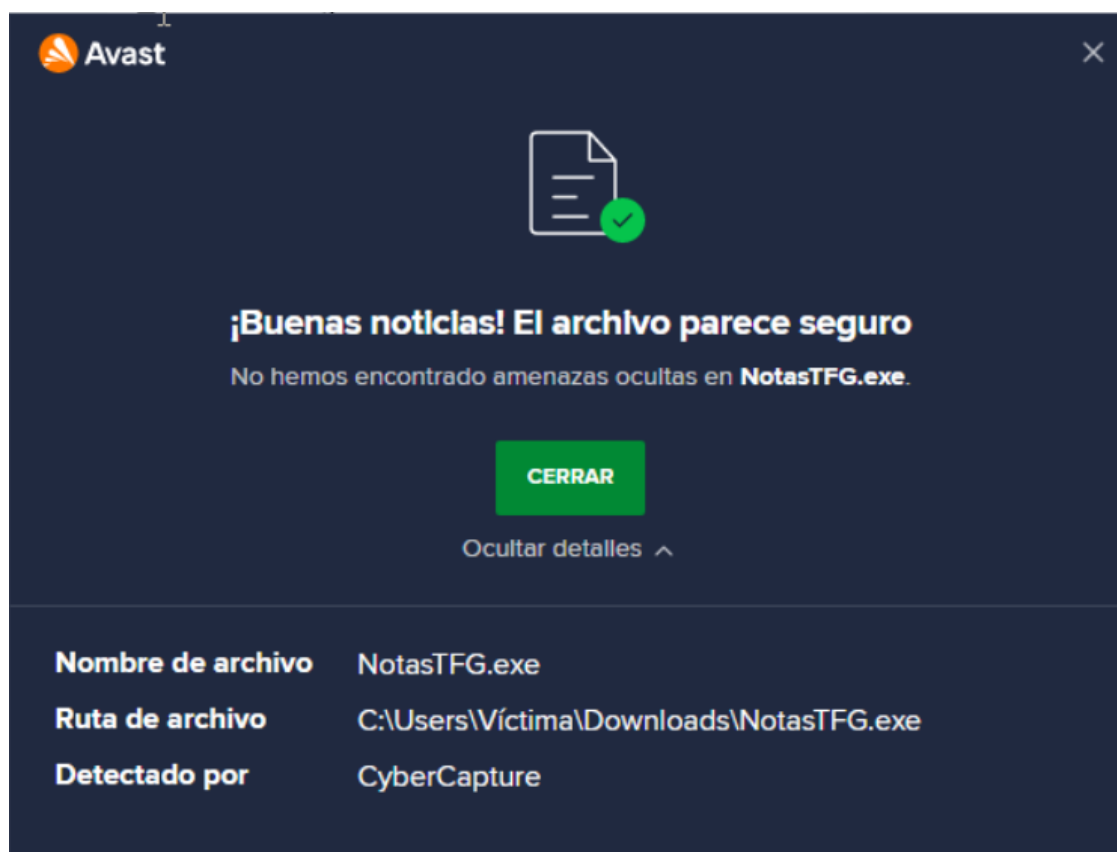


Figura 6.4: Imagen que muestra que Avast considera al ejecutable del *Loader* de la solución 1 (NotasTFG.exe) como seguro.

No obstante, durante las pruebas con Panda, el motor bloqueó la ejecución de ambas soluciones, como se puede apreciar en la figura 6.5. Para determinar se debía a la arquitectura, al método de inyección en memoria u otro motivo, se desarrolló un *script* benigno básico (un simple *script* que imprimía «Hola Mundo» por consola). Este *script* fue convertido a ejecutable utilizando la misma herramienta PyInstaller.

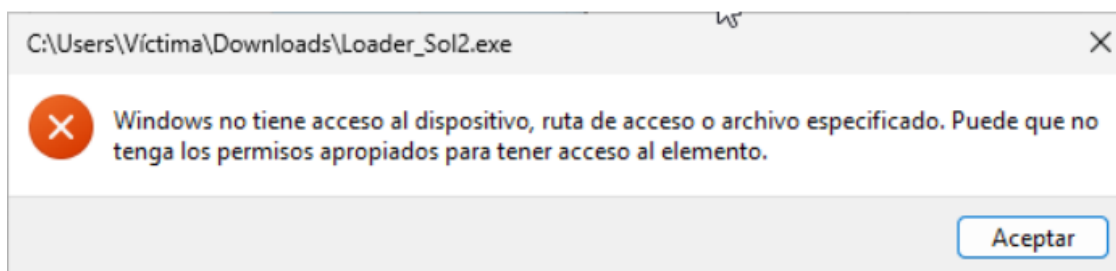


Figura 6.5: Imagen que muestra cómo Panda elimina el ejecutable al considerarlo como virus. Se muestra un error de que no ha sido encontrado el archivo.

Panda volvió a a clasificar y bloquear este ejecutable inofensivo, lo que demuestra que la detección no responde a un fallo en la arquitectura del *Loader* diseñado en este proyecto, sino que se trata de un falso positivo por la firma del PyInstaller en el ejecutable. Esto es una limitación en este ámbito, pues ante la presencia de este tipo de firmas, el antivirus lo cataloga como sospechoso por precaución y lo neutraliza, como se puede apreciar en la figura 6.6 , donde se muestra cómo se neutralizó la solución 1 (NotasTFG.exe).

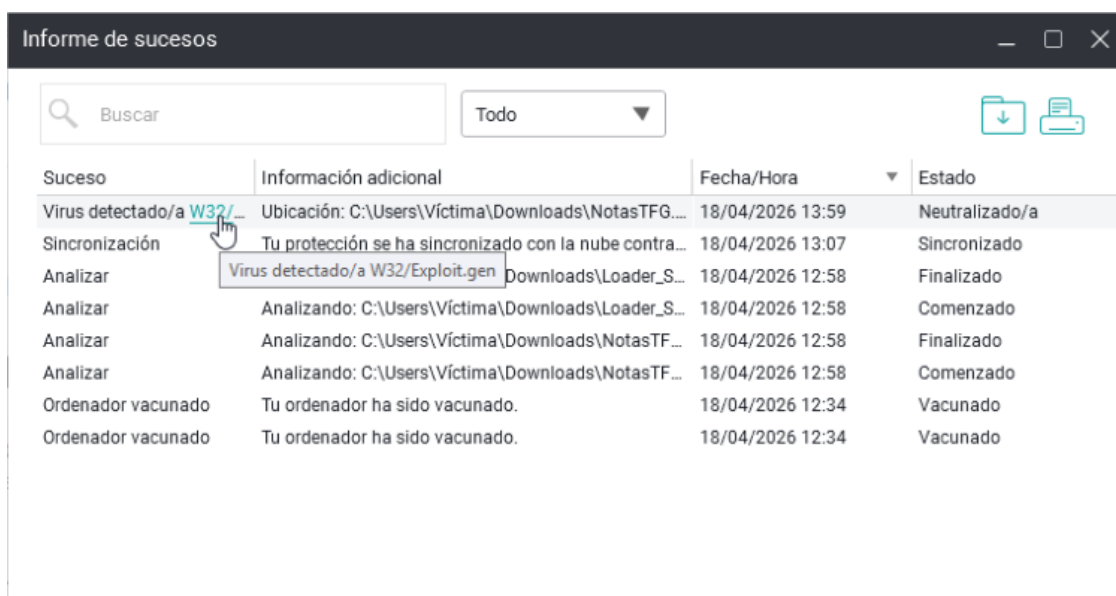


Figura 6.6: Imagen que enseña que Panda Dome ha puesto al archivo Loader como neutralizado.

6.2.2. Análisis y evasión a nivel de red

En esta fase de evaluación, se procedió a auditar el tráfico de red generado por el ejecutable durante el proceso de infección. El objetivo principal fue comprobar si los métodos de evasión y ofuscación conseguían que en caso de ser auditados pudiera pasar desapercibido. Por otro lado, se pretendía observar posibles patrones anómalos o

comportamientos que puedan ser detectados por un Sistema de Detección de Intrusos en la Red. Para lograr una correcta visión, el análisis se ha dividido en dos bloques: monitorización pasiva y activa.

Monitorización pasiva y análisis de flujo (Wireshark)

Al analizar la solución 1, la captura de tráfico revela las n peticiones (una por cada fragmento) sucesivas dirigidas hacia la estructura de AWS. Como se observa en la figura 6.7, cada comunicación consta de una solicitud rápida (*request*) de un fragmento específico (figura 6.8, paquetes 28 y 29) y su correspondiente respuesta por parte del API Gateway (6.9, paquetes 31 y 32). Dado que esta comunicación se realiza a través del protocolo HTTPS (TLS 1.3), el contenido viaja cifrado. Al segmentar el *payload* en múltiples fragmentos de pequeño tamaño, el tráfico se diluye entre las comunicaciones habituales del sistema operativo. Esto resulta indistinguible del uso legítimo de los servicios de Amazon, lo que ayuda a ocultar el verdadero propósito de dichos paquetes.

No.	Time	Source	Destination	Protocol	Length	Info
15	5.875065000	192.168.1.39	80.58.61.250	DNS	106	Standard query 0xb3b7 A b4hr04pdjl.execute-api.us-east-1.amazo...
16	5.880008600	80.58.61.250	192.168.1.39	DNS	122	Standard query response 0xb3b7 A b4hr04pdjl.execute-api.us-east-1...
20	5.990132800	192.168.1.39	18.211.173.202	TLSv1.3	571	Client Hello (SNI=b4hr04pdjl.execute-api.us-east-1.amazonaws.c...
22	6.095022300	18.211.173.202	192.168.1.39	TLSv1.3	1506	Server Hello, Change Cipher Spec, Application Data
25	6.095350300	18.211.173.202	192.168.1.39	TLSv1.3	1506	Application Data, Application Data
26	6.095408000	18.211.173.202	192.168.1.39	TLSv1.3	66	Application Data
28	6.096146700	192.168.1.39	18.211.173.202	TLSv1.3	118	Change Cipher Spec, Application Data
29	6.096317100	192.168.1.39	18.211.173.202	TLSv1.3	240	Application Data
31	6.248497000	18.211.173.202	192.168.1.39	TLSv1.3	1071	Application Data
32	6.248497000	18.211.173.202	192.168.1.39	TLSv1.3	78	Application Data
40	6.367833500	192.168.1.39	18.211.173.202	TLSv1.3	571	Client Hello (SNI=b4hr04pdjl.execute-api.us-east-1.amazonaws.c...
42	6.477703600	18.211.173.202	192.168.1.39	TLSv1.3	1506	Server Hello, Change Cipher Spec, Application Data
45	6.478039400	18.211.173.202	192.168.1.39	TLSv1.3	1506	Application Data, Application Data
46	6.478087200	18.211.173.202	192.168.1.39	TLSv1.3	66	Application Data
48	6.478760000	192.168.1.39	18.211.173.202	TLSv1.3	118	Change Cipher Spec, Application Data
49	6.478891400	192.168.1.39	18.211.173.202	TLSv1.3	242	Application Data
52	6.617771400	18.211.173.202	192.168.1.39	TLSv1.3	428	Application Data
53	6.617771400	18.211.173.202	192.168.1.39	TLSv1.3	78	Application Data
61	6.736468500	192.168.1.39	18.211.173.202	TLSv1.3	571	Client Hello (SNI=b4hr04pdjl.execute-api.us-east-1.amazonaws.c...
63	6.846668500	18.211.173.202	192.168.1.39	TLSv1.3	1506	Server Hello, Change Cipher Spec, Application Data
66	6.846978200	18.211.173.202	192.168.1.39	TLSv1.3	1506	Application Data, Application Data
67	6.847039300	18.211.173.202	192.168.1.39	TLSv1.3	66	Application Data
69	6.847808400	192.168.1.39	18.211.173.202	TLSv1.3	118	Change Cipher Spec, Application Data
70	6.847967600	192.168.1.39	18.211.173.202	TLSv1.3	240	Application Data
73	6.980931700	18.211.173.202	192.168.1.39	TLSv1.3	1272	Application Data
74	6.980931700	18.211.173.202	192.168.1.39	TLSv1.3	78	Application Data
79	12.462431100	192.168.1.39	185.43.181.64	TLSv1.2	78	Application Data

Figura 6.7: Captura del tráfico en la solución 1 en Wireshark donde se aprecia las peticiones de los 3 fragmentos.

Por otro lado, la solución 2 presenta un comportamiento de red mucho más definido y ruidoso. En la figura 6.10 se distinguen claramente las dos comunicaciones que corresponden a la llamada de la función activadora y la descargadora.

Al descargar el *payload* por completo en una sola respuesta, el volumen de datos transferido es significativamente mayor. Esto provoca la aparición

No.	Time	Source	Destination	Protocol	Length	Info
15	5.875065000	192.168.1.39	80.58.61.250	DNS	106	Standard query 0xb3b7 A b4hr04pdjl.execute-api.us-east-1.amazo...
16	5.880008600	80.58.61.250	192.168.1.39	DNS	122	Standard query response 0xb3b7 A b4hr04pdjl.execute-api.us-eas...
20	5.990132800	192.168.1.39	18.211.173.202	TLSv1.3	571	Client Hello (SNI=b4hr04pdjl.execute-api.us-east-1.amazonaws.c...
22	6.095022300	18.211.173.202	192.168.1.39	TLSv1.3	1506	Server Hello, Change Cipher Spec, Application Data
25	6.095350300	18.211.173.202	192.168.1.39	TLSv1.3	1506	Application Data, Application Data
26	6.095408000	18.211.173.202	192.168.1.39	TLSv1.3	66	Application Data
28	6.096146700	192.168.1.39	18.211.173.202	TLSv1.3	118	Change Cipher Spec, Application Data
29	6.096317100	192.168.1.39	18.211.173.202	TLSv1.3	240	Application Data
31	6.248497000	18.211.173.202	192.168.1.39	TLSv1.3	1071	Application Data
32	6.248497000	18.211.173.202	192.168.1.39	TLSv1.3	78	Application Data
40	6.367833500	192.168.1.39	18.211.173.202	TLSv1.3	571	Client Hello (SNI=b4hr04pdjl.execute-api.us-east-1.amazonaws.c...
42	6.477703600	18.211.173.202	192.168.1.39	TLSv1.3	1506	Server Hello, Change Cipher Spec, Application Data
45	6.478039400	18.211.173.202	192.168.1.39	TLSv1.3	1506	Application Data, Application Data
46	6.478087200	18.211.173.202	192.168.1.39	TLSv1.3	66	Application Data
48	6.478760000	192.168.1.39	18.211.173.202	TLSv1.3	118	Change Cipher Spec, Application Data
49	6.478891400	192.168.1.39	18.211.173.202	TLSv1.3	242	Application Data
52	6.617771400	18.211.173.202	192.168.1.39	TLSv1.3	428	Application Data
53	6.617771400	18.211.173.202	192.168.1.39	TLSv1.3	78	Application Data
61	6.736468500	192.168.1.39	18.211.173.202	TLSv1.3	571	Client Hello (SNI=b4hr04pdjl.execute-api.us-east-1.amazonaws.c...
63	6.846668500	18.211.173.202	192.168.1.39	TLSv1.3	1506	Server Hello, Change Cipher Spec, Application Data
66	6.846978200	18.211.173.202	192.168.1.39	TLSv1.3	1506	Application Data, Application Data
67	6.847039300	18.211.173.202	192.168.1.39	TLSv1.3	66	Application Data
69	6.847808400	192.168.1.39	18.211.173.202	TLSv1.3	118	Change Cipher Spec, Application Data

> Transmission Control Protocol, Src Port: 61190, Dst Port: 443, Seq: 582, Ack: 4369, Len: 186
 Transport Layer Security
 [Stream index: 0]
 TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
 Opaque Type: Application Data (23)
 Version: TLS 1.2 (0x0303)
 Length: 181
 Encrypted Application Data [...]: d3f75dd40bf90895a10e7979000c610f959fbba7eb1063694b7ea050627a8e03f4d4e0f926e6b7bc130b5aa9228097a2952f
 [Application Data Protocol: Hypertext Transfer Protocol]

Payload is encrypted application data (tls.app_data), 181 byte(s) Paquetes: 744 · Displayed: 148 (19.9%) Perfil: Default

Figura 6.8: Captura de una petición de fragmento en la solución 1. Se aprecia que la información va encriptada.

> Transmission Control Protocol, Src Port: 443, Dst Port: 61190, Seq: 4369, Ack: 768, Len: 1017
 Transport Layer Security
 [Stream index: 0]
 TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
 Opaque Type: Application Data (23)
 Version: TLS 1.2 (0x0303)
 Length: 1012
 Encrypted Application Data [...]: f6d402518df6340e615234a830a82b8b69d0dec72069bbb8442be3c04a3754eb65da76ac166f2e088d903f56f822476c4d7f
 [Application Data Protocol: Hypertext Transfer Protocol]

Payload is encrypted application data (tls.app_data), 1012 byte(s) Paquetes: 1054 · Displayed: 213 (20.2%) Perfil: Default

Figura 6.9: Parte de un fragmento del *malware* enviado por el servidor como respuesta a la petición del cliente. Va encriptada también.

de múltiples segmentos TCP que se deben reensamblar para reconstruir la *Protocol Data Unit* (PDU) original, evento que se puede observar en el paquete 118 en la figura 6.11. Este aumento abrupto del tamaño de la respuesta, junto con la posibilidad de que se tenga que realizar múltiples comunicaciones por el *polling*, incrementan la posibilidad de que se intercepte y se descubra la conexión.

Interceptación activa y análisis

Además de la monitorización pasiva, se evaluó la capacidad de evasión de la comunicación frente a técnicas de interceptación activa (*Man-in-the-middle*) que pudieran ser empleadas por analistas de *malware*.

No.	Time	Source	Destination	Protocol	Length	Info
72	16.745024400	185.43.181.58	192.168.1.39	TLSv1.3	373	Application Data
73	16.747168100	185.43.181.58	192.168.1.39	TCP	1506	443 → 61222 [ACK] Seq=590 Ack=993 Win=64640 Len=1452 [TCP PDU ...]
75	16.747540300	185.43.181.58	192.168.1.39	TLSv1.3	159	Application Data
79	16.795280700	192.168.1.39	50.19.149.92	TLSv1.3	571	Client Hello (SNI=prb2najqrf.execute-api.us-east-1.amazonaws.c..)
81	16.908045000	50.19.149.92	192.168.1.39	TLSv1.3	1506	Server Hello, Change Cipher Spec, Application Data
84	16.908337900	50.19.149.92	192.168.1.39	TLSv1.3	1506	Application Data, Application Data
85	16.908367300	50.19.149.92	192.168.1.39	TLSv1.3	66	Application Data
87	16.909262000	192.168.1.39	50.19.149.92	TLSv1.3	118	Change Cipher Spec, Application Data
88	16.909407300	192.168.1.39	50.19.149.92	TLSv1.3	239	Application Data
94	19.898001700	50.19.149.92	192.168.1.39	TLSv1.3	474	Application Data
95	19.898001700	50.19.149.92	192.168.1.39	TLSv1.3	78	Application Data
103	20.115291200	192.168.1.39	50.19.149.92	TLSv1.3	571	Client Hello (SNI=prb2najqrf.execute-api.us-east-1.amazonaws.c..)
105	20.221225600	50.19.149.92	192.168.1.39	TLSv1.3	1506	Server Hello, Change Cipher Spec, Application Data
108	20.221745100	50.19.149.92	192.168.1.39	TLSv1.3	1506	Application Data, Application Data
109	20.222378200	50.19.149.92	192.168.1.39	TLSv1.3	66	Application Data
111	20.222743000	192.168.1.39	50.19.149.92	TLSv1.3	118	Change Cipher Spec, Application Data
112	20.222844300	192.168.1.39	50.19.149.92	TLSv1.3	240	Application Data
114	21.583271400	50.19.149.92	192.168.1.39	TCP	1506	443 → 61223 [ACK] Seq=4369 Ack=768 Win=31360 Len=1452 [TCP PDU..]
118	21.583594000	50.19.149.92	192.168.1.39	TLSv1.3	1216	Application Data
120	21.583729700	50.19.149.92	192.168.1.39	TLSv1.3	78	Application Data

Figura 6.10: Captura del tráfico de la solución 2, donde se distinguen dos únicas comunicaciones correspondientes a la petición de invocar a la activadora y la descargadora.

No.	Time	Source	Destination	Protocol	Length	Info
96	19.898179000	192.168.1.39	50.19.149.92	TCP	54	61221 → 443 [ACK] Seq=767 Ack=4813 Win=65024 Len=0
97	19.898277700	50.19.149.92	192.168.1.39	TCP	60	443 → 61221 [FIN, ACK] Seq=4813 Ack=767 Win=31360 Len=0
98	19.898449400	192.168.1.39	50.19.149.92	TCP	54	61221 → 443 [ACK] Seq=767 Ack=4814 Win=65024 Len=0
99	19.898791000	192.168.1.39	50.19.149.92	TCP	54	61221 → 443 [RST, ACK] Seq=767 Ack=4814 Win=0 Len=0
100	20.000148000	192.168.1.39	50.19.149.92	TCP	66	61223 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WScale=0 SACK_PERM=1
101	20.114637300	50.19.149.92	192.168.1.39	TCP	66	443 → 61223 [SYN, ACK] Seq=0 Ack=1 Win=29280 Len=0 MSS=1460 SACK_PERM=1 WScale=1
102	20.114816400	192.168.1.39	50.19.149.92	TCP	54	61223 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
103	20.115291200	192.168.1.39	50.19.149.92	TLSv1.3	571	Client Hello (SNI=prb2najqrf.execute-api.us-east-1.amazonaws.com)
104	20.217352000	50.19.149.92	192.168.1.39	TCP	60	443 → 61223 [ACK] Seq=1 Ack=518 Win=38536 Len=0
105	20.221259000	50.19.149.92	192.168.1.39	TLSv1.3	1506	Server Hello, Change Cipher Spec, Application Data
106	20.221392100	50.19.149.92	192.168.1.39	TCP	1386	443 → 61223 [ACK] Seq=1453 Ack=518 Win=38536 Len=1452 [TCP PDU reassembled in 100]
107	20.221647900	192.168.1.39	50.19.149.92	TCP	54	61223 → 443 [ACK] Seq=518 Ack=2085 Win=65280 Len=0
108	20.221745100	50.19.149.92	192.168.1.39	TLSv1.3	1506	Application Data, Application Data
109	20.222378200	50.19.149.92	192.168.1.39	TLSv1.3	66	Application Data
110	20.222520500	192.168.1.39	50.19.149.92	TCP	54	61223 → 443 [ACK] Seq=518 Ack=4369 Win=65280 Len=0
111	20.222743000	192.168.1.39	50.19.149.92	TLSv1.3	118	Change Cipher Spec, Application Data
112	20.222844300	192.168.1.39	50.19.149.92	TLSv1.3	240	Application Data
113	20.326708000	50.19.149.92	192.168.1.39	TCP	60	443 → 61223 [ACK] Seq=4369 Ack=768 Win=31360 Len=0
114	21.583271400	50.19.149.92	192.168.1.39	TCP	1506	443 → 61223 [ACK] Seq=4369 Ack=768 Win=31360 Len=1452 [TCP PDU reassembled in 118]
115	21.583271400	50.19.149.92	192.168.1.39	TCP	1506	443 → 61223 [ACK] Seq=5821 Ack=768 Win=31360 Len=1452 [TCP PDU reassembled in 118]
116	21.583446400	192.168.1.39	50.19.149.92	TCP	54	61223 → 443 [ACK] Seq=768 Ack=7273 Win=65280 Len=0
117	21.583594000	50.19.149.92	192.168.1.39	TCP	1506	443 → 61223 [ACK] Seq=7273 Ack=768 Win=31360 Len=1452 [TCP PDU reassembled in 118]
118	21.583594000	50.19.149.92	192.168.1.39	TLSv1.3	1216	Application Data
119	21.583655900	192.168.1.39	50.19.149.92	TCP	54	61223 → 443 [ACK] Seq=768 Ack=9587 Win=65280 Len=0
120	21.583729700	50.19.149.92	192.168.1.39	TLSv1.3	78	Application Data

```

> [Timestamps]
> [SEQ/ACK analysis]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (1162 bytes)
TCP segment data (1162 bytes)
[4 Reassembled TCP Segments (5518 bytes): #114(1452), #115(1452), #117(1452), #118(1162)]
[Frame: 114, payload: 0-1451 (1452 bytes)]
[Frame: 115, payload: 1452-2902 (1452 bytes)]
[Frame: 117, payload: 2904-4355 (1452 bytes)]
[Frame: 118, payload: 4356-5517 (1162 bytes)]
[Segment count: 4]
[Reassembled TCP Length: 5518]
[Reassembled TCP Data [-]: 178385158986c1172676889543b613a2bc31155878b689e547aceda852816a7321a665496b7e1ac853444d676823b8e6185173ac474f8b5f11dc472252913cc3f46d8873c645e612a63c8d1f884f9]
Transport Layer Security

```

Figura 6.11: Captura de la respuesta del servidor. Se aprecia cómo hay varias TCP que convergen en el paquete 118, que es la respuesta reensamblada totalmente.

Para ello, se configuró la herramienta Proxifier para redirigir tráfico y Burp Suite como *proxy* intermediario con el objetivo de interceptar el tráfico en tránsito.

Como se observa en la figura 6.12, la herramienta Proxifier logró identificar y capturar las peticiones del proceso notastfg.exe donde se aprecia que el tráfico hacia los *endpoints* de Amazon

(*execute-api.us-east-1.amazonaws.com*) es redirigido hacia el proxy local en 127.0.0.1:8080.

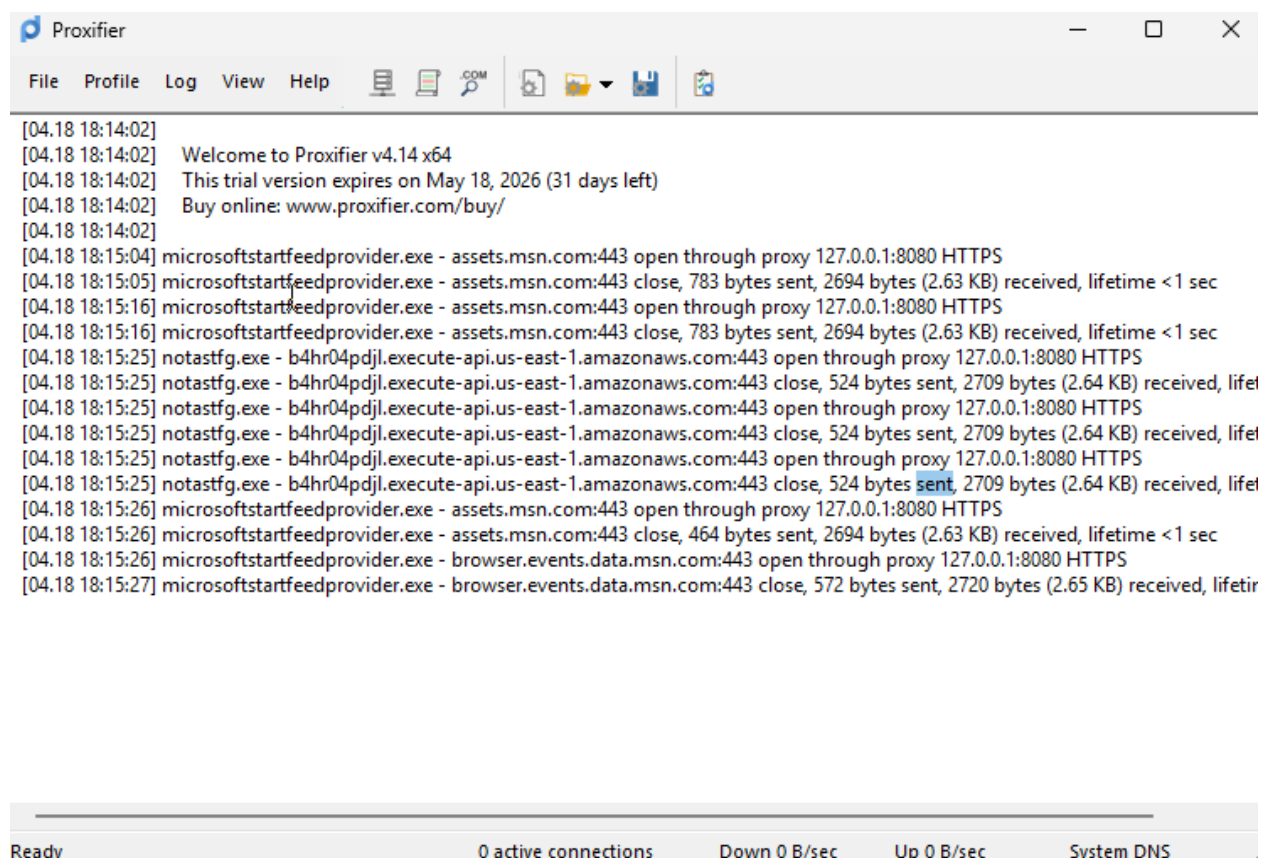


Figura 6.12: Redirección del tráfico con Proxifier.

Sin embargo, a pesar de la redirección exitosa del tráfico, el análisis del contenido se vio frustrado por los mecanismos de seguridad del propio *Loader*. Al intentar el *handshake*² TLS a través del *proxy*, se produce un error que se puede observar en la figura 6.13.

Este error indica que el *Loader* ha detectado que el certificado presentado por Burp proxy no pertenece a la autoridad certificadora legítima de Amazon. Como medida de seguridad, el código aborta la ejecución, protegiendo la integridad de los fragmentos e impidiendo que los analistas puedan interceptarlos.

Además, se ha considerado que la seguridad de la arquitectura sea bidireccional. En el supuesto de que el analista logra forzar la salida de las peticiones desde el *proxy* hacia el servidor de Amazon, la conexión sería rechazada en el extremo de la nube ya que se ha configurado de forma estricta el SNI (*Server Name Indication*) en *API Gateway*, el servidor

²El *handshake* o apretón de manos es el establecimiento de una conexión entre dos pares a través de un canal de comunicación, donde se acuerdan los parámetros de la conexión (p.e. los algoritmos de cifrado).

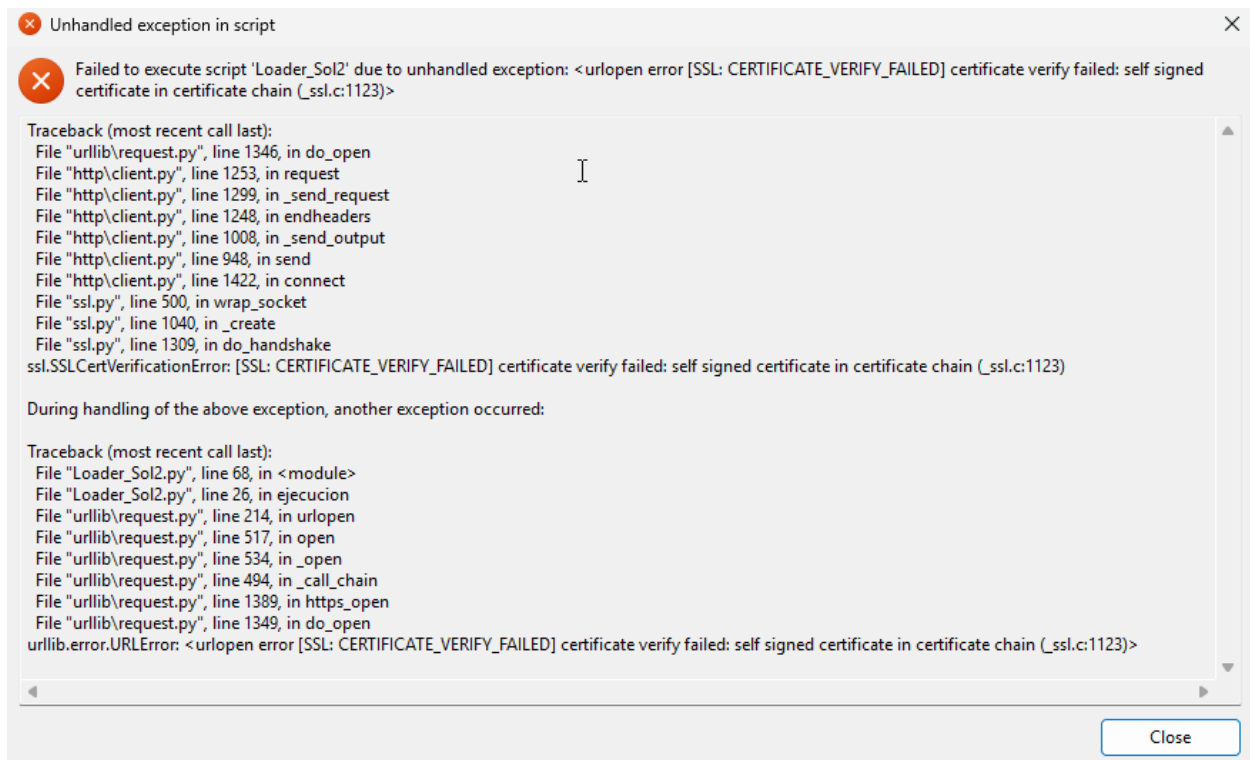


Figura 6.13: Medida de protección de la infraestructura xTyphonTC.

requiere que el nombre de dominio solicitado coincida exactamente con el certificado TLS del *endpoint*. Cualquier discrepancia introducida por el intermediario en la negociación de la capa de transporte resultaría en una denegación de servicio inmediata. Estas medidas de defensa permiten que, incluso si se compromete la confianza en el cliente, la infraestructura *serverless* actúa como una última barrera de protección, manteniendo en secreto los fragmentos del código y protegiendo la infraestructura en la nube.

6.2.3. Rendimiento y huella en memoria

Otro factor crítico a analizar es el impacto de las soluciones en el rendimiento del sistema anfitrión. Un consumo excesivo de recursos puede alertar tanto al usuario legítimo del dispositivo como a los monitores de rendimiento del sistema operativo. Por tanto, en esta fase se evaluaron dos métricas fundamentales: el tiempo de ejecución y el consumo de memoria RAM.

Tiempo de ejecución

Para medir los tiempos de ejecución de forma fiable, se diseñó un *script* de automatización basado en la librería *time* de Python que ejecutó 100

iteraciones para cada una de las soluciones.

Los resultados arrojaron una clara ventaja para la solución 1 con un tiempo medio de 2.1287 segundos y un máximo de 2.9937 segundos. Esto es debido a que esta solución descarga directamente y los reensambla en la memoria.

Por el contrario, la solución 2 registró un tiempo medio de 2.5456 segundos y un tiempo máximo de unos 6 segundos, siendo la primera ejecución. Este incremento es debido al tiempo de cómputo necesario para orquestar los fragmentos en el entorno *cloud* y el retardo producido por el *polling* que realiza el cliente. En operaciones ofensivas, esta latencia es perjudicial para el ataque, pues incrementa el tiempo de exposición del mismo.

Todo esto se puede apreciar en la tabla 6.3.

Arquitectura Evaluada	Tiempo Medio	Tiempo Mínimo	Tiempo Máximo
Solución 1	2.1287 s	2.0709 s	2.9937 s
Solución 2	2.5456 s	2.4168 s	6.0115 s

Tabla 6.3: Resumen estadístico de los tiempos de ejecución tras 100 iteraciones.

Huella en memoria

En cuanto a la huella en memoria, se realizó el mismo proceso que anteriormente, se añadió la librería de Python *psutil* y se midió el uso que hacía el proceso de la memoria durante su ejecución. El resultado fue de 25.29 MB para la primera solución y 25.22 MB para la segunda.

A pesar de que el *Loader* en la primera solución tiene muchas más tareas que en la segunda, no es tanta la diferencia de uso de RAM entre las dos soluciones, por lo que la memoria no es un factor decisivo para la elección de una u otra. Por otro lado, mantener una huella volátil de 25 MB garantiza que el *Loader* se camufle perfectamente entre los cientos de procesos legítimos en segundo plano del sistema operativo.

En conclusión, al no generar picos de CPU ni requerir grandes reservas de memoria para trabajar, la herramienta logra eludir las alertas de aquellos motores que monitoricen anomalías en el consumo de recursos.

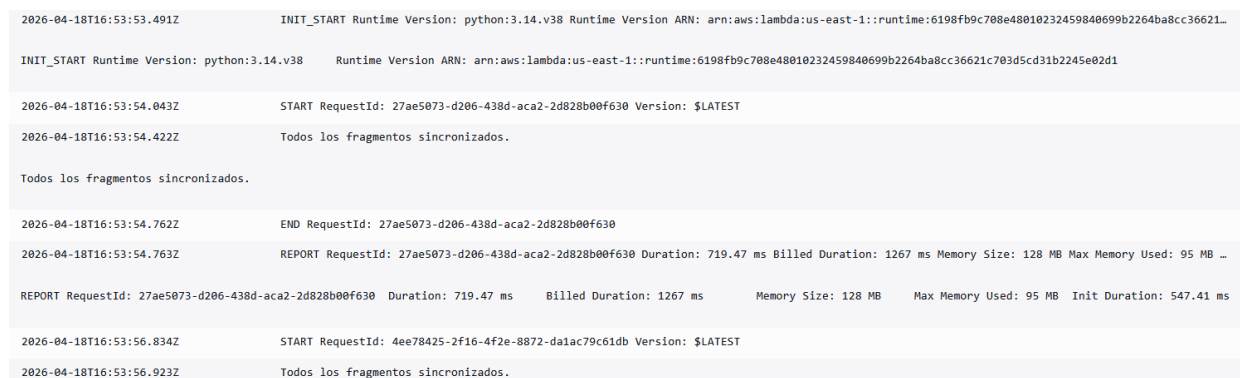
6.2.4. Huella en la nube

La última característica evaluada en esta fase de pruebas es la huella en la nube. En arquitecturas de AWS, se registra por defecto una extensa información de las interacciones a través de servicios como AWS CloudWatch. Un exceso de eventos de escritura o de ejecución en la nube incrementa el riesgo de que el equipo de defensa identifique la infraestructura maliciosa y proceda a su bloqueo.

Al evaluar la solución 1, el rastro generado en la infraestructura de AWS es prácticamente nulo. Al no requerir procesamiento en la nube durante su ejecución, esta solución no levanta sospechas ni activa alarmas, pues la interacción con el *Loader* se limita a peticiones de lectura de los fragmentos.

Por el contrario, la solución 2 genera más huella. Cada vez que un usuario ejecuta el *Loader*, se genera una serie de eventos auditables en AWS:

1. **Ejecución de código:** Se invoca a funciones Lambda, por lo que se generan registros detallados de inicio, duración y fin de ejecución en Amazon CloudWatch (como se aprecia en la imagen 6.14).



2026-04-18T16:53:53.491Z	INIT_START Runtime Version: python:3.14.v38 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:6198fb9c708e48010232459840699b2264ba8cc36621...
INIT_START Runtime Version: python:3.14.v38	Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:6198fb9c708e48010232459840699b2264ba8cc36621c703d5cd31b2245e02d1
2026-04-18T16:53:54.043Z	START RequestId: 27ae5073-d206-438d-aca2-2d828b00f630 Version: \$LATEST
2026-04-18T16:53:54.422Z	Todos los fragmentos sincronizados.
Todos los fragmentos sincronizados.	
2026-04-18T16:53:54.762Z	END RequestId: 27ae5073-d206-438d-aca2-2d828b00f630
2026-04-18T16:53:54.763Z	REPORT RequestId: 27ae5073-d206-438d-aca2-2d828b00f630 Duration: 719.47 ms Billed Duration: 1267 ms Memory Size: 128 MB Max Memory Used: 95 MB ...
REPORT RequestId: 27ae5073-d206-438d-aca2-2d828b00f630	Duration: 719.47 ms Billed Duration: 1267 ms Memory Size: 128 MB Max Memory Used: 95 MB Init Duration: 547.41 ms
2026-04-18T16:53:56.834Z	START RequestId: 4ee78425-2f16-4f2e-8872-da1ac79c61db Version: \$LATEST
2026-04-18T16:53:56.923Z	Todos los fragmentos sincronizados.

Figura 6.14: Informe Amazon CloudWatch.

2. **Escritura en S3:** La función orquestadora ensambla los fragmentos y escribe el código ensamblado completo en el *buffer* S3. Este evento genera un evento *PutObject* que se puede observar.
3. **Borrado de código:** Tras la descarga, se elimina cualquier rastro de código en el S3, generándose un evento *DeleteObject* que añade aún más información a los registros de auditoría.

Como se observa en la Figura 6.15, el uso de Amazon Athena para consultar los *Data Events* de CloudTrail permite visualizar la trazabilidad

completa de la Solución 2. En la captura se aprecia la secuencia exacta de creación y eliminación de los fragmentos y el *payload* final, lo que confirma una huella operativa elevada.

7	DeleteObject	2026-04-30T16:10:37Z	tfg-typhon-bucket	payload_final.json
8	DeleteObject	2026-04-30T16:10:37Z	tfg-typhon-bucket	fragmento2.json
9	DeleteObject	2026-04-30T16:10:37Z	tfg-typhon-bucket	fragmento3.json
10	PutObject	2026-04-30T16:10:37Z	tfg-typhon-bucket	payload_final.json
11	DeleteObject	2026-04-30T16:10:36Z	tfg-typhon-bucket	fragmento1.json
12	PutObject	2026-04-30T16:10:35Z	tfg-typhon-bucket	fragmento3.json
13	PutObject	2026-04-30T16:10:35Z	tfg-typhon-bucket	fragmento2.json
14	PutObject	2026-04-30T16:10:34Z	tfg-typhon-bucket	fragmento1.json

Figura 6.15: Registro de eventos PutObject y DeleteObject capturados en Athena.

En la tabla 6.4 se detallan los servicios de AWS implicados y los eventos específicos que componen el rastro digital de esta solución.

Servicio AWS	Evento	Fuente de Auditoría
AWS Lambda	Invoke	CloudWatch
Amazon S3	PutObject	CloudTrail
Amazon S3	DeleteObject	CloudTrail

Tabla 6.4: Resumen de los eventos observados y los servicios de auditoría utilizados en la solución 2.

Esta necesidad constante de lectura y escritura por cada infección individual hace que la solución 1 presente una clara superioridad frente a la 2 al mantener la infraestructura de entrega de *malware* en un estado más sigiloso y pasivo.

6.3. Análisis comparativo: Solución 1 versus Solución 2

Una vez finalizada la fase de experimentación y análisis individual de cada arquitectura, es menester realizar una comparativa para determinar qué enfoque cumple en mayor grado de eficacia los objetivos propuestos al inicio de este proyecto, lograr la máxima capacidad de evasión con el menor impacto posible.

Para facilitar la visualización de los resultados empíricos que se han venido detallando en este capítulo, se presenta la tabla 6.5, que condensa las métricas clave evaluadas para ambas soluciones.

Métrica	Solución 1	Solución 2
Evasión Antivirus en reposo	Sí	Sí
Evasión Antivirus en ejecución	Mayormente exitosa	Mayormente exitosa
Ejecución del <i>Payload</i>	Sí	Sí
Sigilo de Red	Peticiones atómicas (HTTPS)	Segmentación TCP (PDU Reensamblada)
Tiempo de Ejecución Medio	Menor (~ 2.13 s)	Mayor (~ 2.54 s)
Tiempo de Ejecución Máximo	Menor (~ 2.99 s)	Mayor (~ 6 s)
Consumo RAM	25.29 MB	25.22 MB
Huella en la nube	Mínima (Solo lectura)	Elevada (<i>Put/Delete Object</i>)

Tabla 6.5: Tabla comparativa final entre la arquitectura de ensamblado local y el orquestador en la nube.

Gracias a los datos expuestos, se pueden extraer las siguientes conclusiones comparativas:

- **Evasión de antivirus (Empate):** Ambas soluciones demuestran una alta eficacia al evadir la detección de antivirus comerciales como Windows Defender o Avast. La inyección en memoria del *malware* funciona excepcionalmente bien en ambos enfoques, logrando eludir los entornos defensivos de los antivirus.
- **Sigilo en Red (Solución 1):** La solución 1 presenta una clara superioridad. Al requerir únicamente la descarga atómica de los fragmentos, el tráfico se camufla como peticiones HTTPS hacia Amazon. Por el contrario, la solución 2 penaliza este sigilo debido a la necesidad de hacer peticiones repetitivas (*polling*) y la transferencia del código completo. Esto hace necesario un reensamblaje elevando el riesgo de detección.
- **Rendimiento computacional (Solución 1):** La solución 1 tiene tiempos de ejecución menores al eliminar el tiempo de cómputo en

la nube. La solución 2, en cambio, tiene el problema del *Cold Start* de las funciones *serverless* de AWS y el propio procesamiento de las funciones, lo que hace que tenga tiempos de ejecución más altos (destacando el tiempo máximo). Por otro lado, el consumo RAM tampoco difiere en gran medida en ambos, pues a pesar de que la solución 1 es ligeramente más alta, sigue sin ser suficiente como para ser realmente diferencial.

- **Huella en la nube (Solución 1):** La solución 1 es superior al mantener una huella puramente pasiva y de solo lectura. La segunda solución genera un rastro mayor al tener operaciones de lectura, escritura y eliminación.

6.3.1. Veredicto final

El análisis exhaustivo determina que la solución 1 (ensamblado local) es la arquitectura óptima y definitiva para xTyphonTC. Aunque delega la tarea de orquestación en el cliente, su impacto real en el consumo de recursos en el sistema es mínimo. Además, los beneficios obtenidos en materia de evasión de red, velocidad de despliegue y minimización de rastro en *cloud* superan ampliamente a la arquitectura basada en la delegación de la orquestación en la nube.

Por otro lado, la solución 2 se puede llegar a definir como un *downloader* clásico, pues al final el *Loader* descarga el código íntegro. Esto no es el objetivo marcado al inicio de este trabajo, pues se buscaba la descarga de fragmentos de *malware* como técnica de evasión y ofuscación.

En consecuencia, la solución 1 se consolida como un vector de ataque avanzado, sigiloso y viable y es elegida como solución final para este proyecto.

6.4. Conclusiones

La realización de este Trabajo de Fin de Grado ha permitido evaluar la viabilidad técnica de combinar fragmentación de *malware*, infraestructuras *cloud* y servicios *serverless* dentro de una prueba de concepto ofensiva controlada. El desarrollo y la auditoría de xTyphonTC muestran que el aprovechamiento de servicios legítimos en la nube puede introducir nuevas superficies de evasión y ofuscación que conviene estudiar desde una perspectiva defensiva.

Para evaluar el éxito del proyecto, resulta fundamental contrastar los resultados obtenidos durante la fase de pruebas con los objetivos planteados al inicio del documento:

- **Revisión de literatura sobre evolución del *malware* y servicios *serverless*:** Se ha cumplido este objetivo al integrar este trabajo en el marco actual tanto ofensivo como defensivo. Todo esto se explica en el capítulo 2, donde se trata la evolución del *malware* y en qué punto se halla actualmente la tecnología *serverless*.
- **Selección e integración del laboratorio:** La elección de tecnologías modernas y estandarizadas (como VirtualBox, AWS Lambda o Python) ha demostrado ser un ecosistema idóneo y realista. Esta selección de tecnologías ha permitido ejecutar las pruebas de concepto en un entorno seguro y que replica las condiciones de una red real.
- **Diseño e implementación de la arquitectura xTyphonTC:** Se ha cumplido con éxito la implementación de un sistema de ataque basado en fragmentación de *malware* en entornos *serverless*, desarrollándose además dos variantes para su posterior evaluación.
- **Evaluación de la fragmentación como técnica de evasión:** Los resultados demuestran que la fragmentación del código, combinada con la inyección dinámica en la memoria RAM y el uso de servicios *serverless* evade por completo la detección por parte de antivirus de primer nivel (como Windows Defender o Avast). Además, a nivel de red logra pasar desapercibido debido a la segmentación atómica del tráfico a través de TLS 1.3. Sin embargo, también se han observado limitaciones, como los falsos positivos asociados al uso de PyInstaller.
- **Estudio de costes operativos, disponibilidad y sigilo:** Los resultados confirman que el uso de AWS proporciona a la herramienta unos costes de mantenimiento muy reducidos. Respecto al sigilo, el uso de API Gateway actúa como un canal de comunicación encubierto perfecto, ya que el tráfico se enmascara bajo certificados legítimos de Amazon beneficiándose de la confianza que se otorga a estos dominios. No obstante, esta ventaja debe interpretarse siempre en el contexto controlado de las pruebas realizadas.

6.5. Trabajo Futuro

Aunque la herramienta xTyphonTC ha demostrado alcanzar con éxito los objetivos de evasión propuestos, el campo de la ciberseguridad

ofensiva se encuentra en constante evolución. Como resultado de este proyecto y de las pruebas realizadas, se proponen las siguientes líneas de trabajo futuro para expandir las capacidades de la plataforma:

- **Transición a lenguajes nativos o distinta conversión a ejecutable:** Tal y como se evidenció durante las pruebas con Panda Security, el uso de PyInstaller para pasar de archivo *Python* en ejecutable introduce firmas que pueden hacer que el antivirus lo neutralice. Para ello, se propone reescribir el *Loader* en algún lenguaje de más bajo nivel como C++, lo que eliminaría la dependencia de empaquetadores de terceros. Otra posible solución que se propone es explorar métodos alternativos de empaquetado.
- **Implementación de arquitecturas *multi-cloud*:** Para aumentar la resiliencia de la infraestructura de xTyphonTC frente a posibles bloqueos de cuentas por parte de Amazon, podría diseñarse un *backend* portable. Esto permitiría alternar dinámicamente entre servicios parecidos de diferentes proveedores (por ejemplo, pasar de AWS Lambda a Azure Functions o Google Cloud Functions) en caso de detectar que un canal se considerara comprometido.
- **Evolución hacia un agente bidireccional:** En su versión actual, xTyphonTC actúa como un *Loader* unidireccional (descarga código y lo ejecuta). Una mejora obvia consistiría en implementar rutinas para el envío de información desde el cliente de vuelta hacia AWS a través de peticiones HTTP POST cifradas. Esto haría que la herramienta se convirtiera en un agente de C2 (*Command and Control*) completo.
- **Sustitución de los JSON:** En lugar de transferir el *malware* en formato JSON, que a pesar de estar cifrado podría levantar sospechas, se podría ocultar el *payload* dentro de imágenes almacenadas en un *bucket* público de Amazon S3. De esta forma, el análisis de red únicamente vería la descarga de imágenes o logos que parecen inofensivas.

Chapter 6. Conclusions and future work

6.1. Conclusions

The completion of this Final Year Project has made it possible to evaluate the technical viability of combining malware fragmentation, cloud infrastructures, and serverless services within a controlled offensive proof of concept. The development and auditing of xTyphonTC suggest that the use of legitimate cloud services can introduce new opportunities for evasion and obfuscation that deserve closer defensive study.

To evaluate the success of the project, it is essential to contrast the results obtained during the testing phase with the objectives set at the beginning of the document:

- **Literature review on malware evolution and serverless services:** This objective has been met by integrating this work within the current offensive and defensive framework. This is detailed in Chapter 2, which addresses the evolution of malware and the current state of serverless technology.
- **Laboratory selection and integration:** The choice of modern and standardized technologies (such as VirtualBox, AWS Lambda, or Python) has proven to be an ideal and realistic ecosystem. This selection of technologies allowed for the execution of proofs of concept in a secure environment that replicates real network conditions.
- **Design and implementation of the xTyphonTC architecture:** The implementation of an attack system based on malware fragmentation in serverless environments was successfully achieved, with two variants developed for subsequent evaluation.
- **Evaluation of fragmentation as an evasion technique:** This has been one of the greatest successes of the project. It has been demonstrated that code fragmentation, combined with dynamic RAM injection and the use of serverless architecture, completely evades detection by top-tier antivirus software (such as Windows Security or Avast). Furthermore, at the network level, it remains undetected due to the atomic segmentation of traffic via TLS 1.3.

- **Study of operational costs, availability, and stealth:** The results confirm that using AWS as a C2 (Command and Control) provides the tool with virtually zero maintenance costs. Regarding stealth, the use of API Gateway acts as a perfect secret communication channel, as the traffic is masked under legitimate Amazon certificates, benefiting from the trust granted to these domains.

6.2. Future Work

Although the xTyphonTC tool has successfully achieved the proposed evasion objectives, the field of offensive cybersecurity is in constant evolution. As a result of this project and the tests performed, the following lines of future work are proposed to expand the platform's capabilities:

- **Transition to native languages or alternative executable conversion:** As evidenced during the tests with Panda Security, using PyInstaller to convert Python files into executables introduces signatures that can cause antivirus software to neutralize them. To resolve this, it is proposed to rewrite the Loader in a low-level language such as C++, which would eliminate dependence on third-party packagers. Another proposed solution is to explore alternative methods for executable transformation.
- **Implementation of multi-cloud architectures:** To increase the resilience of the xTyphonTC infrastructure against potential account blocks by Amazon, a highly portable backend could be designed. This would allow the malware to dynamically switch between similar services from different providers (for example, jumping from AWS Lambda to Azure Functions or Google Cloud Functions) if a channel is detected as compromised.
- **Evolution toward a bidirectional agent:** In its current version, xTyphonTC acts as a unidirectional Loader (download and execute). An obvious improvement would be to implement routines for sending information from the client back to AWS via encrypted HTTP POST requests. This would transform the tool into a complete C2 (Command and Control) agent.
- **Replacement of JSON payloads:** Instead of transferring malware in JSON format, which despite being encrypted might raise suspicions, the payload could be hidden within images stored in a public Amazon

S3 bucket. In this way, network analysis would only record the download of seemingly harmless images or logos.

Contribuciones de los autores

David Castro García

Mi contribución a este trabajo ha englobado desde el intensivo estudio de documentación y el Estado del Arte, hasta la concepción de la idea teórica y su ejecución práctica. Además, fue mi labor estructurar tanto la memoria completa, como los tiempos de trabajo, la implementación y las pruebas a realizar.

En la parte introductoria del documento (Capítulo 1) me encargué de escribir la justificación y de diseñar la estructura de la memoria (escribiendo ese apartado también). Además, traduje ese capítulo completo al inglés. Por otro lado, redacté el Resumen, el *Abstract* y las Palabras Clave.

En el Estado del Arte (Capítulo 2), realicé de manera individual dos apartados de gran importancia. Uno de ellos fue el 2.2. *Paradigmas de Detección de Malware: de Firmas a la IA*, que se enfoca en la evolución de la detección de *malware* y en qué situación se halla en la actualidad. Esto es fundamental para entender a qué se podría enfrentar la herramienta. El desarrollo de estas secciones conllevó una exhaustiva labor de búsqueda y comprensión bibliográfica, realizando varias lecturas comprensivas sobre lo que se especificaba en los documentos.

Otro bloque fundamental que hice en este capítulo fue el 2.4. *Convergencia: Malware « Nativo de Serverless» y Estrategias de Evasión Avanzadas*, donde se unen los dos conceptos claves de este proyecto mencionando las tendencias actuales y explicando el porqué de la realización de xTyphonTC. El apartado introductorio del capítulo 2 también fue realizado por mí de manera individual, mientras que el siguiente, el de tecnologías, se escribió en conjunto con mi compañero.

El diseño de la solución (Capítulo 4) fue realizado exclusivamente por mí salvo los apartados de diseño de la lógica central, escritos por Ricardo (4.1.1 y 4.2.1). Es decir, mi trabajo en este capítulo incluye la explicación general de ambas soluciones, los diagramas de flujo explicativos y los requisitos y restricciones.

A la hora de implementar la solución, debido a que Ricardo tenía la cuenta en AWS, me dediqué a hacer la parte del cliente de ambas

soluciones (*Loader*). Además, ayudé a configurar los servicios de AWS y programar las funciones Lambda, diseñar las soluciones y buscar información sobre la implementación. La estructura de este capítulo así como el contenido en cada apartado fue esquematizado por mi parte, mientras que su escritura fue realizada de forma conjunta.

La fase de pruebas se realizó en conjunto. Me encargué de investigar qué pruebas habría que hacer, basándome en la documentación de detección de *malware* mencionada en el Estado del Arte, y en conceptos aprendidos en la asignatura «Seguridad en Redes». Así, determiné que, además de comprobar la evasión de xTyphonTC frente a defensas locales, se debía estudiar su impacto y rendimiento en el sistema víctima, su huella producida en la nube (AWS) y su capacidad de evasión a nivel de red.

Asimismo, redacté la mayor parte del capítulo (salvo el 6.1.1. y las capturas de pantalla, realizados por Ricardo). En este capítulo, plasmé de manera fiel los resultados obtenidos en las pruebas que habíamos realizado, realicé la comparación de versiones y la decisión de la versión final elegida. Además expliqué las conclusiones obtenidas y propuse diferentes vías de trabajo futuro derivado de xTyphonTC. De esta forma, tuve la tarea de concluir el proyecto.

Para finalizar, me encargué de revisar el documento para buscar fallos y realizar correcciones de estilo, de redacción o de formato.

Ricardo Luque Mansilla

Mi participación en el proyecto ha abarcado desde el análisis teórico inicial hasta el despliegue de la infraestructura en la nube, el desarrollo del artefacto de pruebas y las posteriores pruebas del sistema.

En la fase inicial (Capítulo 1), definí y estructuré de manera individual los objetivos generales y específicos del trabajo, mientras que el apartado de la motivación lo redacté en colaboración con mi compañero.

Para el Estado del Arte (Capítulo 2), asumí la responsabilidad de dos bloques principales. Por un lado, el apartado 2.1. *El Panorama Evolutivo de las Amenazas del malware*, enfocado en la transición histórica desde archivos estáticos hacia las amenazas modulares modernas.

Por otro lado, redacté de forma exclusiva la sección 2.3. *La Arquitectura Serverless como Nuevo Campo de Batalla*, donde analicé cómo las tecnologías de computación efímera de los proveedores de nube pueden desviarse de su uso legítimo para distribuir componentes de código malicioso. El Capítulo 3, correspondiente a las tecnologías, se elaboró de manera conjunta.

Durante la fase de diseño (Capítulo 4), me encargué de conceptualizar y redactar la lógica central que gobierna las dos variantes de la plataforma. El desarrollo de los flujos operacionales específicos de cada solución fue el resultado de un diseño conjunto, pero la documentación de la arquitectura en la memoria fue mi responsabilidad.

En el Capítulo 5 (Desarrollo y Arquitectura), al gestionar la cuenta de AWS del proyecto, asumí la implementación, configuración y despliegue de toda la infraestructura (*AWS Lambda, API Gateway, Amazon S3 y las políticas de IAM*). La redacción de la memoria de este capítulo se realizó de forma conjunta.

De forma paralela, programé de manera individual el código en Python del *malware* de prueba para validar el sistema. Diseñé este script específicamente para realizar inyecciones en memoria en entornos Windows x64. El desarrollo incluye la carga del *shellcode* en RAM, el uso de técnicas de evasión, la fragmentación y ofuscación de nombres de las funciones nativas de la API de Windows, y un motor dinámico de código basura (*Junk Code*) para alterar las firmas heurísticas.

Para ejecutar las pruebas de manera segura y evitar riesgos en los equipos personales, configuré de forma individual un laboratorio

virtualizado mediante *Oracle VirtualBox* con Windows 11 Pro. La red se configuró en modo *Adaptador Puente* (Bridge) con una dirección IP propia pero bajo un aislamiento lógico estricto, lo que permitía la salida HTTPS hacia AWS para las peticiones del *Loader* al mismo tiempo que facilitaba la monitorización. Se usó el adaptador puente ya que mi ordenador de sobremesa estaba conectado mediante Ethernet.

En el Capítulo 6 (Resultados, Conclusiones y Trabajo Futuro), las pruebas se ejecutaron conjuntamente, pero yo me encargué de dos tareas específicas:

En primer lugar, redacté la documentación descriptiva de las capacidades operativas del *malware* de prueba y su comportamiento.

En segundo lugar, documenté los análisis y auditorías de tráfico realizados con la herramienta *Burp Suite* (Burp Proxy). Asimismo, fui el responsable de obtener las evidencias y capturas de pantalla de la huella en la nube utilizando herramientas como *AWS CloudTrail*, *Amazon CloudWatch* y la consola de consultas de *Amazon Athena*.

Finalmente, asumí íntegramente la traducción y redacción del apartado de Conclusiones y Trabajo Futuro al inglés. Además de revisar y corregir el documento.

Bibliografía

- [1] R. Tahir, "A study on malware and malware detection techniques," *International Journal of Education and Management Engineering*, vol. 8, no. 2, p. 20, 2018.
- [2] Ö. A. Aslan and R. Samet, "A comprehensive review on malware detection approaches," *IEEE access*, vol. 8, pp. 6249–6271, 2020.
- [3] A. P. Namanya, A. Cullen, I. U. Awan, and J. P. Disso, "The world of malware: An overview," in *2018 IEEE 6th international conference on future Internet of Things and cloud (FiCloud)*. IEEE, 2018, pp. 420–427.
- [4] J. Singh and J. Singh, "Challenge of malware analysis: malware obfuscation techniques," *International Journal of Information Security Science*, vol. 7, no. 3, pp. 100–110, 2018.
- [5] A. Suliman, M. K. Shankarapani, S. Mukkamala, and A. H. Sung, "Rfid malware fragmentation attacks," in *2008 International Symposium on Collaborative Technologies and Systems*. IEEE, 2008, pp. 533–539.
- [6] S. Naval, V. Laxmi, M. Rajarajan, M. S. Gaur, and M. Conti, "Employing program semantics for malware detection," *IEEE Transactions on Information Forensics and Security*, vol. 10, no. 12, pp. 2591–2604, 2015.
- [7] S. K. Cha, I. Moraru, J. Jang, J. Truelove, D. Brumley, and D. G. Andersen, "Splitscreen: Enabling efficient, distributed malware detection," *Journal of Communications and Networks*, vol. 13, no. 2, pp. 187–200, 2011.
- [8] B. S. Mitchell, A. Chandnani, J. Carter, D. Roumelioti, and S. Mancoridis, "Malware detection in cloud native environments," in *Proceedings of the 2024 7th Artificial Intelligence and Cloud Computing Conference*, ser. AICCC '24. New York, NY, USA: Association for Computing Machinery, 2025, p. 555–564. [Online]. Available: <https://doi.org/10.1145/3719384.3719465>
- [9] Y. Birman, S. Hindi, G. Katz, and A. Shabtai, "Cost-effective malware detection as a service over serverless cloud using deep reinforcement

- learning,” in *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, 2020, pp. 420–429.
- [10] S. Shrestha, “Comparing programming languages used in aws lambda for serverless architecture,” 2019.
- [11] D. K. Yadav, G. Sood, S. K. Sonbhadra, S. R. R. BR *et al.*, “Exploring the potential of serverless architecture for cloud automation,” in *2025 International Conference on Automation and Computation (AUTOCOM)*. IEEE, 2025, pp. 1070–1075.
- [12] OWASP Foundation, “Owasp serverless top 10 project,” Open Web Application Security Project, Technical Report, 2017. [Online]. Available: <https://owasp.org/www-project-serverless-top-10/>
- [13] V. Radunović and M. Veinović, “Malware command and control over social media: Towards the server-less infrastructure,” *Serbian Journal of Electrical Engineering*, vol. 17, no. 3, pp. 357–375, 2020.
- [14] N. Ponomarev, “A proof of concept for an aws lambda malicious code generator tool,” Metropolia University of Applied Sciences, Helsinki, Finland, 2022.
- [15] C. Xiang, F. Binxing, Y. Lihua, L. Xiaoyi, and Z. Tianning, “Andbot: Towards advanced mobile botnets,” in *4th USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET 11)*. Boston, MA: USENIX Association, Mar. 2011. [Online]. Available: <https://www.usenix.org/conference/leet11/andbot-towards-advanced-mobile-botnets>
- [16] I. Nisar and D. R. Tiwari, “A COMPARATIVE STUDY OF SERVERLESS COMPUTING PLATFORMS: AWS LAMBDA vs. GOOGLE CLOUD FUNCTIONS vs. AZURE FUNCTIONS,” *International Research Journal of Engineering and Technology (IRJET)*, vol. 12, no. 6, 2025. [Online]. Available: <https://www.irjet.net/archives/V12/i6/IRJET-V12I6112.pdf>
- [17] S. Kumar *et al.*, “An emerging threat fileless malware: a survey and research challenges,” *Cybersecurity*, vol. 3, no. 1, pp. 1–12, 2020.